Institut für Werkzeugmaschinen und Betriebswissenschaften (*iwb*)
TUM School of Engineering and Design
Technische Universität München

Use-Case-basierter Aufbau eines Kern-Skill-Set für Industrieroboter in der Montage

Use-case-based Development of a Core Skill Set for Industrial Robots in Assembly

Wissenschaftliche Arbeit zur Erlangung des Grades

Bachelor of Science (B.Sc.)

an der TUM School of Engineering and Design der Technischen Universität München

Geprüft von	Prof. Dr. Rüdiger Daub Lehrstuhl für Produktionstechnik und Energiespeichersysteme
Eingereicht von	David Müller Johann-Fichte-Straße 21 80805 München
Eingereicht am	29. Dezember 2025 in Garching

Institut für Werkzeugmaschinen und Betriebswissenschaften (*iwb*)
TUM School of Engineering and Design
Technische Universität München

Aufgabenstellung

Deutscher Titel der Bachelorarbeit:

Use-Case-basierter Aufbau eines Kern-Skill-Set für Industrieroboter in der Montage

English Title of the Bachelor's Thesis:

Use-case-based Development of a Core Skill Set for Industrial Robots in Assembly

Verfasst von:	David Müller
Matrikelnummer:	03769337
Studiengang:	Maschinenwesen B. Sc.
TUM School:	TUM School of Engineering and Design
Betreut von:	M. Sc. Sebastian Kurscheid
Zeitraum:	von 01.06.2025 bis 01.12.2025

Ausgangssituation

Angesichts des Fachkräftemangels ist es insbesondere in der variantenreichen Produktion erstrebenswert, der inversen Beziehung zwischen Automatisierungsgrad und Flexibilität der Produktionssysteme entgegenzuwirken. Im Bereich der Industrieroboter ist die skill-basierte Programmierung ein vielversprechender Lösungsansatz zur Flexibilisierung. Dabei werden die Fähigkeiten eines Roboters in Form von symbolischen Skills implementiert. Entwickler oder autonome Planungssysteme können dann einen Aktionsplan aus Skills erstellen. Dieser Aktionsplan legt abstrakt fest, in welcher Abfolge ein Roboter die Skills ausführen muss, um eine Aufgabe zu erfüllen. Eine Herausforderung ist es, die Skills entsprechend parametrisierbar zu implementieren, sodass diese ausreichend flexibel zur Automatisierung möglichst vieler Anwendungen neu kombiniert werden können.

Zielsetzung

Basierend auf dem Stand der Technik und beispielhaften Use Cases soll ein Kern-Skill-Set aufgebaut werden. Die Skills sollen konfigurierbar und parametrisierbar sein, um variable Anforderungen zu erfüllen. Darüber hinaus soll eine Kombination verschiedener Skills eine Durchführung

möglichst vieler Prozesse oder Prozessänderungen ermöglichen. Die Implementierung der Skills erfolgt in Form von Klassen innerhalb eines Softwareframeworks. Für die Kommunikation zwischen den verschiedenen Modulen wird ROS2 verwendet. Um die Funktionalität der entwickelten Skills zu erproben, wird ergänzend eine geeignete Simulationsumgebung erstellt. Diese ermöglicht es, die Interaktion des Roboters mit seiner Umgebung zu testen und unterschiedliche Anwendungsfälle zu erproben.

Vorgehensweise und Arbeitsmethodik

Die folgende Vorgehensweise ist geplant, um die angestrebten Ziele zu erreichen:

- Literaturrecherche und Darstellung des Standes der Wissenschaft und der Technik
- Auswahl geeigneter Montage Prozesse basierend auf beispielhaften Anwendungen
- Definition der benötigten Roboter-Skills anhand der gewählten Prozesse
- Ableitung der Anforderungen an die Skills und Definition eines Bewertungsschemas
- Aufbau der Simulationsumgebung des Roboters zur Erprobung des definierten Skill-Sets
- Implementierung und Erprobung der Skills
- Kritische Bewertung und detaillierte Dokumentation der Ergebnisse

Vereinbarung

Mit der Betreuung von David Müller durch M. Sc. Sebastian Kurscheid fließt geistiges Eigentum des *iwb* in diese Arbeit ein. Eine Veröffentlichung der Arbeit oder eine Weitergabe an Dritte bedarf der Genehmigung durch den Lehrstuhlinhaber. Der Archivierung der Arbeit in der *iwb*-eigenen und nur für *iwb*-Mitarbeiter zugänglichen Bibliothek als Bestand und in der digitalen Studienarbeitsdatenbank des *iwb* als PDF-Dokument stimme ich zu.

Garching, den 06.06.25



Prof. Dr.-Ing. Rüdiger Daub David Müller

Abstract

In an industrial environment characterized by increasing uncertainty and a growing shortage of skilled workers, the demand for automation solutions that can be adapted flexibly is rising. Industrial robots are well suited in principle, yet in practice their use is often limited by the effort required for programming and reconfiguration. This thesis investigates how typical assembly processes can be represented using skill based, parameterizable robot skills so that applications can be configured in a task oriented way with reduced effort. To this end, a pick and place skill and a dispensing skill for applying adhesive are implemented and evaluated. The skills are modeled using behavior trees and are parameterized at a higher level of abstraction with the help of a knowledge model. The results indicate potential for increased flexibility and reusability, while further validation under real process conditions is still required, along with a more consistent representation of process related parameters.

Kurzfassung

In einem Produktionsumfeld mit steigender Unsicherheit und zunehmendem Fachkräftemangel wächst der Bedarf an flexibel anpassbaren Automatisierungslösungen. Industrieroboter eignen sich dafür grundsätzlich, in der Praxis wird ihr Einsatz jedoch häufig durch hohen Programmier und Rekonfigurationsaufwand begrenzt. Diese Arbeit untersucht, wie typische Montageprozesse mithilfe skillbasierter, parametrierbarer Roboterskills so abgebildet werden können, dass Anwendungen aufgabenorientiert und mit geringerem Aufwand konfigurierbar sind. Dafür werden ein Pick-and-Place-Skill sowie ein Dosier-Skill zum Auftragen von Kleber implementiert und erprobt. Die Skills sind über Verhaltensbäume modelliert und werden mithilfe eines Wissensmodells abstrahiert parametriert. Die Ergebnisse zeigen Potenzial für höhere Flexibilität und Wiederverwendbarkeit, zugleich besteht weiterer Bedarf an Validierung unter realen Prozessbedingungen sowie an einer konsistenteren Abbildung prozessbezogener Parameter.

Inhaltsverzeichnis

Abbildungsverzeichnis **ix**

Abkürzungsverzeichnis **xi**

1 Einleitung und Motivation **1**

2 Grundlagen **3**
2.1 Montage in der Industriellen Produktion . . . 3
2.2 Industrieroboter . . . 3
2.2.1 Definition von Industrierobotern . . . 3
2.2.2 Bauformen und Charakteristika . . . 3
2.2.3 Einsatz in der Produktion . . . 5
2.3 Programmierverfahren für Industrieroboter . . . 5
2.3.1 Manuelle Programmierung . . . 5
2.3.2 Teach-In-Verfahren . . . 6
2.3.3 Programmierung durch Training . . . 6
2.3.4 Textbasierte Programmierverfahren . . . 6
2.4 Robotersteuerung . . . 7
2.4.1 Grundlegende Architekturansätze . . . 7
2.4.2 Verhaltensmodellierung mit Verhaltensbäumen . . . 8
2.4.3 Robot Operating System . . . 8
2.4.4 MoveIt 2 . . . 9
2.5 Intelligente Robotersysteme . . . 10
2.5.1 Wissensrepräsentationen . . . 10
2.5.2 Aufgabenplanung . . . 10

3 Stand der Wissenschaft und Technik **13**
3.1 Skill Modelle in der Fertigung . . . 13
3.1.1 Offline Modelle . . . 13
3.1.2 Hybride Modelle . . . 14
3.1.3 Online Modelle im Bereich der Industrieroboter . . . 15
3.2 Skill-Parameter . . . 16
3.3 Skill-basierte Frameworks . . . 17
3.3.1 REpac Framework . . . 17
3.3.2 SkiRos2 . . . 19
3.3.3 Weitere Frameworks und kommerzielle Lösungen . . . 22

4 Methodik und Konzeption **25**
4.1 Ausgangssituation und Rahmenbedingungen . . . 25
4.2 Analyse des Stands der Technik . . . 25
4.2.1 Software Framework . . . 26

4.2.2 Ressourcen und Primitive Skills . . . 27
4.3 Zieldefinition und Anforderungsanalyse . . . 28
4.3.1 Zweck und Zielsetzung . . . 28
4.3.2 Stakeholder . . . 29
4.3.3 Anforderungen . . . 30
4.3.4 Randbedingungen . . . 32
4.4 Konzeption . . . 33
4.4.1 Pick-and-Place . . . 33
4.4.2 Füge-Skill nach *DIN 8593-0* (2003) . . . 37
4.4.3 Ableitung eines Wissensmodell-Konzepts . . . 39

5 Implementierung **41**
5.1 Einführung eines Wissensmodell . . . 41
5.1.1 Umgebungsmodell . . . 41
5.1.2 Node . . . 42
5.1.3 Schnittstelle . . . 43
5.2 Erweiterungen am skillbasiertem System . . . 44
5.2.1 Implementierung einer Basisklasse für Composite Skills . . . 44
5.2.2 Primitive . . . 45
5.3 Skill Implementierung . . . 46
5.3.1 Generischer Skill-Ablauf . . . 46
5.3.2 Basis-Handhabungsklasse . . . 47
5.3.3 Pick-and-Place-Skill . . . 48
5.3.4 Dosier-Skill . . . 54

6 Evaluation und Diskussion **59**
6.1 Evaluationskonzept . . . 59
6.2 Szenarien und Testfälle . . . 59
6.2.1 Szenario 1 – Kleben eines Kameragehäuses . . . 60
6.2.2 Szenario 2 – Montage von Batteriezellen . . . 61
6.2.3 Integrationstests . . . 62
6.3 Evaluation der Anforderungen . . . 63
6.3.1 Evaluierung der funktionalen Anforderungen . . . 64
6.3.2 Evaluierung der nicht-funktionalen Anforderungen . . . 65
6.4 Diskussion der Ziele . . . 67
6.4.1 Z-1– Montageoperationen nach Norm . . . 67
6.4.2 Z-2– Aufgabenorientierte Programmierung . . . 67
6.4.3 Z-3– Flexibilität und Wiederverwendbarkeit . . . 68
6.4.4 Z-4– Integration in die Softwarearchitektur . . . 69
6.5 Limitationen und Ausblick . . . 69
6.5.1 Pick-and-Place-Skill . . . 69
6.5.2 Dosier-Skill . . . 70
6.5.3 Architektur . . . 70

7 Fazit **71**

A Anhangname **73**

Literaturverzeichnis **75**

Abbildungsverzeichnis

3.1 Konzeptuelles Modell eines Skills . . . 15
3.2 Die drei Ebenen der Skill-Architektur . . . 16
3.3 Module des REpac-Frameworks . . . 18
3.4 Enter Caption . . . 21
3.5 ABB Wizard Easy Programming . . . 23
3.6 JAKA Zu Programmieroberfläche . . . 24

4.1 Zustände eines Skills, basierend auf Heuss et al. (2022b) . . . 27
4.2 Interface des Skill Action Servers . . . 27
4.3 Greifprozess mit nicht parallelen Zugangs und Hubvektoren . . . 36

5.1 Abstrahieter Ausschnitt des verwendeten Umgebungsmodells . . . 43
5.2 Generischer Parametrierungsablauf im Startzustand . . . 47
5.3 Verhaltensbaum des Pick-Skills . . . 49
5.4 Abstrahierte Darstellung der Annäherungssequenz . . . 51
5.5 Verhaltensbaum des Place-Skills . . . 52
5.6 Verhaltensbaum des Pick-and-Place-Skills . . . 53
5.7 Frame-Wiederholungslogik beim Pick-and-Place-Skill . . . 54
5.8 Verhaltensbaum des Dosier-Skills . . . 57

6.1 Szenario 1 dargestellt in RViz . . . 60
6.2 Szenario 2 dargestellt in RViz . . . 61

Abkürzungsverzeichnis

DLs	Description Logics
MaRCO	Manufacturing Resource Capability
MTM	Methods Time Measurement
OMPL	Open Motion Planning Library
PTP	Point-To-Point (dt.: Punkt-Zu-Punkt)
ROS	Robot Operating System
SMPA	Sense Model Plan Act

Kapitel 1 Einleitung und Motivation

Die industrielle Produktion befindet sich zunehmend in einem Umfeld, welches von Disruptionen und Unsicherheiten geprägt ist. Der Fachkräftemangel verschärft diese Situation zusätzlich, weil geeignetes Personal für Betrieb, Anpassung und Weiterentwicklung von Produktionssystemen nicht immer in ausreichendem Maß verfügbar ist. Gleichzeitig führen instabilere Lieferketten und wechselnde Rahmenbedingungen dazu, dass die Nachfrage in Richtung der Fertigung stark schwank hinsichtlich Volumen und Variation. Daraus ergibt sich die Notwendigkeit, Produktionssysteme so auszulegen, dass Änderungen der Anforderungen mit möglichst geringem Aufwand umgesetzt werden können. Ein zentraler Hebel dafür ist die Steigerung der Flexibilität von Produktionsmaschinen. Flexibilität kann dabei als die Fähigkeit verstanden werden, verschiedene Operationen auszuführen oder unterschiedliche Produkte herzustellen, ohne lange Stillstandszeiten oder umfangreiche Umrüst- und Rekonfigurationsschritte in Kauf nehmen zu müssen. (Romero et al., 2024)

Vor diesem Hintergrund rücken Industrieroboter als besonders relevantes Werkezug in den Fokus, weil sie in variantenreichen Produktionsumgebungen einen schnellen Wechsel zwischen Aufgaben erlauben. Ihre Umrüstung erfolgt in vielen Fällen weniger über mechanische Umbauten, sondern lässt sich in hohem Maße durch das Leitsystem und eine softwareseitige Konfiguration anstoßen. Dadurch werden Roboter zu einem wesentlichen Befähiger, um flexibel auf Varianten und Änderungen in der Produktion reagieren zu können. (Pott & Dietz, 2019)

Diese Bedeutung zeigt sich auch in der Marktentwicklung. Berichte zur globalen Robotik- und Fertigungslandschaft zeigen, dass die Nachfrage nach Industrierobotern in den vergangenen Jahren zugenommen hat und Prognosen von einem noch weiteren Wachstum ausgehen (Bill et al., 2024). Gleichzeitig bleibt der Einsatz von Industrierobotern in der Praxis für viele Unternehmen mit Hürden verbunden. Insbesondere Bedienbarkeit und Programmieraufwand werden als Hemmnisse sichtbar, zumal häufig auch das dafür notwendige qualifizierte Personal fehlt. (Pott & Dietz, 2019) Damit entsteht eine Spannung zwischen dem Wunsch nach flexibler Automatisierung und dem Aufwand, der zur Umsetzung und Anpassung roboterbasierter Prozesse erforderlich ist.

Ein vielversprechender Ansatz, diese Lücke zu adressieren, ist die skillbasierte Programmierung. Dabei werden Fähigkeiten des Roboters auf einem höheren Abstraktionsniveau gekapselt und als wiederverwendbare Bausteine bereitgestellt. Statt Aufgaben überwiegend über ein low-level Programmierung zu beschreiben, wird eine aufgabenorientierte Sicht unterstützt, in der Anwender über parametrierte Skills arbeiten. Das kann die Programmierung und Rekonfiguration vereinfachen und die Wiederverwendbarkeit von Roboterfunktionen erhöhen. (Pedersen et al., 2016)

Trotzdem bestehen weiterhin offene Fragen, insbesondere wie Skills geeignet beschrieben und so mit ressourcenspezifischen Parametern verknüpft werden können, dass sie in unterschiedlichen Anlagenkonfigurationen zuverlässig einsetzbar bleiben. Ebenso ist ein systematisches Abgleichen zwischen allgemein formulierten Prozessanforderungen und den konkret von Ressourcen angebotenen Skills eine zentrale Voraussetzung, um die gewünschte Wandelbarkeit tatsächlich zu erreichen. (Malakuti et al., 2018)

Diese Arbeit untersucht, wie Roboter-Skills für typische Montageprozesse flexibel und wiederver-

wendbar gestaltet werden können, sodass sich Anwendungen aufgabenorientiert und mit geringem Programmieraufwand konfigurieren lassen. Im Mittelpunkt steht die Frage, wie gängige Montageoperationen auf Ebene des Bewegungsablaufs als parametrierbare Bausteine beschrieben werden können, welche Parameter dafür minimal notwendig sind und wie sich daraus auch komplexeres Verhalten durch die Kombination mehrerer Skills aufbauen lässt.

In den Kapiteln 2 und 3 werden die Grundlagen sowie der Stand von Wissenschaft und Technik des Themengebiets dargelegt. Anschließend wird das V-Modell als methodischer Rahmen der Arbeit herangezogen. In Kapitel 4 werden zunächst die Ziele der Arbeit definiert. Daraus werden Anforderungen abgeleitet und ein Konzept entwickelt. Kapitel 5 präsentiert das implementierte System. In Kapitel 6 werden die in Kapitel 4 formulierten Anforderungen durch Systemtests verifiziert. Auf Grundlage der Ergebnisse dieser Verifikation wird in einer Diskussion die Zielerreichung bewertet. Den Abschluss der Arbeit bildet Kapitel 7.

Kapitel 2 Grundlagen

2.1 Montage in der Industriellen Produktion

VDI-Richtlinie 2860 (1990) definiert Montieren als die Gesamtheit aller Vorgänge, die dem Zusammenbau von geometrisch bestimmten Körpern dienen. Nach Lotter und Wiendahl (2012) umfasst Montieren im Allgemeinen Vorgänge des Fügens, wie in *DIN 8593-0* (2003) spezifiziert, sowie Funktionen der Werkstückhandhabung, wie in der *VDI-Richtlinie 2860* (1990) beschrieben. Zusätzlich kommen noch Tätigkeiten wie Kontrollieren (Prüfen oder Messen), Justieren und Sonderoperationen, wie zum Beispiel Kühlen, Abdichten oder Reinigen, hinzu.

Um den wirtschaftlichen Wirkungsgrad von Füge- und Handhabungstätigkeiten zu beurteilen, wird nach *VDI-Richtlinie 2860* (1990) in der Praxis eine Primär-Sekundär-Analyse durchgeführt. Primärvorgänge (PV) umfassen alle Aufwendungen an Zeit, Energie, Informationen und Objekten, die während der Montage zur Wertschöpfung eines Produkts erforderlich sind. Beispiele hierfür sind das Greifen, Einlegen oder Einschrauben von Teilen zur Vervollständigung des Produkts. Sekundärvorgänge (SV) sind alle notwendigen Aufwendungen an Zeit, Energie und Informationen, die aufgrund des gewählten Montageprinzips anfallen, ohne dass dadurch eine Wertschöpfung des Produktes erfolgt. Beispiele hierfür sind Weitertransportieren, Wenden, Ablegen oder Neugreifen von Teilen. Zur Analyse und Gestaltung menschlicher Arbeitsabläufe in der Montagepraxis ist das *Methods Time Measurement* (Methods Time Measurement (MTM)) ein gängiges Verfahren. Nach Lotter und Wiendahl (2012) können mit der Analyse in der Kleingeräte Montage die fünf Grundbewegungen Hinlangen, Greifen, Bringen, Fügen und Loslassen identifiziert werden.

2.2 Industrieroboter

2.2.1 Definition von Industrierobotern

Nach Weber (2019) können Industrieroboter als Handhabungsgeräte aufgefasst werden. *ISO 8373* (2021) definiert Industrieroboter als automatisch gesteuerte, umprogrammierbare Mehrzweck-Manipulatoren. Sie sind programmierbar in drei oder mehr Achsen, die entweder ortsfest oder auf einer mobilen Plattform befestigt sein können. Sie werden in Automatisierungsanwendungen in einer industriellen Umgebung genutzt.

2.2.2 Bauformen und Charakteristika

Pott und Dietz (2019) definieren verschiedene Bauformen und Eigenschaften von Industrierobotern. Industrieroboter lassen sich anhand mehrerer Eigenschaften charakterisieren.

- **Freiheitsgrade** beschreiben, wie viele unabhängige Bewegungen ein Roboter ausführen kann. Sie bestimmen maßgeblich, welche Positionen und Orientierungen der Endeffektor erreichen kann.
- **Arbeitsraum** ist der Bereich, in dem der Roboter den Endeffektor bewegen kann. Seine Größe wird vor allem durch die Kinematik und die Abmessungen des Roboters beeinflusst.
- **Traglast** gibt an, welche maximale Masse der Roboter am Endeffektor sicher handhaben kann. Sie hängt von der mechanischen Auslegung und der Größe des Roboters ab.
- **Genauigkeit** beschreibt die Abweichung zwischen Soll- und Ist-Position. Dabei wird zwischen Absolutgenauigkeit als mittlerer Abweichung und Wiederholgenauigkeit als Streuung bei wiederholtem Anfahren unterschieden.
- **Bahngenauigkeit** beschreibt, wie genau der Roboter einer vorgegebenen Trajektorie folgt. Sie ist besonders relevant, wenn der Prozess nicht nur einzelne Punkte, sondern eine kontinuierliche Bewegung erfordert.
- **Steifigkeit** beschreibt, wie stark sich Roboterstruktur und Gelenke unter Last verformen. Sie wird vor allem durch die Nachgiebigkeit der Gelenke und die Biegung von Armen oder Führungsstrukturen bestimmt.
- **Bewegungsdynamik** ergibt sich aus Geschwindigkeit und Beschleunigung des Roboters. Sie beeinflusst die Prozesszeit und damit die Produktivität einer Anlage.

Pott und Dietz (2019) zählt außerdem die wichtigsten Bauformen der Industrieroboter auf:

- **Knickarmroboter** werden mit etwa 67% am häufigsten eingesetzt. Sie sind dem menschlichen Arm nachempfunden und besitzen meist sechs Drehgelenke in einer seriellen kinematischen Kette, an deren Ende ein Handgelenk mit Flansch zur Aufnahme des Endeffektors sitzt.
- **SCARA-Roboter** besitzen typischerweise drei bis vier Freiheitsgrade und sind in ihren Orientierungsänderungen eingeschränkt. Meist erlauben zwei in Reihe angeordnete Drehgelenke Bewegungen in der Horizontalen, während eine lineare Achse vertikale Bewegungen übernimmt und ein optionales viertes Gelenk eine Rotation um die Vertikalachse ermöglicht.
- **Portalroboter** verfügen in der Grundausführung häufig über drei translatorische Freiheitsgrade und sind kinematisch mit Portalfräsmaschinen vergleichbar. Sie können bei Bedarf um ein Handgelenk ergänzt werden, wenn zusätzliche Orientierungen des Endeffektors erforderlich sind.
- **Parallelroboter** besitzen parallelgeschaltete kinematische Ketten, sodass sich Gelenkungenauigkeiten weniger stark aufsummieren als bei seriellen Strukturen. Dadurch erreichen sie häufig eine hohe Steifigkeit und Präzision.

Der Endeffektor ist die Komponente, die den Roboter zur Ausführung einer konkreten Aufgabe befähigt. Endeffektoren werden in Greifer, Werkzeuge sowie Mess- und Prüfmittel unterteilt. Greifer werden häufig für Handhabungsaufgaben eingesetzt und können Objekte mechanisch, pneumatisch oder magnetisch aufnehmen. Werkzeuge umfassen beispielsweise Schweiß-, Bohr-, Niet- oder spanende Werkzeuge. Mess- und Prüfmittel sind etwa Kameras, Laserscanner oder Ultraschallsensoren und werden oft zur Inspektion von Bauteilen verwendet. Endeffektoren besitzen einen Tool Center Point (TCP) als zentralen Bezugspunkt für die Programmierung. Bewegungen werden durch

Änderungen der TCP-Koordinaten beschrieben. Bei einigen Robotern, insbesondere Knickarmrobotern, lässt sich neben der Position auch die Orientierung verändern. Position und Orientierung zusammen ergeben die Pose des Endeffektors. (Pott & Dietz, 2019)

2.2.3 Einsatz in der Produktion

Die Besonderheit von Industrierobotern im Vergleich zu anderen Produktionsmaschinen liegt in der frei programmierbaren Beweglichkeit. Diese Eigenschaft ermöglicht einen hohen Grad an Flexibilität, da durch einfaches Umschalten des Programms zwischen verschiedenen Prozessen oder Bauteilen gewechselt werden kann. Der Einsatz von Sensoren ermöglicht es zudem, das Verhalten eines Roboters an Variationen der Umgebung in Echtzeit anzupassen (Pott & Dietz, 2019).

Nach dem Report der *International Federation of Robotics* sind die weltweit am häufigsten vorkommenden Anwendungsfälle für Industrieroboter in absteigender Reihenfolge das Handhaben, das Schweißen und die Montage (Bill et al., 2024).

Für den Einsatz in der Montage identifizieren Lotter und Wiendahl (2012) mithilfe des MTM-Verfahrens die gleichen grundlegenden Bewegungen Hinlangen, Greifen, Bringen, Fügen und Loslassen. Die Grundbewegungen sind dabei weiter unterteilt in roboterspezifische Zeitanteile wie Schalt-, Beschleunigungs- und Verzögerungszeiten sowie in beeinflussbare Zeitanteile wie Fahrzeit bei konstanter Geschwindigkeit, Ausschwingzeit, Greifzeit und Montagezeit.

2.3 Programmierverfahren für Industrieroboter

In der Programmierung industrieller Robotersysteme wird nach Haun (2013) zwischen Online- und Offlineprogrammierung unterschieden. Bei der Onlineprogrammierung wird der Roboter im laufenden Betrieb programmiert, was zu Produktionsunterbrechungen führen kann. Die Offlineprogrammierung erfolgt dagegen ohne direkten Zugriff auf den realen Roboter und ermöglicht die Erstellung und Prüfung von Programmen in einer separaten, z.B. virtuellen Umgebung (Haun, 2013).

Darüber hinaus lassen sich Programmierverfahren nach ihrem grundlegenden Ansatz unterscheiden. Zu den gängigen Verfahren zählen die manuelle Programmierung, Teach-In-Verfahren, Programmierung durch Training sowie textbasierte Verfahren, die in roboterorientierte und aufgabenorientierte Programmierung unterteilt werden können (Haun, 2013).

2.3.1 Manuelle Programmierung

Die manuelle Programmierung ist eine einfache Form der Onlineprogrammierung. Bewegungen werden durch mechanische Anschläge in Achskoordinaten begrenzt und im Punkt-zu-Punkt-Modus ohne Rechnerunterstützung eingerichtet. Änderungen erfordern mechanische Eingriffe, wodurch das Verfahren wenig flexibel ist. Typische Einsatzgebiete sind einfache Einlegevorgänge an Bearbeitungsmaschinen (Haun, 2013).

2.3.2 Teach-In-Verfahren

Teach-In-Verfahren sind in der Praxis weit verbreitet. Sie können direkt erfolgen, indem der Roboter manuell entlang der gewünschten Bahn geführt wird. Alternativ werden Positionen indirekt über ein Handbediengerät angefahren, gespeichert und anschließend als Programm abgearbeitet. Bei komplexeren Anwendungen steigt die Anzahl der benötigten Punkte deutlich, wodurch der Einrichtungsaufwand zunimmt. Da Teach-In häufig online durchgeführt wird, kann dies Stillstandszeiten und damit verbundene Kosten verursachen (Haun, 2013).

2.3.3 Programmierung durch Training

Bei der Programmierung durch Training wird dem Roboter eine Aktion vorgeführt, die er mithilfe von Sensoren erfasst und anschließend reproduziert. Die Ausführung wird iterativ verbessert, bis geforderte Qualitätskriterien wie Genauigkeit oder Geschwindigkeit erreicht sind. Externe Sensoren können Abweichungen messen, die zur Anpassung und Optimierung der Bewegung genutzt werden (Haun, 2013).

2.3.4 Textbasierte Programmierverfahren

Textbasierte Verfahren nutzen formale Beschreibungen von Bewegungen oder Aufgaben. Sie werden häufig offline vorbereitet und anschließend auf den Roboter übertragen. Innerhalb dieser Gruppe werden roboterorientierte und aufgabenorientierte Programmierung unterschieden (Haun, 2013).

Roboterorientierte Programmierung

Die roboterorientierte Programmierung nutzt herstellerspezifische Programmiersprachen mit expliziten Bewegungsbefehlen, etwa für lineare oder punktweise Fahrten. Ein Entwickler formuliert daraus das gewünschte Bewegungsprogramm und legt notwendige Parameter fest, beispielsweise Zielpunkte als Raumkoordinaten, die definiert oder eingelernt werden müssen. Häufig kann das Programm in einer Simulationsumgebung getestet werden, bevor es auf den Roboter übertragen wird. Mit zunehmender Aufgabenkomplexität wächst jedoch der Umfang solcher Programme, wodurch die Übersichtlichkeit sinkt und Änderungen aufwendiger werden. Zudem sind diese Programme für Anwender ohne vertiefte Expertise oft schwer nachvollziehbar (Haun, 2013).

Aufgabenorientierte Programmierung

Die aufgabenorientierte Programmierung setzt an diesen Einschränkungen an. Statt eine Abfolge von Steuerungsbefehlen vorzugeben, wird eine Aufgabe als Zielzustand der Umgebung beschrieben, etwa dass ein Objekt auf einer Palette liegt. Ausgehend vom aktuellen Zustand bestimmt das System anschließend, welche Handlungen erforderlich sind, um den Zielzustand zu erreichen. Dadurch können auch Anwender ohne tiefgehende Programmierkenntnisse Roboterprogramme erstellen und anpassen (Haun, 2013).

2.4 Robotersteuerung

In diesem Abschnitt wird gezeigt, wie sich grundlegende Konzepte der Robotersteuerung in Softwarestrukturen abbilden lassen. Zunächst werden Steuerungsarchitekturen als konzeptioneller Rahmen beschrieben. Anschließend wird mit Verhaltensbäumen eine Struktur eingeführt, die reaktive Ausführung und die Zerlegung komplexer Aufgaben unterstützt. Abschließend werden ROS 2 als Infrastruktur für modulare Systeme und MoveIt 2 als Werkzeug zur Bewegungsplanung eingeordnet.

2.4.1 Grundlegende Architekturansätze

In der Robotik umfasst der Begriff Roboter Architektur nach Siciliano und Khatib (2016) zwei unterschiedliche Aspekte: die architektonische Struktur und den architektonischen Stil. Die Architektonische Struktur befasst sich mit dem Aufbau und Zusammenhang der Module. Der Architektonische Stil hingegen beschreibt, wie diese Struktur in Software umgesetzt wird. Nach Hertzberg et al. (2012) gibt es im Kontext der Softwarestruktur grundlegend zwei verschiedene Möglichkeiten Module zu strukturieren: sequenziell oder parallel. Eine sequenzieller Ablauf bedeutet demnach, dass in einem Kontrollzyklus des Roboters Module nacheinander abgearbeitet werden. Eine typische nach diesem Prinzip arbeitende Architektur ist die Sense Model Plan Act (SMPA)-Architektur. In jedem Zyklus werden die Sensor Daten aufgenommen und basierend darauf ein Modell der Umgebung gebaut. Mit Informationen aus diesem Modell wird die nächste Aktion geplant und anschließend ausgeführt. Verhaltensbasierte Architekturen hingegen besitzen eine parallele Struktur. Module, die Verhaltenselemente des Roboters abbilden, analysieren in einem Zyklus parallel die Sensordaten und berechnen daraus ihre jeweiligen Kommandos. Aus diesen Kommandos wird anschließend ein einzelnes Kontrollkommando errechnet. Module können auch interne Zustände haben und müssen dadurch nicht in jedem Zyklus ein Kommando liefern. Verhaltensbasierte Architekturen sind im Vergleich zur SMPA-Architektur reaktiver. SMPA-Architekturen eignen sich dafür besser für die Ausführung langfristiger Ziele, die durch eine mehr symbolisierten Interpretation der Umgebung erreicht werden. Weiterhin gibt es Hybride Architekturen, die die Stärken sequenzieller und verhaltensbasierter Roboterkontrollsysteme kombinieren. Reaktive, feingranulare Aufgaben wie Reflexe werden durch verhaltensbasierte Komponenten ausgeführt, während symbolische, langfristig orientierte Aufgaben wie Planung und Umgebungsinterpretation von einer SMPA-ähnlichen Struktur übernommen werden. Typisch ist eine Drei-Schichten-Architektur. Eine strategische Ebene plant auf hoher Abstraktionsebene, eine taktische Ebene wählt konkrete Aktionen aus und bereitet sie auf, und eine operative Ebene überwacht die Ausführung reaktiv. Besonders herausfordernd ist dabei die mittlere Ebene der Aktionsauswahl, da sie zwischen Planung und Reaktion vermitteln muss. Trotz offener Forschungsfragen gelten hybride Architekturen als angemessene Struktur für moderne Roboterkontrollsysteme.(Hertzberg et al., 2012)

Nach Siciliano und Khatib (2016) erhöht eine gut durchdachte und saubere Roboter Architektur die Verständlichkeit, Wiederverwendbarkeit und Testbarkeit und schafft gleichzeitig sinnvolle Rahmenbedingungen für die weitere Systementwicklung. In der Praxis ist die Architektur vieler Robotersysteme schwer eindeutig zuzuordnen, da häufig mehrere Stile kombiniert werden und klare Abgrenzungen zwischen Subsystemen fehlen. Zudem sind Architektur und umgebungsabhängige Implementierung oft eng miteinander verflochten.

2.4.2 Verhaltensmodellierung mit Verhaltensbäumen

Die bisherigen Ausführungen beschreiben die Organisation von Modulen und den Kontrollfluss im Robotersystem. Darauf aufbauend wird im Folgenden eine Modellierungsform vorgestellt, mit der das ausführbare Verhalten als strukturierte Logik beschrieben werden kann, hierfür werden Verhaltensbäume eingeführt.

Ein Verhaltensbaum (engl. Behaviour tree) ist ein Baum aus Knoten, der das Verhalten eines Systems als hierarchische Entscheidung und Ausführung modelliert. Der Wurzelknoten steht für das Gesamtverhalten, darunter werden Teilaufgaben in kleinere Schritte zerlegt. Typische Knotenklassen sind Kompositknoten, Dekoratorknoten und Blattknoten. Kompositknoten fassen mehrere Kinder zusammen und bestimmen, in welcher Reihenfolge diese ausgewertet werden, häufig als Sequenz oder Auswahl. Eine Sequenz verarbeitet Kinder nacheinander und liefert Fehlschlag, sobald ein Kind fehlschlägt. Eine Auswahl verarbeitet Kinder nacheinander und liefert Erfolg, sobald ein Kind erfolgreich ist. Dekoratorknoten besitzen genau ein Kind und verändern dessen Ablauf oder Ergebnis, etwa durch Wiederholung oder Abbruchbedingungen. Blattknoten bilden die Schnittstelle zur Anwendung, sie führen Aktionen aus oder prüfen Bedingungen. (Jones, 2020)

Die Ausführung erfolgt über das Prinzip des Tickens. In festen oder ereignisgetriebenen Abständen wird ein Tick an der Wurzel ausgelöst und rekursiv an relevante Kindknoten weitergegeben. Jeder Knoten liefert dabei einen Status zurück, üblich sind Erfolg, Fehlschlag und Laufend. Laufend bedeutet, dass der Knoten begonnen hat, aber noch nicht abgeschlossen ist, wodurch der Tick in späteren Zyklen erneut an diesen Teilbaum gegeben wird. Dadurch entsteht ein gut nachvollziehbarer Kontrollfluss, der reaktiv auf Zustandsänderungen reagieren kann, ohne dass ein monolithischer Zustandsautomat notwendig ist. (Jones, 2020)

Verhaltensbäume besitzen mehrere vorteilhafte Eigenschaften. Sie sind modular, da jeder Teilbaum wieder ein vollständiger Verhaltensbaum ist und umgekehrt ein vollständiger Verhaltensbaum als Teilbaum in größere Strukturen eingebettet werden kann. Dadurch lassen sich nützliche Verhaltensbausteine kapseln und wiederverwenden. Sie sind erweiterbar, da mit wachsender Anzahl an Knoten typischerweise keine kombinatorische Explosion entsteht, wie sie bei Zustandsautomaten durch stark ansteigende Übergangszahlen auftreten kann. Außerdem sind Verhaltensbäume prinzipiell gut lesbar, was Analyse und Nachvollziehbarkeit unterstützt, auch wenn Verhaltensstrukturen automatisch erzeugt oder optimiert wurden. (Jones, 2020)

Für die praktische Implementierung in Python kann die Bibliothek `py_trees` verwendet werden. Sie stellt zentrale Knotenarten und die Ausführungslogik bereit und bildet den oben beschriebenen Tick Mechanismus direkt ab. In `py_trees` wird jeder Knoten als Klasse modelliert, typischerweise als Spezialisierung von `Behaviour`. Der Lebenszyklus eines Knotens kann abstrakt über drei Funktionsblöcke beschrieben werden, eine Initialisierungsfunktion, eine Updatefunktion und eine Finalisierungsfunktion. Die Initialisierungsfunktion wird aufgerufen, wenn ein Knoten erstmals aktiv wird, also beim Übergang in den Status Laufend. Dort werden interne Variablen zurückgesetzt und benötigte Ressourcen vorbereitet. Die Updatefunktion wird bei jedem Tick ausgeführt, solange der Knoten aktiv ist. Sie berechnet den aktuellen Status und gibt ihn zurück. Die Finalisierungsfunktion wird aufgerufen, wenn der Knoten den aktiven Zustand verlässt, also wenn er nicht länger Laufend ist oder durch übergeordnete Logik abgebrochen wird. (Stonier, n. d.)

2.4.3 Robot Operating System

Die Basis vieler Robotik-Softwarearchitekturen bildet das Robot Operating System (Robot Operating System (ROS)). Im Folgenden wird speziell auf die Weiterentwicklung ROS 2, wie in Macenski

et al. (2022) beschrieben eingegangen. ROS 2 ist ein Open-Source-Software-Development-Kit für die Robotik, das eine Vielzahl in diesem Bereich häufig verwendeter Algorithmen bereitstellt, beispielsweise für Wahrnehmung und Planung. In ROS 2 bilden sogenannte Nodes (dt. Knoten) die Instanzen einzelner Softwaremodule. Diese Nodes kommunizieren über Topics, Services oder Actions miteinander.

Topics ermöglichen eine asynchrone, publish/subscribe-basierte Kommunikation zwischen mehreren Nodes. Das bedeutet, dass Nodes unabhängig voneinander Daten auf einem Topic abonnieren (subscribe) oder Daten auf diesem Topic veröffentlichen (publish) können. Services ermöglichen hingegen eine synchrone, anfragenbasierte Kommunikation zwischen zwei Modulen. Ein Client sendet eine Anfrage an einen Server und erhält daraufhin eine Antwort. Besonders für ROS 2 ist, dass ein Client eine Service-Anfrage auch nicht-blockierend senden kann, das heißt, die Ausführung des Clients wird bis zum Eintreffen der Antwort nicht angehalten.

Actions stellen eine weitere Form der asynchronen Kommunikation dar, die insbesondere für länger andauernde Aufgaben konzipiert ist. Intern bestehen ROS-2-Actions primär aus einem Goal-Service, einem Feedback-Topic und einem Result-Service. Zu Beginn wird ein Ziel (Goal) über den Goal-Service an den Action-Server gesendet, am Ende wird das Ergebnis über den Result-Service abgerufen. Während der Ausführung wird der Fortschritt über das Feedback-Topic übertragen. Darüber hinaus verfügen Actions über einen separaten Cancel-Service, über den laufende Zielanforderungen (Goal-Requests) abgebrochen werden können.

Ein weiteres nützliches Feature in ROS 2 ist, dass innerhalb Nodes in unkomplizierte Weise Parameter deklariert werden können. Diese können Standardwerte besitzen, die durch Konfigurationsdateien vor dem Start oder durch andere Nodes zur Laufzeit geändert werden können.

Zusätzlich zu den Kommunikationsmechanismen und den bereitgestellten Algorithmen stellt ROS 2 eine Reihe von Entwicklungstools zur Verfügung. Dazu zählt zum Beispiel RViz, das für die Visualisierung von Roboterumgebungen genutzt werden kann. (Macenski et al., 2022; „ROSActions“, n. d.)

2.4.4 MoveIt 2

MoveIt 2 ist ein Open-Source-Framework für robotische Manipulation in ROS 2 und stellt eine einheitliche Software-Schicht für typische Manipulationsaufgaben bereit. Dazu gehören insbesondere Bewegungsplanung, Kollisions- und Nebenbedingungsprüfung sowie die Ausführung geplanter Trajektorien über Controller-Schnittstellen. Die Integration in ROS 2 erfolgt über die üblichen Kommunikationsmechanismen (Nodes, Topics, Services und Actions). Zentral ist dabei die Node `move_group`, die als serverseitige Kernkomponente die Planung und Ausführung für definierte Gelenkgruppen des Roboters bereitstellt. Eine wesentliche Datenstruktur in MoveIt 2 ist die *Planning Scene* (dt. Planungsszene). Sie repräsentiert den aktuellen Roboterzustand und ein geometrisches Weltmodell der Umgebung und dient als Referenz für Kollisionsprüfungen. Die Szene kann zur Laufzeit aktualisiert werden, indem Objekte hinzugefügt, entfernt oder an den Roboter angehängt werden. Die Bewegungsplanung wird durch eine Planungsanfrage an die `move_group`-Node angestoßen. MoveIt 2 bindet verschiedene Planungsbibliotheken über eine Plugin-Schnittstelle ein, sodass je nach Anwendung unterschiedliche Planer ausgewählt werden können. Während der Planungs-Pipeline werden Kandidatenzustände typischerweise gegen die aktuelle Planungsszene hinsichtlich Kollisionsfreiheit geprüft. Das Ergebnis ist eine Trajektorie, die anschließend über die jeweilige Controller-Infrastruktur ausgeführt werden kann. („MoveIt 2 Documentation“, 2025)

2.5 Intelligente Robotersysteme

Siciliano und Khatib (2016) argumentieren, dass Robotersysteme für komplexe Aufgaben über Schlussfolgerungs Fähigkeiten verfügen müssen. Das bedeutet Roboter sollten anhand Wissen über den Ist-Zustand der Umgebung schlussfolgern können, was der angemessenste Weg ist, um in einen bestimmten Soll-Zustand zu gelangen. Für die Umsetzung eines solchen Planungssystem gibt es nach den Autoren drei wichtige Punkte, die zu beachten sind:

- Wie kann die Umgebung des Roboters formal dargestellt werden?
- Wie können Aktionen dargestellt werden?
- Wie wird anhand der ersten beiden Punkten ein Plan gefunden um in einen Zielzustand zu gelangen?

2.5.1 Wissensrepräsentationen

Der erste Punkt adressiert das Problem, dass Planungsmodelle nutzbare Informationen über den Ist-Zustand der Umgebung benötigen. Diese Informationen über die Umgebung und der in der Umgebung enthaltenden Objekte müssen ein präzises und kompaktes Format aufweisen, und gleichzeitig einem Aufgabenplaner erlauben effektive und schnelle Schlussfolgerungen zu treffen. Diese Aspekte stehen im Gegensatz zueinander, weshalb es viele Varianten der Formulierungsanssätze gibt. Ein Beispiel hierfür sind die Beschreibungslogiken (engl. Description Logics, (Description Logics (DLs))). In diesen Ansätzen werden Konzepte und Rollen definiert, welche zusammen als Ontologie bezeichnet werden. Ein Ort ist ein Beispiel für ein Konzept. Konzepte werden hierarchisch strukturiert. So ist ein Raum zum Beispiel ein Unterkonzept von Ort. Rollen sind Zusammenhänge zwischen Konzepten. Das Konzept Tür zum Beispiel führt in einen Raum. Durch konkrete Konzepte und Rollen in der Roboterumgebung kann dann neues Wissen inferiert werden. Die Web Ontology Language (OWL), konkret OWL-DL ist eine weitverbreitete Umsetzung einer DLs basierten Sprache, mit welcher Ontologien formal erstellt werden können. (Siciliano & Khatib, 2016)

2.5.2 Aufgabenplanung

Die beiden weiteren Punkte werden durch einen Aufgabenplaner adressiert. Für den zweiten Punkt muss der Aufgabenplaner über eine formalisierte Planungsumgebung verfügen. Diese repräsentiert alle möglichen Zustände eines Systems und die Aktionen die ausgeführt werden können, um von einem Zustand in einen anderen zu kommen. Weiterhin muss der Aufgabenplaner ein Planungsproblem definieren, bestehend aus dem Ist-Zustand der Umgebung und dem Zielzustand der erreicht werden soll. Anhand des Planungsproblem und der Planungsumgebung können Algorithmen eine Sequenz an Aktionen berechnen, die zum Zielzustand führen. Die Planning Domain Definition Language (PDDL) ist eine weit verbreitete Sprache, um Planungsprobleme und Umgebungen zu beschreiben. (Heuss et al., 2024)

Weiter identifiziert Heuss et al. (2024) drei verschiedene Möglichkeiten PDDL Planungsumgebungen zu erstellen:

- **Mannuelles Modellieren**: Für generische Aufgaben wie das Handhaben können existierende Umgebungen genutzt werden. Allerdings müssen diese jedoch für jeden spezifischen Anwen-

dungsfall manuell angepasst werden. Dies erfordert hohe Expertise in PDDL und Automatisierungsplanung und ist sehr arbeitsintensiv.

- **Lernen durch Demonstration**: Ansätze wie „Learning by Demonstration“ erfassen Aktionen und deren Zustandsänderungen aus Beispieldaten oder menschlichen Demonstrationen. Zwar lassen sich so neue Aktionen einlernen, jedoch ist die Anpassung an neue Objekttypen aufwändig, denn diese müssen in der PDDL neu definiert werden.
- **Übersetzung aus Wissensrepräsentationen**: Wissen aus anders formatierten vorhandenen Repräsentationen kann übersetzt werden in eine PDDL Planungsumgebung. Hierfür können zum Beispiel Umgebungsmodelle im OWL-Format benutzt werden. Bei bereits vorhandenen Umgebungsmodellen oder mit größerer Expertise in anderen Formaten, kann dieser Ansatz die Implementierung erleichtern

Kapitel 3 Stand der Wissenschaft und Technik

3.1 Skill Modelle in der Fertigung

Der Begriff Skill lässt sich zwar mit „Fähigkeit“ ins Deutsche übersetzen, in der Literatur besteht jedoch Uneinigkeit über die genaue Definition sowie die Abgrenzung zu Capability (Van De Ginste et al., 2021). Da beide Begriffe wörtlich dasselbe bedeuten würden, werden im Folgenden die englischen Fachbegriffe beibehalten, um Missverständnisse zu vermeiden.

In der Literatur existieren unterschiedliche Perspektiven auf das Skill-Konzept. Van De Ginste et al. (2021) und Heuss et al. (2019) identifizieren, dass ein Teil der Arbeiten Skills als ein herstellerunabhängiges Vokabular versteht, das Produkt, Prozess und Ressource miteinander verknüpft. Beispiele hierfür sind Järvenpää et al. (2019) und Veit und Gunther (2018). Dieses formalisierte Vokabular soll die autonome und flexible Ressourcenplanung sowie die Rekonfiguration von Produktionskapazitäten unterstützen. Diese Sichtweise folgt einem Top-down-Ansatz, da Modelle aus einer übergeordneten Perspektive auf den gesamten Produktionsprozess definiert werden, um Ressourcenplanung und Prozessgestaltung daraus abzuleiten.

Andere Arbeiten, wie Pedersen et al. (2016) beschreiben nach Van De Ginste et al. (2021) und Heuss et al. (2019) Skills als ausführbare Funktionen, die direkt an die Capabilities einer Ressource geknüpft sind. Der Begriff Capability wurde ursprünglich verwendet, um die übergeordneten Qualifikationen von Menschen zu beschreiben, wird aber mittlerweile auch auf abstrakte Ressourcenkonzepte übertragen. Skill wird in diesem Zusammenhang häufig synonym zu Capability genutzt, bezeichnet jedoch oft konkrete Maschinenfunktionen. In diesem Kontext zielen Skills darauf ab, einzeln oder kombiniert eine aufgabenorientierte Programmierung zu ermöglichen und so die Programmierung von Maschinen wie Robotern zu erleichtern. Diese Sichtweise entspricht einem Bottom-up-Ansatz, da sie von der konkreten Ausführbarkeit einzelner Ressourcenfunktionen ausgeht und daraus schrittweise höhere Prozessebenen ableitet.

Van De Ginste et al. (2023) bezeichnet den Top-down-Ansatz als Offline-Perspektive. Der Bottum-up-Ansatz wird als Online-Perspektive beschrieben. Im Folgenden werden ausgewählte Skill-Modelle aus beiden Perspektiven vorgestellt.

3.1.1 Offline Modelle

Järvenpää et al. (2019) argumentiert, dass für autonome Entscheidungen über die Rekonfiguration von Produktionsprozessen, ein einheitliches Modell benötigt wird, dass Produktanforderungen und Ressourcen Funktionen sowie Einschränkungen verknüpft. Die Autoren identifizieren, dass in diesem Modell Prozess, Produkt, Ressource und Capability vorkommen und verknüpft werden müssen. Mit der Manufacturing Resource Capability Ontology (Manufacturing Resource Capability (MaRCO)) entwickeln die Autoren ein OWL-basiertes Ressourcen und Capability Modell. Mit diesem

können Hersteller ihre Ressourcen mit Capabilitys versehen. Designer können dann anhand der angebotenen Capabilitys die zu ihren Produktanforderungen passende Ressource auswählen. Das Ressourcen Modell unterscheidet zwischen Fabrikeinheiten und Devices. Devices besitzen alleine oder in Kombination bestimmte Capabilitys. An diesem Punkt wird die verknüpfung zum Capability Modell hergestellt. Capabilitys können sowohl allein oder kombiniert auftreten und werden durch einen Namen und Parameter beschrieben.

Veit und Gunther (2018) präsentieren eine Skill-Taxonomie mit der Fähigkeiten von Ressourcen semantisch beschrieben werden können und dadurch von Aufgabenplanern genutzt werden können. Skills werden in erster Ebene aufgeteilt in Skills und Composite Skills. Die Autoren definieren Skills als lösungsneutrale, von einer einzelnen Ressource bereitgestellte Fähigkeit, die nicht weiter unterteilt werden kann. Ein Composite Skill ist hingegen eine lösungsneutrale kombinierte Fähigkeit die durch mehrere Ressourcen bereitgestellt wird. Composite Skills bestehen somit aus einer Sequenz aus Skills einzelner Ressourcen. Veit und Gunther (2018) versuchen anhand verschiedener Literaturquellen und Normen diese Aufteilung zu erweitern. Ihre Grundlegende Idee dabei ist, aus in Normen oder Lieraturquellen definierten Prozessschritten und kombinierten Prozessschritten, jeweils dazugehörige Skills beziehungsweise kombinierte Skills zuzuordnen. Skills sind eine lösungsneutrale Beschreibung einer Fähigkeit, bedeutet nicht gebunden an eine Umgebung. Trotz dieser angestrebten Hardware Unabhängigkeit lassen die Autoren Spielraum für individuelle Lösungen indem die Primitven Skills aufgeteilt werden in Standardisierte und Benutzerspezifische Skills. Benutzer sollen neben der Möglichkeit standardisierte Skills zu verwenden, die Möglichkeit besitzen Skills zu entwickeln die zugeschnitten sind auf ihre spezifische Anwendung. Die Einführung benutzerspezifischer Skills, soll nach Möglichkeit aber vermieden werden. Die standardisierten Skills sind unterteilt in Konfigurations- und standardisierte Montageskills. Die Konfiguartions Skills beinhalten unter anderem Skills die zur Steuerung und Automatisierung genutzt werden. Weiter sind die Montageskills strukturiert in Handhaben, Fügen, Inspizieren, Informationsaustausch, Mensch-Maschine Interaktion sowie Spezialaufgaben. Die Strukturierung wird weiter geführt und es entsteht am Ende eine baumartige Struktur, dessen Enden einzelne Skills darstellen die nicht weiter unterteilt werden können. Für Composite Skills entfällt die unterteilung zwischen benuterdefinierten und standardisierten Skills. Es gibt nur Standardisierte Skills welche unterteilt sind in Handhabungs-, Füge-, Inspektions und Spezialskills. Wie bei den Primitven Skills wird diese Strukturierung weitergeführt bis am ende einzelne Composite Skills stehen.

3.1.2 Hybride Modelle

Kocher et al. (2020) präsentieren eine OWL-Ontologie, die Capabilitys mit Ressourcen verknüpft. Es ist ein hybrider Ansatz, da ihr Modell, anders als zum Beispiel MaRCO, das Konzept der Ausführbaren Skills beinhaltet. Außerdem wird ein Rahmen gegeben wie Skills auf Ressourcen zu implementieren sind. Skills besitzen in dem Modell Zustände und Übergänge. Zustände können kontrolliert oder durch Transitionen herbeigeführt werden, beispielsweise durch OPC UA befehle. Das Modell besteht nicht aus einer alleinstehenden OWL Ontologie sondern aus sogenannten ontology design patterns, die zu einer gesamt Ontologie verknüpft werden. Somit ist Modularität und damit Erweitbarkeit des Modells möglich. Zudem versuchen die Autoren in der Ontologie standardisiertes Vokabular aus einer Vielzahl an Normen zu verwenden, um dadurch die Wiederverwendbarkeit zu erhöhen. Zum Beispiel werden Capabilities äquivalent gesetzt zu Prozessen nach VDI 3682. Diese sind wiederum unterteilt in Fertigungsverfahren nach DIN 8580 und Handhaben nach VDI 2860.

Van De Ginste et al. (2023) stellen ebenfalls ein formales Skill Modell vor, welches einem hybridem Ansatz folgt. Offline-Ansätze werden im vorgestellten Skill-Modell integriert, indem Ressourcen formal in Verbindung gebracht werden zu Prozessen die damit realisiert werden können und Produkte

die dadurch hergestellt werden können. Skills beschreiben den Übergang zwischen Zuständen von Ressourcen, wobei mit Ressourcen hierbei sowohl Equipment als auch Produkt gemeint ist. Die Autoren versuchen die Verbindung zu Online Ansätzen zu schlagen, indem mit dem Modell explizite Zustände beschrieben werden können und somit zur Laufzeit nach Skills gesucht werden kann, die einen Übergang in einen anderen Zustand herbeiführen können. Das Modell ist gekoppelt an den ISA-95 standard wodurch es sowohl von Enterprise Resource Planning Systemen (ERPS) als auch von Manufacturing Execution Systemen (MES) genutzt werden soll. Im Gegensatz zu dem im vorherigen Abschnitt beschriebenen Modell gibt es keine Zuordnung zwischen standardisierten Prozessschritten aus Normen und den dazugehörigen Skills.

3.1.3 Online Modelle im Bereich der Industrieroboter

Pedersen et al. (2016) präsentieren ein konzeptionelles Modell für objektzentrierte Roboterskills. Objektzentriert meint dabei, dass Skills nicht bezogen auf Raumkoordinaten ausgeführt und parametrisiert werden, sondern über Objekte und deren, in einem Umgebungsmodell gespeicherten Eigenschaften. Durch diese intuitive Parameter können die Skills ohne Expertenwissen parametriert werden. Das Konzeptuelle Modell eines Skills ist in Abbildung 3.1 zu sehen. Skills bekommen als Input (dt. Eingang) den Ist-Zustand der Umgebung und Parameter. Skills besitzen Vor- und Nachbedingungen die sowohl vor, während und nach der Ausführung überprüft werden. Der geänderte Zustand der Umgebung wird als Output (dt. Ausgang) an das Umgebungsmodell weitergegeben.

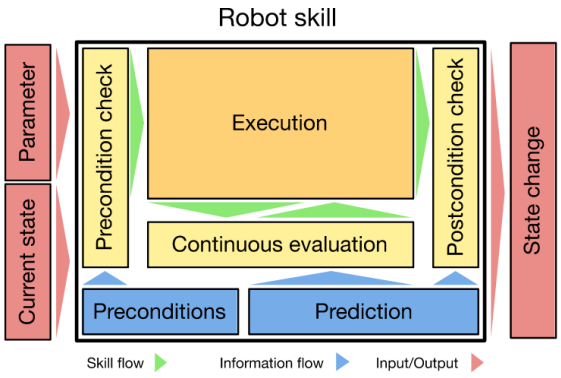


Abbildung 3.1: Konzeptuelles Modell eines Skills

Neben diesem Skill-Konzept definieren Pedersen et al. (2016) eine pyramidenförmige Skill-Architektur (Abbildung 3.2). Die Basis des Gebildes besteht aus den Device Primitiven, im Folgenden Primitive Skills genannt. Diese sind Hardwarespezifische Programmierbausteine, ähnlich zu denen, die für die textuelle Roboterorientierte Programmierung verwendet werden. Sie umfassen einfachste Bewegungsbefehle mit dem Zusatz, dass sie auf Basis von Sensordaten über ein Interface in Echtzeit angehalten oder verändert werden können. Eine Ebene darüber befinden sich die Skills, im Folgenden Composite Skills genannt. Diese sind Kombinationen mehrerer Primitiven und anderen Composite Skills. Sequentiell in bestimmter Reihenfolge ausgeführt ergeben Composite Skills eine Task. Diese bilden die oberste Ebene. Sie werden von einem Bediener oder Planungssystem erstellt und verfolgen ein übergeordnetes Ziel wie zum Beispiel das Beladen einer Maschine. Das Ziel des präsentierten Skill Konzeptes ist zum einem der bessere Umgang mit Produktvarianten, die Senkung des Wissens und zeitlichen Aufwands die für die Programmierung benötigt wird, das ermöglichen einer aufgabenorientierten Programmierung und die Wiederverwendung von Skill auf unterschiedlichen Robotern durch eine entsprechende Substitution der primitiven.

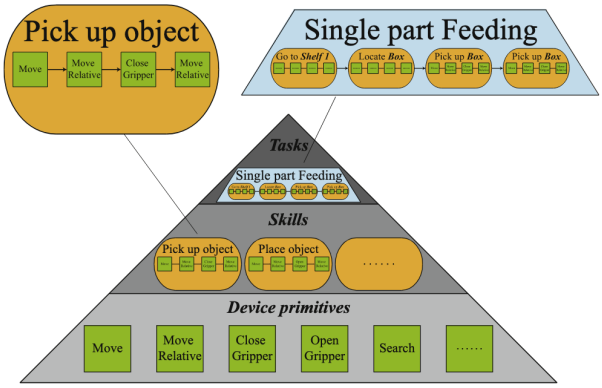


Abbildung 3.2: Die drei Ebenen der Skill-Architektur

3.2 Skill-Parameter

Parameter-Arten

Wie schon in 3.1.3 von Pedersen et al. (2016) erwähnt, benötigen Skills Parameter als Input. Parameter werden von Veit und Gunther (2018) aufgeteilt in zwei Klassen: Prozessbezogene Parameter und produktbezogene Parameter. Prozessbezogene Parameter dienen dazu, Anforderungen an eine Fähigkeit, etwa die Greifkraft oder die Messgenauigkeit, zu spezifizieren. Diese Parameter sind Inputs, welche der Skill vor der Ausführung benötigt. Da Skills dem Input-Processing-Output-Prinzip folgen, besitzen sie ebenso Output Werte. Diese stellen die Ergebnisse eines Skills dar. Für jeden Prozessbezogenen Parameter werden Kennungen, Datentypen, Werte oder Wertebereiche, Einheiten definiert. Zusätzlich muss noch spezifiziert werden, ob es sich um einen Input oder Output handelt und ob es ein optionaler oder zwingend benötigter Parameter ist. Ein typischer Prozessparameter ist die Zeitdauer die ein Skill für die Ausführung maximal benötigen soll. Separat zu diesen Prozesswerten stehen Geometrieparameter. Diese beschreiben an welchem räumlichen Ort der Skill ausgeführt werden soll, indem die sogenannte *Region-of-Interest* definiert wird.

Produktbezogene Parameter sollen die technische Eignung eines Geräts für einen Skill sicherstellen. Beispielsweise kann ein induktiver Sensor nur mit leitfähigen Materialien eingesetzt werden. Solche Eigenschaften, etwa Materialkennwerte, Masse oder geometrische Abmessungen, werden typischerweise aus digitalen Produktabbildern oder Datenbanken entnommen.

Methoden zur Parameter Identifizierung

Zur Ermittlung der benötigten Parameter existieren nach Veit und Gunther (2018) zwei methodische Ansätze. In Variante A werden alle bekannten Parameter, aller typischerweise benutzten Geräte gesammelt. Ein Nutzer muss jeden dieser Parameter für die Ausführung des Skills angeben. Somit ist sichergestellt, dass ein Skill aufjedenfall ausgeführt werden kann, allerdings bringt diese Variante auch einige Nachteile mit sich. Es besteht die Gefahr, dass Geräte nicht berücksichtigt werden und somit eine garantierte Ausführung auf diesen nicht gegeben ist. Außerdem sind viele Parameter sehr Gerätespezifisch und nicht prozessorientiert, was die Bedienung weniger intuitiv macht. Zuletzt führt dieser Ansatz zu einer sehr hohen Anzahl an Parametern von welchen viele nicht genutzt werden. Eine andere Variante der Parameterdefinition ist, nur prozessbezogene Paramter zu betrachten. Dadurch ist sichergestellt, dass der Skill Geräteunabhängig funktioniert. Im Falle, dass weitere Parameter für bestimmte Geräte gebraucht werden, können diese durch Teach-In, weitere Algorithmen oder manuell durch den Benutzer geholt werden.

Bestimmung von Parametern

Um Skills schließlich für eine konkrete Ausführung zu parametrieren, existieren verschiedene Varianten. Neben der manuellen Eingabe von Parametern (Archibald & Petriu, 1993), können auch Varianten wie Lernen durch Demonstration (Zhu et al., 2022) genutzt werden. Auch können Informationen aus dem Produktdesign oder der Fertigungsplanung genutzt werden um Parameter abzuleiten (Volkmann et al., 2020). Kurscheid und Daub (2024) nennt diese Informationen im allgemeinen *Task Specification*. Diese kann in verschiedenen Formaten wie beispielsweise CAD-Dateien auftreten. Kurscheid und Daub (2024) stellt dabei ein Konzept vor, dass vorsieht diese *Task Specifications* zu formalisieren und in einem Wissensmodell zu speichern. Lücken innerhalb diese Wissensmodells können dann identifiziert und über Reasoning geschlossen werden.

3.3 Skill-basierte Frameworks

Wie in 3.1.3 festgestellt, sind Skills alleine nicht ausreichend für die Ausführung von Tasks. Im Sinne einer aufgabenorientierten Programmierung funktionieren sie nur im Zusammenspiel mit anderen Software-Modulen, wie einem Umgebungsmodell. Nachfolgend soll eine Auswahl an Software-Frameworks vorgestellt werden, welche eine skillbasierte Programmierung vorsehen.

3.3.1 REpac Framework

Heuss et al. (2022b) stellt das ROS basierte REpac Framework vor. REpac ist aufgeteilt in drei aufeinander aufbauenden Stufen (s. Abb. 3.3). Dabei bildet Stufe eins die Basis während mit Stufe zwei und drei Module hinzukommen, die genutzt werden können, um den Grad der Autonomie zu erhöhen. Außerdem soll der Nutzer durch diese Aufteilung schrittweise hingeführt werden zum Entwickeln eines intelligenten Roboters.

Expansionsstufen

Stufe eins stellt die grundlegende Kontrollplattform des Roboters dar, bestehend aus den Modulen Skills, Skills manager, Task controller und Robot controller. In Skills werden die Roboter Skills gespeichert. Da nicht beliebig viele Skills auf dem Roboter gespeichert werden können, sieht das Framework vor, Skills aus einer Database auf den Roboter zu laden. Der Skills Manager lädt beim Einschalten des Roboters alle installierten Skills. Nachdem Laden der Skills registrieren diese sich beim Skill Manager, um sicherzustellen, dass sie korrekt installiert wurden. In der ersten Stufe des Softwareframeworks wird ein vollständiger Aktionsplan, bestehend aus einer sequenziellen Abfolge parametrisierter Skills, an den Robot Controller übergeben. Dieser überprüft den Plan hinsichtlich formaler Korrektheit und technischer Durchführbarkeit. Nach erfolgreicher Validierung leitet der Robot Controller den Aktionsplan an den Task Controller weiter. Der Task Controller übernimmt in der Folge die Überwachung und Steuerung der Ausführung, indem er den Status der beteiligten Skills während der Laufzeit kontrolliert. Darüber hinaus meldet er den aktuellen Ausführungsstatus an den Robot Controller zurück, um eine übergeordnete Koordination und ggf. eine adäquate Fehlerbehandlung zu ermöglichen. In der zweiten Stufe wird das Softwareframework um das Skill-Blackboard-Modul erweitert. Das Blackboard speichert konkrete Informationen über die Objekte in der Umgebung des Roboters. Skills können diese Informationen zur Parametrisierung nutzen und sie zudem im Blackboard aktualisieren. Auf diese Weise wird die Kommunikation zwischen

den Skills ermöglicht. Darüber hinaus kann das Blackboard auch für logische Schlussfolgerungen verwendet werden. Gespeicherte Informationen können beim Einschalten des Systems wieder geladen werden. In der dritten Stufe werden die Module Task Planner und Symbolic Knowledge Base hinzugefügt, wodurch eine autonome Aufgabenplanung ermöglicht wird. Aufgaben werden nun ausschließlich durch die Angabe der Zielzustände der Umgebung definiert. Der Task Planner erstellt basierend auf den vorhandenen Skills und deren Skill-Beschreibungen einen Aktionsplan in Form einer Sequenz von auszuführenden Skills. Hierbei nutzt der Task Planner auch die Symbolic Knowledge Base, die ähnlich dem Skill-Blackboard funktioniert, jedoch ausschließlich symbolische Informationen speichert.

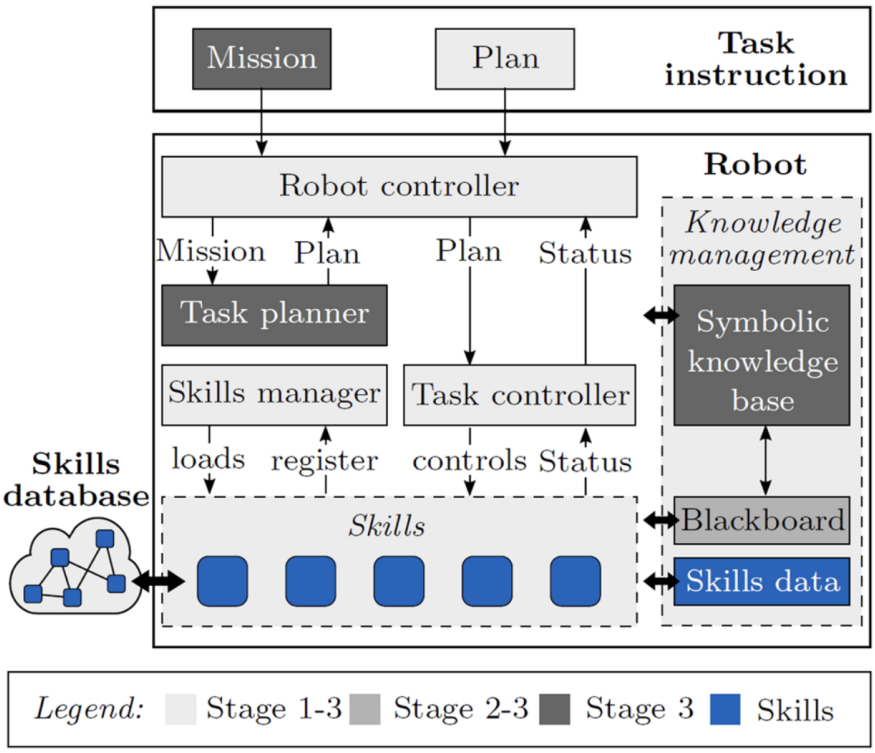


Abbildung 3.3: Module des REpac-Frameworks

Implementierung von Skills

Heuss et al. (2022b) stellt an Roboter-Skills vier wesentliche Anforderungen:

1. Wiederverwendbarkeit in unterschiedlichen Situationen, für verschiedene Aufgaben und auf diversen Robotersystemen
2. Modularität, Unabhängigkeit und in sich abgeschlossene Entwicklung
3. Klare Beschreibung der Funktionalität und der charakteristischen Merkmale
4. Externe Steuerbarkeit und Kontrollierbarkeit

Die Umsetzung dieser Anforderungen erfolgt durch eine einheitliche Kontrollstruktur und eine standardisierte Kommunikationsschnittstelle, die auf PackML basiert. Die Ausführung der Skills erfolgt

in Form von Zustandsautomaten mit Handlungs- und Warte-Zuständen. Kontrollstruktur und Kommunikationsschnittstelle werden durch eine Skill-Metaklasse bereitgestellt, die als Basis für die Entwicklung von Skills dient. Ergänzend dazu existieren zwei separate Metaklassen für Primitive und Composite Skills. Diese beiden Klassen erben jeweils von der Skill-Metaklasse und enthalten zusätzliche, für die jeweilige Klasse spezifische Methoden. Dadurch wird eine klare Strukturierung und Erweiterbarkeit der Roboter-Skills ermöglicht.

Der Prozess der Skill-Implementierung besteht im REpac-Framework insgesamt aus zwei Schritten.

Schritt 1: Erstellung der Skill-Beschreibung.
Für Primitive Skills besteht diese aus dem Namen des Skills und einer semantischen Beschreibung seiner Funktion. Weiterhin wird definiert, welche Parameter vor der Ausführung an den Skill übergeben werden müssen und was die Outputs des Skills sind. Wenn nur die erste Stufe des Frameworks genutzt wird, ist diese rein prozessorientierte Beschreibung ebenfalls ausreichend für Composite Skills. Bei Nutzung der zweiten Stufe muss in der Skill-Beschreibung definiert werden, welche Objekte manipuliert werden und welche Objektparameter benötigt werden. Die Objektparameter holt sich der Skill dann zur Ausführung aus dem Blackboard. Zusätzlich müssen die Objekt-Endzustände beschrieben werden, damit diese nach der Ausführung des Skills zurück an das Blackboard gegeben werden können. Wenn Stufe drei genutzt wird, muss in der Beschreibung eines Composite Skills zusätzlich noch definiert werden, welche Vorbedingungen der Skill benötigt und welche Endbedingungen er verursacht, damit die automatische Aufgabenplanung umsetzbar wird.

Schritt 2: Modellierung des Skill-Ablaufs.
Basierend auf der Beschreibung wird anschließend der Ablauf der Skills modelliert. Die Abläufe innerhalb eines Skills können sowohl sequenziell geskriptet als auch in Form von Zustandsautomaten modelliert werden. Für Primitive Skills genügt reines Skripting aufgrund ihrer geringen Komplexität, während für Composite Skills eine komplexere Modellierung mittels endlicher Zustandsautomaten sinnvoller ist. Im Falle einer Prozessmodellierung mittels endlicher Zustandsautomaten wird das ROS package SMACH zur Modellierung eingesetzt. Die Zustände repräsentieren die untergeordneten Skills des Composite Skills. Wenn ein untergeordneter Skill mehrfach verwendet wird, das heißt ein Zustand mehrmals eintritt, ist es wichtig, dass die Parameter zwischen den Ausführungen im übergeordneten Skill neu definiert werden.

3.3.2 SkiRos2

Mayr et al. (2023) führen mit SkiROS2 ebenfalls eine skillbasierte Steuerungsplattform für Industrieroboter ein. SkiROS 2 ist die Nachfolgeversion des bereits zuvor vorgestellten SkiROS 1 und wurde grundlegend überarbeitet, um den Anforderungen moderner Robotiklösungen in industriellen Umgebungen gerecht zu werden. Ziel ist es, eine Plattform zu schaffen, die sowohl symbolisches Wissen als auch reaktives Verhalten und lernbasierte Ansätze in einem einheitlichen Rahmen integriert.

Die Architektur von SkiROS 2 folgt dem Ansatz hybrider Steuerungsarchitekturen und kombiniert deliberative, planungsbasierte Komponenten mit reaktiven Verhaltenseinheiten. Um ein reaktives Verhalten zu gewährleisten setzt SkiRos auf Verhaltensbäume für die Modellierung von Aufgaben. Skills werden in SkiRos 2 als parametrischer Prozess beschrieben, der die Umgebung von einen bestimmten Anfangszustand in einen bestimmten Endzustand bringt. Die Definition ist dabei sehr offen gehalten, damit Skills sowohl einfache motorische Fähigkeiten inkludieren als auch komplexere Programme wie deep learning basierte Objektlokalisierung.

Skill-Beschreibung

Es wird ebenfalls zwischen primitiven Skills und Composite Skills unterschieden, als grundlegende Roboterfunktionen und komplexeren Fähigkeiten. Beide Skill-Arten beinhalten eine Skill-Beschreibung, welche Parameter sowie Pre-, Hold- und Post Conditions definiert (dt. Vor-, Hold- und Nachbedingungen).

Alle Beschreibungen sind eigene Klassen, welche von der *SkillDescription*-Klasse erben. Eine Beschreibung kann für mehrere Skill-Implementierungen verwendet werden. Jede Skill-Implementierung enthält allerdings genau eine Beschreibung. In der Skill-Implementierung kann die Beschreibung genauer spezifiziert oder ergänzt werden. Diese Zusammenhänge zwischen Beschreibungen und Skill-Implementierungen ermöglichen es, dass Beschreibungen wiederverwendet werden können, wenn zum Beispiel eine Fähigkeit aufgrund von Hardware-Abhängigkeiten mehrere Skill-Implementierungen benötigt.

Die Skill-Parameter definieren den In- und Output eines Skills und können benutzt werden, um auf Pre- und Post Conditions zu schließen. Es gibt required, optional und inferred Parameter. Ohne die required Parameter kann ein Skill nicht ausgeführt werden. Optionale Parameter können zum Beispiel die Geschwindigkeit sein und müssen nicht zwingend gesetzt werden. Inferred Parameter können aus den Relationen im Weltmodell hergeleitet werden.

Parameter können fundamentale Datentypen wie zum Beispiel *Int* oder *String* sein. Zusätzlich können Konzepte aus dem Weltmodell als Parameter mitgegeben werden. Während der Ausführung des Skills werden dann konkrete Instanzen dieser Konzepte zu Skill-Parametern.

Die Pre-, Hold- und Post Conditions beschreiben den Zustand des Weltmodells vor, während und nach der Ausführung des Skills. Hold Conditions werden in jedem Tick des Verhaltensbaum überprüft. Die Skill Node gibt *failure* zurück, wenn die Post Conditions nicht erreicht wurden.

Es kann vier Typen an Conditions geben. Eine Condition kann das Überprüfen von Beziehungen im Weltmodell sein, zum Beispiel ob an dem Roboterarm ein Greifer angebracht ist. Ein weiterer Typ an Conditions ist die Prüfung, ob ein Element im Weltmodell bestimmte Eigenschaften hat, zum Beispiel ob der Greifer einen Wert besitzt, der seine Länge spezifiziert. Außerdem können Werte verglichen werden, wie die Länge des Greifers mit einer bestimmten Zahl. Schließlich kann bei abstrakten Beziehungen überprüft werden, ob diese laut der Ontologie im Weltmodell überhaupt erlaubt sind.

Skill-Implementierung

Jede Implementierung eines Primitiven Skills ist eine Klasse die von der PrimitiveBase Klasse erbt. Diese stellt Methoden zur Vefügung die verwendet werden für die Initialisierung, das Starten, Ausführen, vorzeitige Stoppen und für das Ende der Skill Ausführung. Wenn ein Primitiver Skill im Verhaltensbaum angetickt wird, wird die Methode zur Skill Ausführung aufgerufen. Für Primitive Skill wird in dieser Methode geskriptet was passieren soll. Die Methode leitet den Status des Skills (running, success oder failure) durch den return Wert der Methode an den Verhaltensbaum weiter. Composite Skills erben von der SkillBase Klasse. Sie können eine Vielzahl an Primitven oder Composite Skills enthalten. Für die Modellierung der Composite Skills reicht aufgrund der höheren Komplexität einfaches Skripten, wie bei den Primitiven Skills nicht mehr aus, weshalb auf eine Modellierung über Verhaltensbäume gesetzt wird.

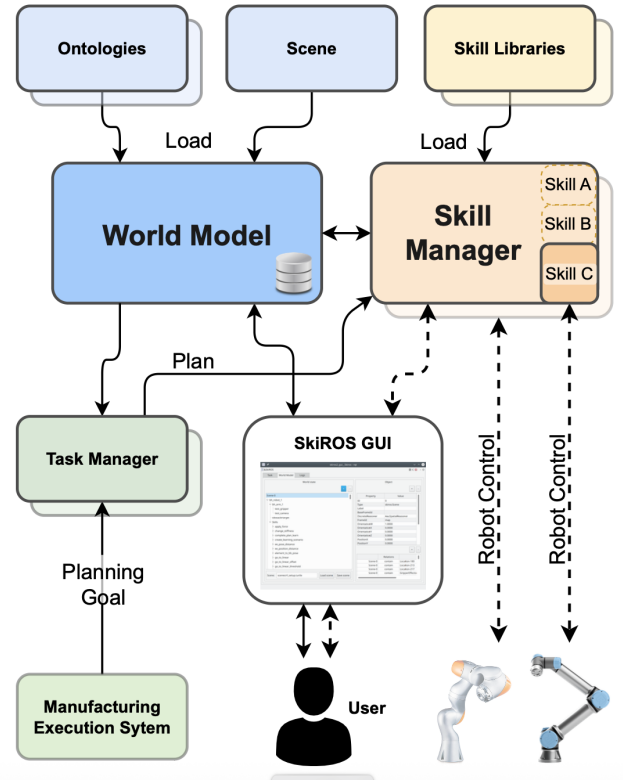


Abbildung 3.4: Enter Caption

Module

Die Kern Module in SkiRos 2 sind der Skill Manager und das World Model (dt. Weltmodell). Der Skill Manager lädt ein festgelegtes Set an Skills aus Skill Libraries, initialisiert sie und ergänzt das Weltmodell um semantische Informationen, wie zum Beispiel die expliziten Instanzen der Skill-Parameter und Pre- und Postconditions. Nach der Initialisierung bietet der Skill-Manager Services zum Starten, Stoppen, Debuggen und Überwachen der Skills an. Der Skill Manager nimmt einen Skill oder eine Sequenz an Skills von dem Task Manager Modul entgegen. Der Skill Manager erstellt aus dieser Sequenz eine neue Aufgabe und versieht diese mit einer ID. Innerhalb einer Aufgabe werden die Parameter der ausgeführten Skills über ein Blackboard geteilt. Dadurch ist ein Informationsaustausch zwischen den Skills möglich. Das Weltmodell speichert die Ontologien und deren Instanzen in der Umgebung des Roboters, wodurch es für die Palnung und Parametrisierung von Skills verwendet werden kann. SkiRos verwendet dafür OWL-DL basierte Ontologien wie zum Beispiel CORA. Ein vorher gespeichertes Weltmodell kann erneut geladen werden, so dass zum Beispiel Informationen über das Robotersystem oder wichtige Positionen in seiner Umgebung erhalten bleiben. Das Weltmodell wird außerdem mit Informationen über die verfügbaren Skills ausgestattet. Der Task Manager stellt den planungsbasierten Teil der hybriden Softwarearchitektur dar. Er besitzt einen Aufgabenplaner, basierend auf der PDDL, der für die Erstellung einer Sequenz an geeigneten Skills verantwortlich ist. Der Aufgabenplaner berücksichtigt dabei automatisiert Informationen aus dem Weltmodell inklusive sämtlicher verfügbaren Skills mit Pre- und Post-Conditions. Die erstellte Skill Sequenz wird anschließend an den Skill Manger weitergegeben. Für die Interaktion mit Endanwendern bietet SkiROS 2 eine grafische Benutzeroberfläche. Diese erlaubt es, Skills zu inspizieren, zu konfigurieren und zu starten, sowie den Zustand des Weltmodells zu visualisieren und zu manipulieren. Durch die Kopplung mit Visualisierungswerkzeugen wie können Positionen und Relationen intuitiv bearbeitet werden, was besonders für nicht-technische Anwender einen erheblichen Mehrwert bietet.

Mit mehreren Anwendungsfällen demonstrieren die Autoren die Flexibilität und Leistungsfähigkeit des Frameworks. So wurde SkiROS2 etwa für Pick-and-Place-Aufgaben mit mobilen Manipulatoren eingesetzt, bei denen Zielzustände automatisch geplant und die zur Ausführung benötigten Parameter vollständig aus dem Weltmodell abgeleitet wurden. Ein weiterer Anwendungsfall beinhaltete die präzise Zweiarm-Manipulation für die Insertion von Kolben in Motorblöcke, bei der Skills zur Kraftüberwachung und visuellen Objekterkennung kombiniert wurden.

3.3.3 Weitere Frameworks und kommerzielle Lösungen

Sorensen et al. (2020)

Die Autoren beschreiben ein visuelles Programmier-Framework für Industrieroboter, das auf sogenannten *SkillBlocks* basiert. Der Fokus der Arbeit liegt darin den realen Roboter mit einem digitalen Zwilling zu verknüpfen. In der digitalen Abbildung wird sowohl der Ist-Zustand des realen Systems als auch der Zustand des Zwillings gezeigt und miteinander verknüpft. Dadurch kann zum Beispiel die aktuelle Position des realen Roboters als Ausgangspunkt für ein neues Programm dienen. Die Autoren grenzen sich ab von der objektzentrierten Natur der Skills wie sie von Pedersen et al. (2016) definiert wurde. Diese kommt daher, dass Pedersen et al. (2016) beabsichtigen Roboter in der realen Welt zu teachen. In Simulationen hingegen können Koordinaten und Frames intuitiver herübergebracht werden. Außerdem vermeiden die Autoren eine Unterteilung in Composite und Primitive Skills. Jeder visuelle *SkillBlock* ist einem Skill zugeordnet, der die wirkliche Funktionalität enthält. Skills sind klar getrennt in Funktionalität, Parameterdaten (Property Board) und Ein-/Ausgänge (IO-Board). Die Property Boards enthalten veränderbare Parameter wie Zielpositionen oder Geschwindigkeiten, während das IO-Board die Verbindung zu anderen *SkillBlocks* ermöglicht, sodass ganze Aufgaben sequenziell aufgebaut werden können.

Die Ausführung eines Skills wird über Execution Engines gesteuert. Ein Skill besitzt zwei Execution Engines jeweils für die reale und die digitale Abbildung. So kann der exakt gleiche Ablauf zuerst virtuell getestet und anschließend ohne Codeänderung auf den realen Roboter übertragen werden. Die Execution Engines sind austauschbar, um auf Änderungen der Hardware zu reagieren.

Das Framework wurde in einem Montageszenario mit zwei Robotern validiert. Ein großer Roboter positionierte ein Werkstück, während ein kleiner Roboter mit montiertem Schraubwerkzeug Schrauben eindrehen musste. Der komplette Ablauf wurde zunächst als Sequenz von *SkillBlocks* erstellt und in der Digital-Twin-Simulation getestet. Anschließend wurde derselbe Ablauf unverändert auf die reale Roboterzelle übertragen. Die Ergebnisse zeigten, dass Simulation und reale Ausführung deckungsgleich funktionierten und der Wechsel zwischen virtueller und physischer Umgebung reibungslos möglich war.

ABB Wizard Easy Programming

ABB Wizard Easy Programming ist eine von ABB entwickelte Software zur vereinfachten Roboterprogrammierung, die auf einem modularen, blockbasierten Low-Code-Ansatz basiert. Durch die Bereitstellung grundlegender Skills als Funktionsbausteine wie *Move*, *Select*, *Place* oder *Repeat* ermöglicht das System eine intuitive Umsetzung typischer Roboteraufgaben über eine visuelle Benutzeroberfläche. Wie in Abb. 3.5 zu sehen ist, erfolgt die Programmierung per Drag-and-Drop, der einzelnen Skills zu einer ausführbaren Sequenz. Manuell werden verschiedene Parameter festgelegt, wie zum Beispiel vorgespeicherte Positionen. Die Ausführung und das Verhalten des Roboters lassen sich anschließend in Echtzeit testen und anpassen, wodurch Iterationen innerhalb weniger

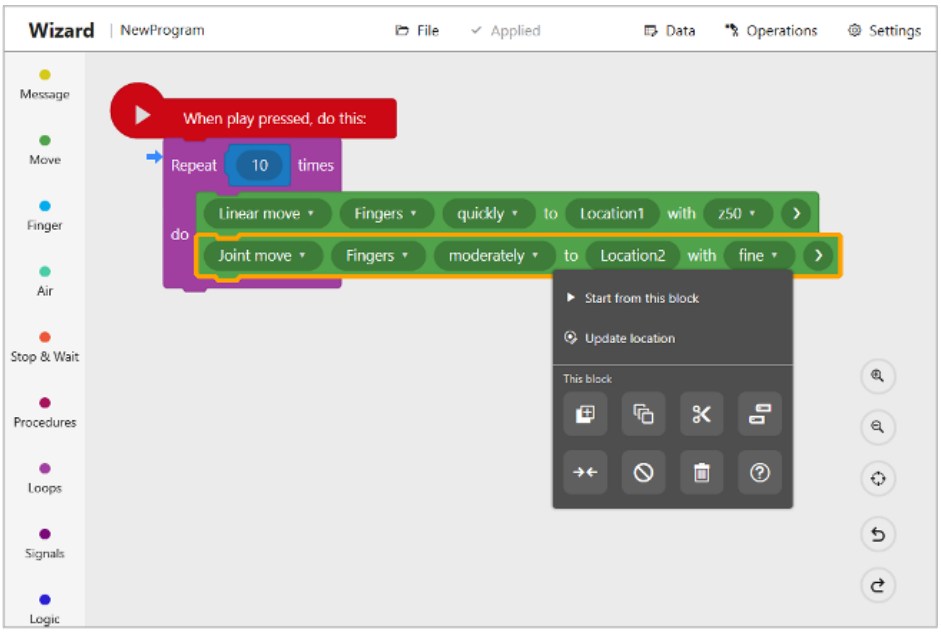


Abbildung 3.5: ABB Wizard Easy Programming

Sekunden möglich sind. Nutzer können dadurch auch ohne Vorkenntnisse in kurzer Zeit einfache Roboterprogramme erstellen. Für erfahrene Anwender bietet Wizard die Möglichkeit, mit dem Skill-Creator eigene, komplexe Skills zu erstellen und in das System zu integrieren. Dadurch wird die einfache, visuelle Programmierung um professionelle Erweiterbarkeit ergänzt und erlaubt sowohl standardisierte als auch individuelle Programmabläufe. („Wizard Easy Programming“, n. d.)

JAKA Robotics

JAKA Robotics bietet mit ihrer JAKA Zu Software eine graphische Oberfläche zur Programmierung, Überwachung und Steuerung von JAKA Robotern an. Die Anwendung ist sowohl für Windows Rechner, als auch für mobile Geräte über den Android oder Apple Appstore erhältlich. Die Benutzeroberfläche, wie in Abb. 3.6 zu sehen, bietet die Möglichkeit per Drag-and-Drop, Programmbausteine sequentiell anzuordnen und dadurch ein Roboterprogramm graphisch aufzubauen. Die Programmbausteine können dabei aus einfachen Bewegungsbefehlen, Befehle zum Überwachen und Steuern der Inputs und Outputs (I/O) oder aus Rechenfunktionen bestehen. Die Programmbauteine stellen dabei grundlegende Fähigkeiten der Roboter dar, und sind sich dadurch sehr ähnlich zu den Primitiven Skills, wie sie in Abschnitt 3.1.3 definiert wurden. Die Programme können in der App entweder Simuliert werden oder direkt auf den Roboter gespielt werden. Somit kann sowohl Online als auch Offline programmiert werden (JAKA Robotics, 2024).

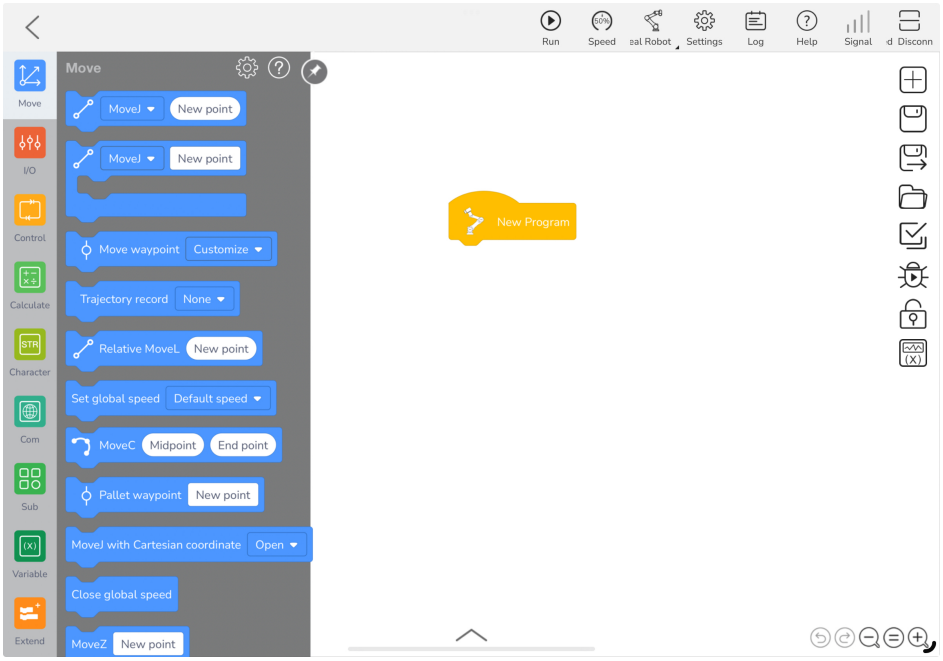


Abbildung 3.6: JAKA Zu Programmieroberfläche

Kapitel 4 Methodik und Konzeption

4.1 Ausgangssituation und Rahmenbedingungen

Folgend soll die Ausgangssituation für diese Arbeit dargelegt werden. Hierzu zählt eine fundierte Analyse des Stand der Wissenschaft und Technik, sowie eine Darstellung der für diese Arbeit zur Verfügung stehenden mittel.

4.2 Analyse des Stands der Technik

Zusammenfassend zeigt der aktuelle Stand der Wissenschaft und Technik, dass das Skill-Konzept sowohl aus Sicht der Produktion als auch speziell aus Sicht der Robotik bedeutende Potenziale bietet. Skills können in der Produktion für das Design und die Rekonfiguration von Fertigungslinien genutzt werden, da sie eine formale, für Computer verständliche Verbindung zwischen Produkt, Ressource und Prozess schaffen.

Aus Sicht der Robotik sind Skills wiederverwendbare Funktionsbausteine, die durch ihre Kombination ein Roboterprogramm ergeben. Häufig wird zwischen Primitiven Skills und Composite Skills unterschieden. Primitive Skills bilden hardwarenahe Funktionen auf unterster Ebene ab und sind daher in der Implementierung gerätespezifisch. Bei einem Austausch von Geräten, die dieselben Skills bereitstellen, sollte sich die Codestruktur idealerweise nicht ändern. Composite Skills sind hingegen komplexere, hardwareunabhängige Funktionen, die sich aus der Kombination verschiedener Primitiven und/oder Composite Skills zusammensetzen.

Skills zeichnen sich dadurch aus, dass sie durch eine Reihe symbolischer Vor- und Nachbedingungen beschrieben werden können. Diese Bedingungen ermöglichen den Einsatz autonomer Aufgabenplanungssysteme und unterstützen eine aufgabenorientierte Programmierung, bei der primär Zielzustände spezifiziert werden. Über diese Vor- und Nachbedingungen kann zudem eine Verbindung zu skillbasierten Produktionskonzepten hergestellt werden.

Für einen konkreten Prozess werden Skills mit Parametern versehen, die spätestens kurz vor der Ausführung festgelegt werden müssen. Dabei existieren unterschiedliche Parametertypen, beispielsweise optionale sowie zwingend erforderliche Parameter. Im Sinne der aufgabenorientierten Programmierung ist es dabei zweckmäßig, Parameter objektbezogen zu definieren.

Bei der Implementierung von Skills ist außerdem festzulegen, wie ein einzelner Skill modelliert wird. Während bei Primitiven Skills häufig ein einfaches Aneinanderreihen von Befehlen ausreichend ist, werden für Composite Skills häufig Zustandsautomaten oder Verhaltensbäume eingesetzt. Dabei repräsentieren die Zustände bzw. Blattknoten jeweils Skills, die in Kombination den Composite Skill bilden.

Ein Bereich, in dem weiterer Forschungs- und Entwicklungsbedarf gesehen werden kann, ist die

Verknüpfung der Skill-Konzepte aus der Robotik und der Produktionsplanung. Während in der Produktionsplanung bereits Ansätze existieren, die von einer Ressource bereitgestellte Skills standardisierten Fertigungsprozessen zuzuordnen, ist eine vergleichbar systematische Verankerung in robotikzentrierten Skill-Konzepten bislang weniger ausgeprägt.

Darüber hinaus ist die Wiederverwendbarkeit von Skills über verschiedene Anwendungsfälle, Umgebungen und Hardwareplattformen hinweg noch nicht umfassend untersucht. Die Evaluation bestehender Frameworks erfolgt häufig in vergleichsweise fokussierten Szenarien. Verschiedene Robotertypen oder Endeffektoren werden in der Literatur bislang seltener einbezogen. Geng et al. (2023) konnten zwar eine Teilwiederverwendung bestehender Skills demonstrieren, für den Einsatz eines Sauggreifers war jedoch die Entwicklung neuer primitiver Pick-and-Place-Skills erforderlich.

Ein weiterer Aspekt betrifft die Gestaltung der Skill-Ausführung und die damit verbundene Parametrierung. Während Veit und Gunther (2018) für einige standardisierte Skills grundlegende Parameter definiert haben, wurde eine weitergehende Ausarbeitung bislang vor allem in Ansätzen behandelt. Geng et al. (2023) verdeutlichen exemplarisch, dass ein Pick-Skill nicht nur den eigentlichen Greifvorgang umfasst, sondern aus mehreren Bewegungsphasen bestehen kann. Solche mehrstufigen Bewegungsabläufe sind bisher jedoch noch nicht durchgängig beschrieben worden.

4.2.1 Software Framework

Task-Controller

Es wird eine auf ROS2 portierte Version des REpac-Frameworks benutzt. Diese Version enthält bis dahin als Modul einen Task-Controller als ROS Node. Der Task-Controller besitzt einen Action Server über den in Form einer JSON-Datei neue Aktionspläne entgegengenommen werden können. Die Aktionspläne bestehen dabei aus einer Sequenz an Skill-Aufrufen. Aus dem Plan baut der Task-Controller einen Zustandsautomaten, welcher anschließend asynchron ausgeführt wird. Während der Task-Controller sich in der Ausführung eines Aktionsplanes befindet, werden keine neuen Aktionspläne über den Action Server angenommen. Parameter für die Skills können direkt in der JSON-Datei stehen oder alternativ über die Skill Ergebnisse zwischen verschiedenen Skills weitergeleitet werden. Die Ausführung kann erfolgreich enden, bei Fehlern abbrechen, oder vorzeitig beendet werden können.

Skill-Basisklasse

Für die Implementierung von Skills existieren bereits wie im REpac-Framework beschrieben Basisklassen. Die Klasse `Applet` stellt das Basis-Framework für die Skills bereit. Die Klasse sorgt dafür, dass bei der Skill-Initialisierung ein Zustandsautomat in einem eigenen Thread erstellt wird. Die verschiedenen Zustände sind in Abbildung 4.1 dargestellt. In jedem Zustand wird eine entsprechende Methode des jeweiligen Zustandes ausgeführt. In der Basisklasse selbst sind diese Methoden leer und jede erbende Klasse kann diese Methoden überschreiben oder ergänzen. Um den Zustand zu wechseln, kann aus jedem Zustand heraus eine Methode aufgerufen werden.

Zusätzlich zu dem Zustandsautomaten, erstellt die `Applet` Klasse die nötigen ROS2 Schnittstellen für die Skills. Dazu zählt der Action Server, über welchen neue Anfragen angenommen werden. Das Interface dieses Action Servers ist in Abbildung 4.2 dargestellt. Parameter, Ergebnis und Status werden als String im JSON-Format erwartet. In der Ziel-Callback Funktion des Action Servers wird überprüft, in welchem Zustand sich der Zustandsautomat gerade befindet. Das neue Ziel wird

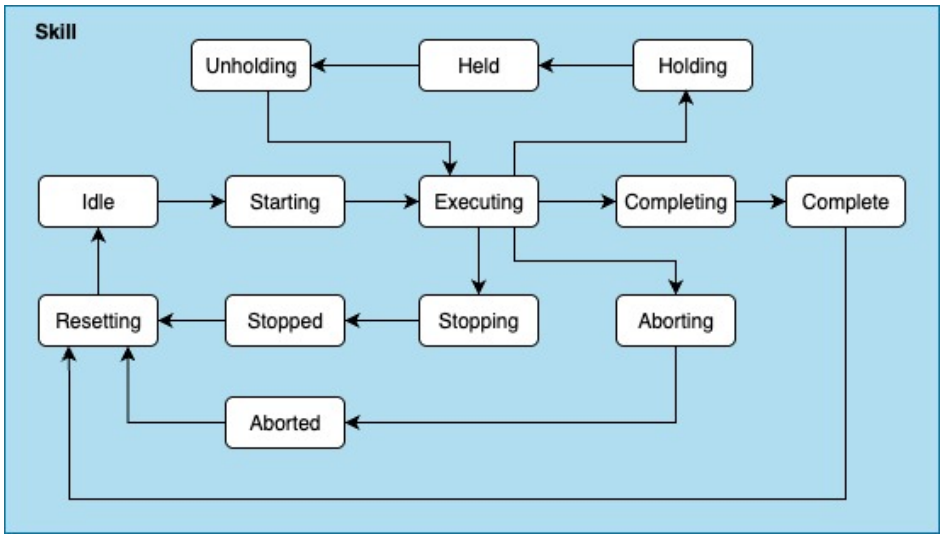


Abbildung 4.1: Zustände eines Skills, basierend auf Heuss et al. (2022b)

ExecuteSkill.action
#Goal string parameters
#Result int32 outcome string results
Feedback string status

Abbildung 4.2: Interface des Skill Action Servers

nur akzeptiert, wenn sich der Skill in `Idle`, `Complete`, `Aborted`, `Stopped` oder `Resetting` befindet. Zu Beginn der Ausführung des Action-Requests, wird der Skill in den `Starting`-Zustand gesetzt. Fortlaufend wird intervallweise geprüft, ob ein Zustandswechsel stattgefunden hat. Wird einer der Zustände `Complete`, `Aborted` oder `Stopped` erreicht, wird der Skill finalisiert und das Ergebnis an den Client zurückgeschickt. Neben dem Action-Server werden auch Services jeweils für `Hold`-, `Unhold`- und `Stop`- Anfragen erstellt.

Die `Applet` Klasse vererbt ihre Funktionalitäten weiter Basis-Klassen jeweils für Composite Skills und für Primitive Skills. Diese bieten zu diesem Zeitpunkt allerdings noch keine Funktionalitäten. Gefunden

4.2.2 Ressourcen und Primitive Skills

Weiterhin bestehen bereits ROS Launch-Dateien, die einen UR30-Knickarmroboter und einen Parallelgreifer (Zwei-Backen-Greifer) in einer Simulationsumgebung inklusive MoveIt 2 MoveGroup-Node starten. Im Folgenden werden die bereits zur Verfügung stehenden Primitiven Skills aufgzeigt. Eine Zusammenfassung der Skills und ihrer Schnittstellen ist in Tabelle 4.1 dargestellt.

Greifprimitive

Die Greifprimitive kapselt die Greiferansteuerung und stellt eine reduzierte, aufgabennahe Schnittstelle bereit. Als Eingabe wird ein einfacher Greifbefehl (`command`) verwendet. Als Ausgabe wird der Ausführungsstatus und optional die erreichte Endstellung und ggf. eine Fehlerursache zurückgegeben.

PTP-Bewegungsprimitive

Die PTP-Bewegungsprimitive realisiert kollisionsbewusste Punkt-zu-Punkt-Bewegungen des Roboterarms zu einer vorgegebenen Zielpose des TCP des Roboters (ohne Werkzeug, fortlaufend mit *tool0* bezeichnet). Die Bewegungsplanung und Ausführung erfolgt über MoveIt. Als Rückgabe liefert die Primitive einen Status sowie bei Fehlern eine textuelle Ursache.

Lineare Bewegungsprimitive

Die lineare Bewegungsprimitive beschreibt eine relative Zustellbewegung entlang eines Richtungsvektors um eine Weglänge. Die Ausführung erfolgt über eine MoveIt-basierte Planung. Die resultierende Trajektorie ist dabei nicht zwingend geometrisch strikt linear. Die Primitive gibt den Ausführungsstatus sowie im Fehlerfall eine Ursache zurück.

Primitive	**Eingänge (abstrakt)**	**Ausgänge (abstrakt)**
Greifprimitive	Kommando (Öffnen/Schließen)	Erfolg/Fehlschlag, optional finale Endstellung und Fehlertext
PTP-Bewegungsprimitive	Zielpose von *tool0* (Weltkoordinaten)	Erfolg/Fehlschlag, optional Fehlertext
Lineare Bewegungsprimitive	Richtungsvektor und Weglänge (Weltkoordinaten)	Erfolg/Fehlschlag, optional Fehlertext

Tabelle 4.1: Abstrahierte Übersicht der Schnittstellen der vorhandenen Primitiven Skills.

4.3 Zieldefinition und Anforderungsanalyse

Basierend auf der Analyse des Stand der Wissenschaft und Technik, werden im folgenden Ziele für diese Arbeit festgelegt. Ausgehend von den Zielen und den beteiligten Stakeholdern, werden anschließend Anforderungen definiert.

4.3.1 Zweck und Zielsetzung

Das Ziel dieser Arbeit besteht darin, Roboter-Skills für die Montage zu entwickeln, die sich den Kategorien der Handhabung gemäß *VDI-Richtlinie 2860* (1990) sowie dem Fügen gemäß *DIN 8593-0* (2003) zuordnen lassen. Im Fokus stehen dabei insbesondere ein bahngeführter Klebeprozess (Dosier) und ein Pick-and-Place-Prozess als typische Vertreter dieser Normklassen.

Es soll demonstriert werden, dass sich auf dieser Basis Skills realisieren lassen, die wiederverwendbar sind und typische Montageaufgaben abdecken können, ohne auf einen einzelnen, starr modellierten Anwendungsfall beschränkt zu sein. Die Skills sollen eine aufgabenorientierte Programmierung unterstützen, bei der gewünschte Endzustände der Umgebung über Sequenzen von Skillaufrufen beschrieben werden und sich so prinzipiell auch durch autonome Planungssysteme instanziieren lassen.

Darüber hinaus soll die Arbeit zeigen, dass sich die entwickelte Sammlung von Skills in die bestehende skillbasierte Softwarearchitektur integrieren lässt und dass die Architektur geeignet ist, entsprechende Montageprozesse abzubilden.

Auf Grundlage dieser Beschreibung lassen sich die folgenden übergeordneten Ziele formulieren. Die Ziele werden fortlaufen nach dem Schema Z-... referenziert.

Z-1– Montageoperationen nach Norm: Entwicklung von Roboter-Skills, die grundlegende Operationen der Handhabung gemäß *VDI-Richtlinie 2860* (1990) sowie Operationen des Fügens gemäß *DIN 8593-0* (2003) abdecken.

Z-2– Aufgabenorientierte Programmierung: Unterstützung einer aufgabenorientierten Programmierung von Montageprozessen, sodass gewünschte Endzustände der Umgebung über Sequenzen von Skillaufrufen beschrieben werden können.

Z-3– Flexibilität und Wiederverwendbarkeit: Nachweis, dass die entwickelten Skills flexibel gegenüber Variationen der Umgebung und der zu handhabenden Objekte sind und dass sie wiederverwendbar in unterschiedlichen Szenarien eingesetzt werden können, ohne auf einen spezifischen Anwendungsfall beschränkt zu sein.

Z-4– Integration in die Softwarearchitektur: Integration der entwickelten Skills in die bestehende skillbasierte Softwarearchitektur. Im Mittelpunkt stehen dabei insbesondere die Interaktion mit dem vorhandenen Task-Controller sowie die korrekte Einbindung in den Zustandsautomaten, dem jeder Skill während seiner Ausführung unterliegt (z. B. Behandlung von `Hold`-, `Stop`- und `Abort`-Ereignissen).

4.3.2 Stakeholder

Im Kontext des entwickelten skillbasierten System lassen sich zwei zentrale Stakeholder unterscheiden:

- **Entwickler:** Implementieren neue Primitive- und Composite-Skills. Für diese Rolle sind insbesondere Aspekte wie einfache Kommunikationsschnittstellen oder einfache Verhaltensimplementierung von Bedeutung.
- **Anwender:** Erstellen Aktionspläne auf Basis gewünschter Endzustände der Umgebung. Anwender können sowohl menschlich, als auch in Form von autonomen Planungssystemen auftreten. Aus Sicht der Anwender ist entscheidend, dass Skills verständlich parametrierbar und in unterschiedlichen Szenarien flexibel einsetzbar sind.

Bei der Implementierung müssen diese berücksichtigt werden.

4.3.3 Anforderungen

Auf Basis der Zielsetzung, der beschriebenen Stakeholder, der Rahmenbedingungen dieser Arbeit sowie des in Kapitel 3 dargestellten Standes der Wissenschaft und Technik werden im Folgenden Anforderungen an die zu entwickelnden Skills sowie an die umgebende Architektur abgeleitet. In Anlehnung an ISO 29148 werden funktionale und nicht-funktionale Anforderungen unterschieden. Die Anforderungen sind in Tabelle 4.2 und Tabelle 4.3 zusammengefasst. Sie werden fortlaufend nach dem Schema FA-... für funktionale Anforderungen und NFA-... für nicht-funktionale Anforderungen referenziert.

Zentrales Ziel ist die Unterstützung einer aufgabenorientierten Programmierung (Z-2). Hierfür ist zunächst erforderlich, dass Composite Skills Parameter entgegennehmen, die in diesem Kontext zu erwarten sind, etwa Referenzen auf Objekte, Zielzustände oder prozessbezogene Optionen (FA-1). Damit die Skills in unterschiedlichen Umgebungen wiederverwendbar sind und nicht alle Detailinformationen direkt vom Anwender vorgegeben werden müssen, soll das System ein Wissensmodell enthalten, über das auf Umgebungsinformationen zu Objekten, Ressourcen und Orten zugegriffen werden kann (FA-2).

Die Skills müssen ihrerseits mit diesem Wissensmodell kommunizieren können (FA-3). Aus Sicht der Skill-Entwickler bedeutet dies, dass sowohl das Lesen als auch das Zurückschreiben von Informationen über klar definierte Schnittstellen mit vertretbarem Aufwand möglich sein muss. Beispiele hierfür sind das Auflösen symbolischer Referenzen in konkrete Objektpositionen oder Greifposen sowie das Aktualisieren von Objektzuständen nach erfolgreicher Ausführung eines Skills.

Um der in Z-3geforderten Flexibilität und Wiederverwendbarkeit gerecht zu werden, müssen Skills eine Vielzahl unterschiedlicher Parameter und Parametertypen für die Parametrisierung zulassen (FA-4). Hierzu zählen neben numerischen Werten auch symbolische Referenzen auf Einträge im Wissensmodell sowie weitere Datentypen, etwa Dateipfade. Darüber hinaus sollen für zentrale Eingaben alternative Parametrisierungsvarianten unterstützt werden, sodass äquivalente Informationen je nach Anwendungsfall auf unterschiedliche Weise übergeben werden können. Dadurch erhöht sich unter anderem auch die Flexibilität und Wiederverwendbarkeit für Entwickler bei der Nutzung dieser Skills in anderen Composite Skills. Für die Abbildung der in Z-1adressierten Montageoperationen sind teils komplexe Verhaltensweisen erforderlich, die nicht sinnvoll durch einfache Skripte oder monolithische Ablaufpläne beschrieben werden können. Daher sollen Composite Skills in der Lage sein, komplexes Verhalten intern auf einfach Weise zu modellieren. Im Rahmen dieser Arbeit soll dies mittels Verhaltensbäumen umgesetzt werden. Hierfür soll eine Einheitliche Verhaltensbaum-Logik implementiert werden (FA-5). Dies unterstützt Entwickler bei der Implementierung neuer Skills. Um dieses Verhalten modular zu strukturieren, müssen Composite Skills mit untergeordneten Skills kommunizieren können (FA-6). Dies umfasst insbesondere das Aufrufen anderer Skills, deren Parametrisierung sowie die Auswertung ihrer Rückgaben. Darüber hinaus muss der Composite Skill seinen eigenen Ausführungszustand (z. B. `Hold`, `Unhold`, `Stop`) an die Sub-Skills weitergeben, damit diese konsistent auf Zustandsänderungen reagieren können. Die Fähigkeit zur Skill-Komposition ist zugleich eine Voraussetzung für die Integration in die bestehende Softwarearchitektur (Z-4).

Neben diesen funktionalen Aspekten ergeben sich aus der Zielsetzung und den Stakeholdern weitere nicht-funktionale Anforderungen. Um die in Z-3geforderte Flexibilität zu erreichen, sollen Skills von außen konfigurierbar sein (NFA-1). Zudem sollen Skills möglichst unabhängig von der konkreten Roboterhardware und der zugrunde liegenden Steuerungstechnik funktionieren (NFA-2), sodass ein Wechsel der Hardware primär in der Treiber- und Bewegungsplanungsschicht stattfindet und nicht in der Skillschicht selbst.

Sicherheitsaspekte spielen ebenfalls eine zentrale Rolle. Skills sollen für eine sichere und kollisions-

freie Bewegung des Roboters sorgen (NFA-3). Dies umfasst unter anderem die geeignete Konfiguration von Planungskomponenten wie MoveIt 2 sowie das Erkennen und angemessene Behandeln von Fehlerzuständen der Bewegungsplanung (z. B. wenn keine kollisionsfreie Trajektorie gefunden werden kann).

Schließlich sollen die Skills robust gegenüber unvollständigen Eingaben sein (NFA-4). Parameter, die nicht zwingend für den Prozess erforderlich sind oder zur Laufzeit ermittelt werden können, sollen entfallen dürfen. In solchen Fällen sollen Skills entweder auf sinnvolle Standardwerte zurückgreifen oder zur Laufzeit Methoden verwenden, um geeignete Werte zu bestimmen. Dies erhöht die Flexibilität und Wiederverwendbarkeit (Z-3) für Anwender, die nicht alle Details explizit parametrisieren möchten oder können.

ID	Beschreibung	Ziel
FA-1	Composite Skills müssen Parameter entgegennehmen, die im Sinne einer aufgabenorientierten Programmierung zu erwarten sind (z. B. Referenzen auf Objekte, Zielzustände oder einfache Prozessoptionen).	Z-2
FA-2	Das System soll ein Wissensmodell beinhalten, über das auf relevante Umgebungsinformationen (z. B. Objektpositionen, Greifposen, Pfade) zugegriffen werden kann.	Z-2, Z-3
FA-3	Skills müssen mit dem Wissensmodell kommunizieren können. Dies umfasst für Skill-Entwickler sowohl einfache Lesezugriffe (z. B. Auflösen symbolischer Referenzen in konkrete Posen) als auch das Zurückschreiben von geänderten Objekt- oder Systemzuständen nach Ausführung eines Skills.	Z-2, Z-3
FA-4	Skills müssen eine Vielzahl unterschiedlicher Parameter und Parametertypen für die Parametrisierung zulassen (z. B. numerische Werte, symbolische Referenzen, Dateipfade) und alternative Parametrisierungsvarianten für dieselbe Information unterstützen.	Z-2, Z-3
FA-5	Innerhalb von Composite Skills soll komplexes Verhalten implementiert werden können, in Form von Verhalensbäumen.	Z-1
FA-6	Composite Skills sollen mit untergeordneten Skills kommunizieren können. Dies umfasst den Aufruf und die Parametrisierung anderer Skills, die Auswertung ihrer Rückgaben sowie die Weitergabe des eigenen Ausführungszustands (z. B. `Hold`, `Unhold`, `Stop`) an die Sub-Skills.	Z-4

Tabelle 4.2: Funktionale Anforderungen an Skills und Architektur

Die nicht-funktionalen Anforderungen werden in dieser Arbeit entlang der in ISO/IEC 25010 beschriebenen Qualitätsmerkmale strukturiert. Relevante Kategorien sind insbesondere Benutzbarkeit, Zuverlässigkeit, Portierbarkeit und Sicherheit.

ID	Kategorie	Beschreibung	Ziel
NFA-1	Benutzbarkeit	Skills sollen von außen konfigurierbar sein, um unterschiedlichen Anforderungen und Szenarien gerecht zu werden (z. B. durch Parameter, Konfigurationsdateien oder Einträge im Wissensmodell).	Z-3
NFA-2	Portierbarkeit	Skills sollen möglichst unabhängig von der verwendeten Roboterhardware und Steuerungstechnik funktionieren.	Z-3
NFA-3	Sicherheit	Skills sollen für eine sichere und kollisionsfreie Bewegung des Roboters sorgen. Fehlerzustände der Bewegungsplanung (z. B. keine kollisionsfreie Bahn gefunden) müssen erkannt und angemessen behandelt werden.	Z-1, Z-2
NFA-4	Zuverlässigkeit	Skills sollen robust gegenüber unvollständigen Eingaben sein. Es sollen optionale Parameter entfallen dürfen, ohne dass der Skill unbrauchbar wird. Außerdem sollen die Skills sich wenn möglich aus Fehlerzuständen selbst hinaus manövrieren.	Z-3

Tabelle 4.3: Nicht-funktionale Anforderungen an Skills und Architektur

4.3.4 Randbedingungen

Die im Rahmen dieser Arbeit entwickelten Skills werden ausschließlich virtuell validiert. Eine physikalische Simulation wird nicht durchgeführt; stattdessen wird der Bewegungsablauf in *RViz* (MoveIt-Visualisierung) überprüft, insbesondere hinsichtlich Bewegungsablauf, Trajektorienausführbarkeit und Kollisionserkennung. Kontaktkräfte und dynamische Effekte werden dabei nicht abgebildet. Der Verzicht auf eine Simualtion ist durch praktische Integrationsgrenzen begründet, insbesondere durch eine nicht hinreichend verfügbare realistische Modellierung des verwendeten Greifers.

Für die Skills stehen keine sensorischen Informationen zur Verfügung (z. B. Kameras zur Posenerkennung oder Messgrößen zur Kraft-/Drehmomenterfassung). Folglich wird keine Drehmoment- bzw. Kraftregelung verfügbar sein.

Weiterhin wird angenommen, dass erforderliche Posen und Prozessparameter bereits vorab bestimmt und im Wissensmodell hinterlegt sind. Auch die Bestimmung von Greifposen ist nicht Teil dieser Arbeit.

4.4 Konzeption

Im folgenden soll konzipiert werden, wie bei der Implementierung vorgegangen werden soll. Basierend auf den vorgestellten Konzepten sollen danach Anforderungen an ein Wissensmodell gestellt werden.

4.4.1 Pick-and-Place

Auswahl

Als repräsentativer Skill aus *VDI-Richtlinie 2860* (1990) wird ein Pick-and-Place-Skill umgesetzt. Pick-and-Place zählt in der Forschung zu den zentralen Fähigkeiten von Robotern (Heuss et al., 2022a). Zudem bildet Pick-and-Place die Grundlage, um weitere Teilfunktionen des Handhabens gemäß *VDI-Richtlinie 2860* (1990) zu realisieren. Teilfunktionen wie Speichern oder das Verändern von Mengen sind auf einer höheren Abstraktionsebene angesiedelt, da hierbei häufig Zustandsänderungen von Objektmengen im Vordergrund stehen. Voraussetzung dafür ist jedoch stets die Fähigkeit, einzelne Objekte gezielt aufzunehmen und abzulegen. Daher wird ein entsprechender Skill implementiert.

Das Pick-and-Place wird in drei Skills gegliedert, einen Skill zum Aufnehmen (engl. Pick), einen Skill zum Ablegen (engl. Place) und einen übergeordneten Skill, der beide Schritte zu einem Pick-and-Place Prozess verbindet. Diese Aufteilung ist sinnvoll, da Montageprozesse auftreten können, bei denen ein Roboter ein Objekt aufnimmt und anschließend in einer definierten Pose hält, sodass beispielsweise ein weiterer Roboter das Objekt lackieren kann. Erst im Anschluss erfolgt das Ablegen. Im Folgenden wird die Methodik für den Gesamtprozess des Pick-and-Place erarbeitet. Das Vorgehen für die einzelnen Skills ergibt sich daraus.

Modellierung über *VDI-Richtlinie 2860* (1990)

Unter Berücksichtigung der Randbedingungen aus Abschnitt 4.3.4 wird deutlich, dass beim Pick-and-Place aufgrund der reinen Positionsregelung ein Objekt aus einer vordefinierten Pose in eine neue vordefinierte Pose überführt wird. In *VDI-Richtlinie 2860* (1990) wird diese Grundbewegung als Weitergeben bezeichnet. Grundsätzlich wird dieser Anteil bereits durch den Primitiven PTP-Skill abgedeckt. Zusätzlich muss das Objekt jedoch durch den Greifer gesichert werden, da der Primitive Skill zunächst ausschließlich den TCP führt. Zudem ist es erforderlich, den Greifer zunächst an das Objekt heranzuführen. Für eine vollständige Modellierung kann dieser Ablauf als Handhaben des Greifers verstanden werden. Nach *VDI-Richtlinie 2860* (1990) lässt sich der Bewegungsablauf daher wie folgt beschreiben.

Weitergeben (Greifer) → Halten → Weitergeben (Objekt) → Lösen

Modellierung über die MTM Grundbewegungen nach Lotter und Wiendahl (2012)

Die MTM Analyse beschreibt den Pick-and-Place Prozess stärker aus der Sicht des Roboters. Nach MTM kann der Prozess durch folgende Grundbewegungen beschrieben werden.

Hinlangen → Greifen → Bringen → Fügen → Loslassen

Praktischer Bewegungsablauf

Beide zuvor genannten Modellierungsansätze fassen die Annäherung an das Objekt und das Weitergeben als einen zusammenhängenden Bewegungsablauf auf. In der Praxis ist dies mit den vorhandenen Primitiven Skills jedoch nicht direkt umsetzbar, da die Bewegung über einen MoveIt 2 Planer generiert wird. Unmittelbar vor dem Greifen und vor dem Loslassen befindet sich der Roboterarm in unmittelbarer Nähe zu Kollisionsobjekten. Dadurch kann es für Planungsalgorithmen bei PTP-Bewegungen schwierig sein, in angemessener Zeit eine kollisionsfreie Trajektorie zu finden. Für andere Bewegungsarten wie lineare Bewegungen kann unter Umständen keine Lösung existieren. Daher ist es sinnvoll, Bewegungen zu unterteilen in Bewegungen durch große Bereiche des Arbeitsraums und in kurze, präzise Bewegungen zur Zielpose (Owen-Hill, 2019). Für den zu implementierenden Pick-and-Place-Skill ergibt sich daraus folgender Bewegungsablauf.

Bewegen → Annähern → Greifen → Zurückziehen → Bewegen → Annähern → Loslassen → Zurückziehen

Das Zurückziehen ist erforderlich, damit anschließende großräumige Bewegungen zuverlässig planbar bleiben. Bereits die Nähe zur Ablageumgebung kann die Trajektorienplanung erschweren. Zusätzlich kann es notwendig sein, das Objekt aus einer Umgebung herauszuführen oder in diese einzuführen, etwa beim Einsetzen oder Herausziehen eines Bolzens aus einer Bohrung.

Grundlegende Posenbestimmung

Damit ein Roboter ein Objekt greifen kann, muss bekannt sein, wie der Greifer relativ zum Objekt positioniert und orientiert sein soll. Diese Pose wird im Folgenden durch die homogene Transformationsmatrix ${}^{\text{obj}}T_{\text{tcp}}$ beschrieben.

Wie bereits festgestellt, soll der Pick-and-Place-Skill ein Objekt aus einer gegebenen Pose aufnehmen und in einer neuen Pose ablegen. Die vorhandenen Bewegungsprimitiven führen den *tool0* Punkt des Roboters, also den Montagepunkt für Werkzeuge, relativ zum Weltkoordinatensystem. Folglich werden die zugehörigen Posen des *tool0* Koordinatensystems im Weltkoordinatensystem benötigt. Diese Posen lassen sich jeweils durch eine homogene Transformationsmatrix ${}^{W}T_{\text{tool0}}$ beschreiben. Diese Matrix ergibt sich aus

$$ {}^{Welt}T_{\text{tool0}} = {}^{Welt}T_{\text{obj}} \cdot {}^{\text{obj}}T_{\text{tool0}} \tag{4.1} $$

Die Ist-Pose und die Soll-Pose des Objekts in Weltkoordinaten sind häufig nur indirekt bekannt. In dem Wissensmodell können Objektposen beispielsweise relativ zu einem anderen Koordinatensystem hinterlegt sein. Der Pick-and-Place-Skill muss diese Angaben daher in Weltkoordinaten auflösen.

Die direkte Modellierung von ${}^{\text{obj}}T_{\text{tool0}}$ ist dabei unhandlich, da sie Relationen zwischen Objekt, Roboter und Greifer umfasst. Zweckmäßiger ist eine Aufteilung über den TCP des Greifers:

$$ {}^{Welt}T_{\text{tool0}} = {}^{Welt}T_{\text{obj}} \cdot {}^{\text{obj}}T_{\text{tcp}} \cdot {}^{\text{tcp}}T_{\text{tool0}} \tag{4.2} $$

Damit zerfällt die Modellierung in zwei Anteile. Zum einen beschreibt ${}^{\text{tcp}}T_{\text{tool0}}$ die Pose des *tool0* relativ zum TCP des Greifers. Diese Relation ist hardwareabhängig. Zum anderen beschreibt ${}^{\text{obj}}T_{\text{tcp}}$ die Pose des Greifers relativ zum Objekt. Diese Pose wird im Folgenden als Greifpose bezeichnet und kann als Prozessparameter verstanden werden.

Ein Objekt kann theoretisch mehrere Greifposen besitzen. Welche Greifposen kollisionsfrei ausführbar sind, hängt jedoch stark von der konkreten Umgebung und der Orientierung des Objekts ab. Für die Ist-Pose und die Soll-Pose des Objekts ergeben sich daher jeweils zulässige Mengen an Greifposen. Da das Objekt während des Weitergebens im Greifer gesichert bleibt, muss dieselbe Greifpose sowohl in der Ist- als auch in der Soll-Situation verwendbar sein. Gesucht ist daher die Schnittmenge beider Mengen. Der Pick-and-Place-Skill soll eine Greifpose aus dieser Schnittmenge auswählen.

Definition der Zwischenschritte

Wie zuvor erläutert, soll der Bewegungsablauf des Pick-and-Place-Skill sowohl beim Greifen als auch beim Platzieren nicht aus einer einzelnen PTP-Bewegung bestehen. Für den Greifprozess ergeben sich mit den vorhandenen Skills zwei mögliche Abläufe.

1. PTP-Bewegung in Objektnähe, anschließend PTP-Bewegung in die Greifpose
2. PTP-Bewegung in Objektnähe mit Greiforientierung, anschließend lineare Bewegung in die Greifposition

In dieser Arbeit wird Option zwei verwendet. Bei Option eins ist die Anzahl freier Parameter höher, da zusätzlich eine Zwischenposition und eine Zwischenorientierung festgelegt werden müssen. Durch die steigende Anzahl an Parametern wird die Parametrierung aufwendiger und daher fehleranfälliger. Zudem ist beim Platzieren in umgebende Geometrien häufig ein deterministisches Verhalten in Form einer linearen Bewegung erforderlich. Beim Einsetzen eines Passstifts in eine Bohrung ist beispielsweise eine lineare Bewegung entlang der Bohrungsachse notwendig, um Kollisionen zuverlässig zu vermeiden. Ein PTP-Bewegungsplaner, wie er bei Option eins benutzt werden würde, findet in solchen Situationen keine Lösung in angemessener Zeit.

Mit Option zwei werden zusätzlich Positionen für Annähern und Zurückziehen benötigt. Dies gilt sowohl für den Pick- als auch für den Place-Teil. Insgesamt sind damit vier weitere Positionsparameter zu bestimmen.

Weitere Prozessparameter und Konzept zur Parameterreduktion

Ein weiterer wesentlicher Prozessparameter ist die Greifkraft. Sie ist entscheidend für die zuverlässige Ausführung des Skills und hängt unter anderem von der Objektmasse, dem Abstand zum Schwerpunkt und den auftretenden Beschleunigungen ab. Bei mechanischen Greifern bezeichnet die Greifkraft die Normalkraft, mit der die Greifbacken das Objekt klemmen. Die resultierende Reibkraft, die das Objekt tatsächlich hält, hängt zusätzlich von der Kontaktfläche und vom Reibwert zwischen Objekt und Greifer ab. Weitere Prozessparameter sind die Geschwindigkeit und die Beschleunigung der Roboterbewegung.

Ein Teil der genannten Parameter ist für die Beschreibung des Pick-and-Place unvermeidlich, etwa Ist-Pose, Soll-Pose und Greifpose. Andere Parameter, insbesondere die vier Annäherungs- und Rückzugpositionen, entstehen vor allem aus funktionalen Anforderungen. Daher liegt es nahe, hier eine Parameterreduktion zu prüfen, um die Komplexität zu verringern.

Ein möglicher Ansatz wäre, Annäherungsposition und Rückzugposition gleichzusetzen. Dies steht jedoch im Konflikt zu Z-3, der Wiederverwendbarkeit der Skills, da es Szenarien gibt, in denen die Zugangsrichtung und die Hubrichtung nicht identisch sein dürfen. In einer einfachen Situation kann der Greifer von oben linear anfahren, greifen und anschließend entlang derselben Richtung

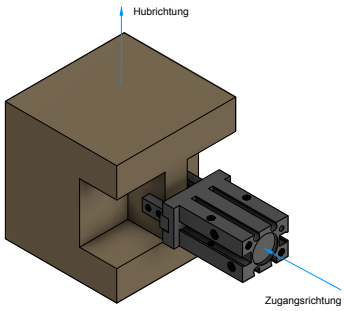


Abbildung 4.3: Greifprozess mit nicht parallelen Zugangs und Hubvektoren

zurückkehren. In anderen Situationen ist dies nicht möglich, etwa wenn das Objekt auf einer ebenen Platte aufliegt und nur eine von der Seite zugängliche Greifpose vorliegt. Würde hier die Hubrichtung parallel gesetzt werden mit der Zugangsrichtung, würde das gegriffene Objekt über die Oberfläche schleifen. Auch weil es sein könnte, dass die Kollisionserkennung diesen Kontakt nicht zuverlässig als Kollision bewertet. Somit ist beim Greifen die Hubrichtung entscheidend, da das Objekt aus einem Kontaktzustand mit der Umgebung gelöst werden muss. Auch das umgekehrte Parallelsetzen der Zugangsrichtung mit der Hubrichtung funktioniert nicht immer, da eine Greifpose nicht zwingend über die Hubrichtung zugänglich ist. Beim Platzieren ergibt sich das Problem analog, allerdings mit vertauschter Rolle von Zugang und Hubrichung. Eine Parameterreduktion durch Gleichsetzen der Raumpunkte bzw. Parallelsetzen der Richtungen ist daher nicht allgemein möglich. Abbildung 4.3 illustriert dieses Problem.

Stattdessen bietet sich eine Aufteilung der Annäherungs- und Rückzugpunkte in Richtung und Distanz an. Mithilfe der Bewegungsplanung und der Kollisionserkennung von MoveIt 2 können verschiedene Distanzen heuristisch getestet werden. Zusätzlich können bei linearen Bewegungen ohne Objekt im Greifer auch verschiedene Richtungen ausprobiert werden, da ein Entlangschleifen ausgeschlossen ist und Kollisionen in der Regel zuverlässig erkannt und in der Planung verworfen werden können.

Zusammenfassung

Es wird ein Pick-and-Place-Skill implementiert, der intern jeweils einen Pick- und einen Place-Skill aufruft. Der Skill soll nach *VDI-Richtlinie 2860* (1990) das Weitergeben eines Objekts in einer großen Bandbreite von Situationen ermöglichen. Die Grundbewegung des gesamten Pick-and-Place umfasst dabei acht Aufrufe von Primitiven Bewegungs und Greifskills.

Für die Ausführung werden die folgenden grundlegenden Parameter benötigt.

- **Ist-Pose**: Aktuelle Pose des Objekts in Weltkoordinaten
- **Soll-Pose**: Zielpose des Objekts in Weltkoordinaten
- ${}^{\text{obj}}\boldsymbol{T}_{\text{tcp}}$: Greifpose, die sowohl in Ist-Pose als auch in Soll-Pose zulässig ist
- ${}^{\text{tcp}}\boldsymbol{T}_{\text{tool0}}$: Pose des *tool0* Koordinatensystems relativ zum TCP des Greifers
- **Zugangsrichtung und Distanz**: Richtung und Distanz der linearen Annäherung beim Greifen

- **Hubrichtung und Distanz**: Richtung und Distanz des linearen Zurückziehens nach dem Greifen
- **Annäherungsrichtung und Distanz**: Richtung und Distanz der linearen Annäherung beim Platzieren
- **Rückzugsrichtung und Distanz**: Richtung und Distanz des linearen Zurückziehens nach dem Platzieren
- **Greifkraft**: Kraft, mit der das Objekt festgehalten wird

Ein Teil dieser Parameter muss nicht zwingend extern vorgegeben werden, sondern kann innerhalb des Skills aus weiteren Eingaben abgeleitet oder heuristisch bestimmt werden.

- **Zugangs und Rückzugsrichtungen**: Richtungen können durch Ausprobieren bestimmt werden, da Kollisionen durch die Planung in der Regel erkannt und verworfen werden
- **Distanzen der linearen Bewegungen**: Distanzen können heuristisch variiert und über Kollisionserkennung validiert werden
- **Greifkraft**: Greifkraft kann aus Objektinformationen, etwa der Masse, abgeleitet werden

4.4.2 Füge-Skill nach *DIN 8593-0* (2003)

Auswahl des Fügeverfahrens nach DIN 8593

Nach *DIN 8593-0* (2003) werden die Fertigungsverfahren der Hauptgruppe »Fügen« in verschiedene Untergruppen unterteilt. Für die vorliegende Arbeit ist daraus ein Prozess auszuwählen, der in das skillbasierte Programmiermodell integrierbar ist und mit den vorhandenen Ressourcen ohne Kraftregelung umgesetzt werden kann.

Ein naheliegendes Einsatzfeld von Industrierobotern ist die Montage, insbesondere das An- oder Einpressen von Bauteilen nach *DIN 8593-3* (2003). Zu dieser Gruppe gehören unter anderem Schraubverbindungen und Pressverbindungen. Den Verfahren ist gemeinsam, dass sie Prozessparameter voraussetzen, die Kraft oder Drehmoment explizit enthalten. Wie in Abschnitt 4.3.4 beschrieben, steht in der verwendeten Versuchsumgebung jedoch keine Kraft- oder Drehmomentregelung zur Verfügung. Damit scheiden An- und Einpressprozesse nach *DIN 8593-3* (2003) als Demonstrationsprozess für diese Arbeit aus.

Eine weitere, für die Industrierobotik relevante Untergruppe ist das Fügen durch Schweißen nach *DIN 8593-6* (2003). Aus Sicht der Skill-Modellierung reduzieren sich viele Schweißanwendungen zwar auf die Kombination aus Bahnbewegung und synchronisierter Werkzeugaktion; die prozessspezifische Parametrierung und die notwendige Peripherie (Schweißquelle, Brenner, Schutzgasführung etc.) sind im vorliegenden Aufbau jedoch nicht verfügbar. Eine belastbare Umsetzung und Bewertung eines Schweißprozesses ist daher im Rahmen dieser Arbeit nicht vorgesehen.

Stattdessen wird ein Dosier- und Auftragprozess gewählt, der dieselbe strukturelle Anforderung abbildet, in form einer definierten Bahnbewegung mit synchronisierter Werkzeugaktion. Eine geeignete Einordnung findet sich im Fügeverfahren Kleben nach *DIN 8593-8* (2003). In der Untergruppe »Kleben mit chemisch abbindenden Klebstoffen« lässt sich der Gesamtprozess in einen Auftragsvorgang und einen anschließenden Füge- und Aushärteprozess gliedern. Der hier betrachtete Teilprozess besteht im kontrollierten Auftragen eines Flüssigklebstoffs entlang einer vorgegebenen

Bahn auf einem Bauteil. Da dabei keine relevanten Kontaktkräfte zwischen Werkzeug und Bauteil auftreten, ist keine Kraftregelung erforderlich. Der Prozess eignet sich daher für die Umsetzung im vorhandenen Robotersystem.

Im Rahmen dieser Arbeit wird also ein *Dosier*-Skill definiert, der eine Menge geordneter Posen als Bahn beschreibt und entlang dieser Bahn synchron Klebstoff aufträgt. Im Mittelpunkt steht die Modellierung und Ausführung der Bahnbewegung, während der Materialauftrag abstrahiert wird. Hierzu werden zwei Primitive benötigt.

1. eine Bewegungsprimitive, die eine geordnete Liste kartesischer Posen entgegennimmt und als kontinuierliche Bahn ausführt,
2. eine Werkzeugprimitive, die die Ressource zum Klebstoffauftrag mit geeigneten Parametern ansteuert

Da in der vorhandenen Softwareumgebung weder ein Modell eines Schweißsystems noch eines Dosiersystems zur Verfügung steht, wird die Werkzeugprimitive als Dummy-Primitive implementiert. Sie abstrahiert das Verhalten eines Materialauftragssystems und ermöglicht die spätere Integration realer Ressourcen.

Bewegungsablauf

Wie zuvor erläutert, muss der Dosier-Skill einen definierten Pfad abfahren und vor sowie nach der Bahnausführung die Dosier-Primitive ein- und ausschalten. Vor Beginn der eigentlichen Bahnbewegung wird zunächst eine PTP-Bewegung in die Startpose des Pfades ausgeführt.

Bei diesem Skill ist die zeitliche Koordination besonders entscheidend. Die Zeitspanne zwischen dem Aktivieren des Dosierers und dem Start der Bahnbewegung sollte möglichst klein sein, da sonst am Startpunkt übermäßig viel Material aufgetragen wird. Daher ist es sinnvoll, die Bahnplanung von der Ausführung zu entkoppeln. Zunächst wird die Bahn vollständig geplant und zeitparametrisiert. Anschließend werden Dosierer-Aktivierung und Bahnbewegung zeitlich abgestimmt ausgeführt.

Parameterdefinition

Der Dosier-Skill benötigt als Eingangsparameter eine Beschreibung der abzufahrenden Bahn. Die einfachste Form dieser Darstellung ist eine geordnete Liste von Posen. Diese Posen sind im Objektkoordinatensystem definiert, da davon ausgegangen wird, dass sie außerhalb des betrachteten Systems typischerweise in dieser Form bestimmt werden. Im Rahmen der Parametervorbereitung transformiert der Skill die Posen in das Weltkoordinatensystem und überführt sie analog zum Pick-and-Place-Skill vom TCP des Werkzeugs auf den Roboterreferenzpunkt *tool0*.

Unter Berücksichtigung von FA-4sollen die entwickelten Skills unterschiedliche Parametertypen zulassen. Daher wird als zweite Option eine Parametrierung über den Verweis auf eine `.nc`-Datei vorgesehen, die G-Code enthält. Dieser könnte innerhalb eines CAM-Programms erstellt werden. G-Code ist zwar Verbunden mit einigen grundsätzlichen Probleme, wie dass maschinenspezifische CAM-Postprozessoren maschinenabhängigen Code produzieren (Latif et al., 2021). Allerdings dient er gut als einfache Demonstrationsmöglichkeit, wie Parameter aus Produktdesign bzw. Fertigungsdaten abgeleitet werden können. Entsprechend wird ein Skript implementiert, das den G-Code einliest und in eine Liste von Roboterposen überführt.

Für die Ausführung des Dosier-Skills ist ein Parameter zur Steuerung der Vorschubgeschwindigkeit notwendig. Zusätzlich wird dem Dosier-Skill eine Volumenstromvorgabe als Parameter übergeben.

Dieser Parameter dient in der vorliegenden Arbeit primär der Demonstration, da wie zuvor beschrieben kein physikalisches Modell eines Dosierwerkzeugs zur Verfügung steht.

4.4.3 Ableitung eines Wissensmodell-Konzepts

Nach den zuvor definierten Anforderungen brauchen die Skills insbesondere für das Umsetzen einer aufgabenorientierten Programmierung ein Umgebungsmodell, in welchem Informationen über die Umgebung gespeichert werden soll. Mit der Nutzung von ROS2 bietet es sich hier an dieses in Form einer eigenen Node umzusetzen. Wichtig ist, dass Skills einen kontinuierlichen Zugriff auf das Modell haben. Das Publizieren über ein ROS Topic bietet sich hier somit an, da über dieses mehrere Nodes gleichzeitg Zugriff auf dieses haben können. Weiterhin müssen Skills das Modell editieren können. Da es sich hier nicht um lang andauernde Aktionen handelt, soll hierfür ein klassischer ROS Service erstellt werden.

Das Untersuchen einer geeigneten Modellierungsstruktur der Umgebung ist nicht Ziel dieser Arbeit. Der Fokus liegt auf der Implementierung der Skills. Dennoch ist es für die Skills im Sinne der Wiederverwendbarkeit und Flexibilität (Z-3) wichtig, dass das Modell Wissen möglichst nicht anwendungsspezifisch speichern soll. Bei der Implementierung soll dies berücksichtigt werden.

Kapitel 5 Implementierung

5.1 Einführung eines Wissensmodell

Nun wird aufgezeigt, wie die Implementierung des Wissensmodell umgesetzt worden ist. Diese Untergliedert sich in die Modellierung der Umgebung, die Verwaltung und Integration des Modells in ROS2 als Node, sowie das Erstellen einer Client-seitigen Schnittstellenklasse, welche Skills einfachen Zugang zu dem Datenmodell gibt.

5.1.1 Umgebungsmodell

Um die Umgebung im Wissensmodell abzubilden, wird ein objektorientiertes Datenmodell verwendet. Die Umgebung wird als Teilmenge einer Montagelinie verstanden, zum Beispiel als einzelne Roboterzelle. In ihr können sich mehrere Objekte, Ressourcen, Kombinationen von Ressourcen und Koordinatensysteme (fortlaufend Frames genannt) befinden. Alle diese Klassen erben von einer Basisklasse, die ihnen gemeinsame Attribute wie eine eindeutige ID zuweist.

Die Wahl der Terminologie orientiert sich an den in Abschnitt 3.1.1 vorgestellten Modellen. Diese stellen Skills als Verknüpfung von Produkt, Prozess und Ressource dar. Durch diese Definition ist gewährleistet, dass das Modell möglichst allgemein einsetzbar ist. Ressourcen sind bewusst breit gefasst und beziehen sich nicht nur auf Roboter oder deren Werkzeuge, sondern können auch andere in Montagezellen vorkommende Maschinen umfassen.

Um sicherzustellen, dass Elemente des Umgebungsmodells den erwarteten Daten- und Modelltypen entsprechen, wird die Bibliothek Pydantic eingesetzt. Pydantic ermöglicht die Definition und Validierung strukturierter Datenmodelle auf Basis von Type Hints. Beim Instanziieren dieser Modelle werden Eingabedaten anhand der Typannotationen geprüft, bei Bedarf konvertiert und anschließend in standardisierte Formate wie Python-Objekte oder JSON überführt („Pydantic“, 2025).

Objekte können entweder einzelne Teile (Parts) oder Zusammenbauten (Assemblies) sein. Beide Klassen erben von einer gemeinsamen Objektklasse. Diese besitzt verschiedene physikalische Attribute wie das Gewicht eines Objekts oder einen Vektor, der die Position des Masseschwerpunkts beschreibt. Gewicht und Schwerpunktlage können von Handhabungs-Skills genutzt werden, beispielsweise zur Überprüfung, ob das Gewicht des Objekts über dem zulässigen Maximal-Gewicht der jeweiligen Ressource liegt.

Weiterhin gibt es innerhalb des Umgebungsmodells Frames. Frames beschreiben die Pose von Elementen relativ zu einem anderen Frame. Sie besitzen ein String-Attribut welches die ID des Referenz-Frames enthält. Durch die Referenz ist sichergestellt, dass jede Pose in Weltkoordinaten aufgelöst werden kann. Jedes Objekt besitzt einen Basis-Frame, über welchen die Objektpose beschrieben wird.

Zusätzlich besitzt jedes Objekt die Attribute `Mesh` und `CAD-Datei`. Dies sind einfache Strings, die als Dateipfade auf eine STL- bzw. CAD-Datei des entsprechenden Objekts dienen. Eine `Mesh`-Datei wird benötigt, um das Objekt später geometrisch korrekt als Kollisionsobjekt in die MoveIt 2 Planungsszene einzubinden. Die CAD-Datei kann von anderen Skills genutzt werden, beispielsweise als Ausgangspunkt für die Bestimmung von Greifposen. Weiterhin hat jedes Objekt ein Hüllkörper-Attribut, im welchem die maximalen Objektabmessungen gespeichert sind.

Neben diesen geometrischen und physikalischen Attributen besitzen Objekte auch Prozessattribute, wie die Pick- und Place-Frames. Diese sind Listen, die aus `GraspFrame`-Objekten bestehen. `GraspFrame`-Objekte sind spezielle Frames, die für das Greifen mit einem Greifer definiert sind. Die Pick-Frame-Liste enthält aktuell mögliche Greifposen für das Objekt in seiner Ausgangskonfiguration, während die Place-Frame-Liste Greifposen enthält, die anhand des Zielzustandes des Objektes bestimmt wurden. `GraspFrame` erbt von `Frame` und wird um einige Attribute erweitert. Den Frames werden über ein String Attribut IDs zugeordnet die auf den Greifer zeigen für welchen die Greifpose ermittelt wurde. So kann sichergestellt werden, dass keine Kollsionen entstehen aufgrund nicht zusammenpassender Greifposen und Greifer. Außerdem enthalten die `GraspFrame`-Objekte, die in Abschnitt 4.4.1 konzipierten Richtungen und Distanzen sowie zusätzliche weitere Attribute, wie Greifweite oder Greifkraft.

Ressourcen beschreiben im Modell alle verfügbaren technischen Mittel. Ihre Pose in der Umgebung ist ebenfalls durch einen Basis-Frame definiert. Weiterhin besitzen sie das Attribut `TCP` für die Pose des Tool-Center-Point. Insbesondere Handhabungs-Ressourcen wie Greifer kommen in direkten Kontakt mit Objekten. Innerhalb einer MoveIt 2 Planungsszene würde dies zu Kollisionen führen. Soll ein Objekt in Kontakt mit einer Ressource kommen, muss daher die Kollisionsprüfung zwischen Ressource und Objekt deaktiviert werden. Dafür sind die Link-Namen der spezifischen Ressource erforderlich, weshalb ein Dateipfad zur URDF-Datei bereitgestellt wird.

Werden in der Umgebung Operationen über Skills ausgeführt, benötigen die Skills den entsprechenden Namen des Action Servers, den sie ansprechen müssen, um einen bestimmten Skill aufzurufen. Darüber hinaus können spezifischere Ressourcen hinzugefügt werden, etwa Roboter oder Greifer. So können zusätzliche Informationen gespeichert werden, wie beispielsweise im Falle eines Roboters die maximal zulässige Traglast.

Neben den einzelnen Ressourcen können auch Ressourcenkombinationen definiert werden. Diese Ressourcenkombinationen können Composite Skills bereitstellen. Ein Beispiel hierfür ist ein Handhabungs-Roboter. Dieser besteht mindestens aus einem Roboter und einem Greifer. In einer Ressourcenkombination werden die IDs der enthaltenen Ressourcen gespeichert.

Schließlich besitzt eine Umgebung noch Frames, also Koordinatensysteme die markante Orte markieren oder bei der Konfiguration zur einfacheren Posenangabe der Objekte dienen können.

5.1.2 Node

Das Wissensmodell ist innerhalb einer eigenen ROS Node gespeichert. Beim Starten der Node kann ein Datei-Pfad zu einer Initialisierungs Datei mitgegeben werden. Das Wissensmodell wird dann periodisch durch ein ROS Topic an andere Nodes als JSON-String veröffentlicht. Die Node besitzt einen Server über welchen das Wissensmodell modifiziert werden kann. Skills wird so die Möglichkeit gegeben die drei Basis Operationen `add, remove und set` am Wissensmodell durchzuführen. Diese Operationen beziehen sich immer auf einem ganzen Element, also ein Element der Kategorie `Objekt, Ressource, Ressourcen Kombination oder Frames`. Es können nicht explizit einzelne Attribute dieser Kategorien geändert werden sondern nur das ganze Objekt. Außerdem besitzt die Node Funktionalitäten um das Wissensmodell mit der MoveIt 2 Planungsszene

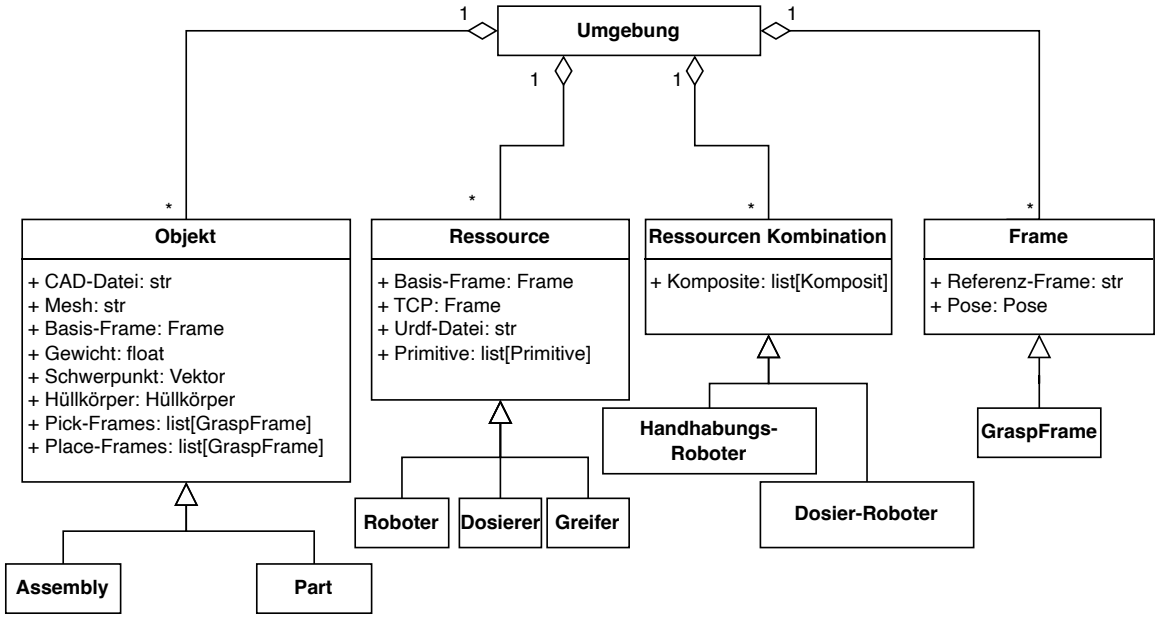


Abbildung 5.1: Abstrahieter Ausschnitt des verwendeten Umgebungsmodells

zu synchronisieren. Dabei wird im Falle, dass das `set, add oder remove` Element ein Objekt ist, die gleiche Änderung ebenfalls in der Planungsszene durchgeführt. Sollte der Referenz-Frame des Objektes auf einen Frame zeigen, der zu einem TCP einer Ressource gehört, so wird das Objekt in der Planungsszene als an der Ressource befestigtes Objekt eingetragen. Dadurch ist sichergestellt, dass die Kollisionsüberprüfung von MoveIt 2 das Objekt berücksichtigt bei der Planung von Bewegungen. Zu dem werden über die URDF-Referenz alle Link Namen der Ressource gesammelt. Die Planungsszene wird dann so konfiguriert, dass das Objekt mit diesen Links kollidieren darf. Andernfalls würde MoveIt 2, beispielsweise beim Greifen eins Objekts, eine Kollision erkennen die dazu führt, dass weitere Bewegungen nicht geplant werden können.

5.1.3 Schnittstelle

Als einheitliche Unterstützung für die Skills wurde eine Schnittstellenklasse implementiert. In dieser Klasse wird ein Service-Client der Wissensmodell-Node erstellt, um die zuvor genannten Operationen `set, add und remove` am Wissensmodell durchzuführen. Dadurch können Entwickler innerhalb eines Skills in einer Programmzeile das Umgebungsmodell verändern. Weiterhin hört die Klasse dem Topic zu, über welches das Wissensmodell veröffentlicht wird. Der JSON-String wird dann wieder in das Pydantic-Modell umgewandelt und als Attribut der Schnittstellen Klasse gespeichert. Dadurch haben Skills bzw. Entwickler sehr schnell und einfach die Möglichkeit auf Elemente im Wissensmodell zuzugreifen. Außerdem wurden innerhalb der Schnittstellen Klasse noch Helfer-Funktionen implementiert, die genutzt werden können, um schnell Informationen aus dem Wissensmodell zu bekommen. So existiert beispielsweise eine Funktion, die dazu dient einen einzelnen Frame im Wissensmodell anhand seiner ID zu suchen und als Rückgabewert zurück zu geben. Dieser Helfer wird insbesondere regelmäßig benötigt, wenn Posen in andere Koordinaten Systeme Rücktransformiert werden müssen. Weiterhin existieren Helfer-Funktionen für häufig verwendete geometrische Rechnungen. Diese Helfer-Funktionen sorgen dafür, den Aufwand für Skill Entwickler erheblich zu reduzieren, so dass diese sich mehr auf die Skill Funktionalität selber kon-

zentrieren können.

5.2 Erweiterungen am skillbasiertem System

Zur Umsetzung der in Kapitel 4.3.3 abgeleiteten Anforderungen wurden neben der Implementierung der Montage-Skills auch Erweiterungen am bestehenden skillbasierten System vorgenommen. Diese betreffen Erweiterungen an der Basisklasse für Composite Skills sowie Anpassungen der Primitiven Skills, um deren Benutzbarkeit für die spätere Wiederverwendung in den Composite Skills zu verbessern.

5.2.1 Implementierung einer Basisklasse für Composite Skills

Im Rahmen der Implementierung der Verhaltensbaum-Funktionalitäten sowie der Kommunikation mit dem Wissensmodell, wurde die Basisklasse der Composite Skills um eine verhaltensbaumbasierte Ausführung und Mechanismen zum Aufruf und zur Steuerung untergeordneter Skills erweitert. Ergänzend wurden zwei Blattknoten eingeführt, die den Verhaltensbaum mit der bestehenden Applet- und ROS-2-Infrastruktur verbinden.

Bei der Initialisierung eines Skills werden nun zusätzlich zur Wissensmodell-Schnittstelle eine Verhaltensbaum-Referenz sowie ein Blackboard angelegt. Das Blackboard dient als gemeinsame Ablage für den Namen eines aufzurufenden Skills, zugehörige Parameter und spätere Ergebnisse. Für den Verhaltensbaum und das Blackboard wird py_trees verwendet (Stonier, n. d.).

Die Ausführung des Verhaltensbaums ist in den Ausführungszustand des Applets integriert. Pro Zyklus innerhalb dieses Zustands wird der Verhaltensbaum einmal angetickt. Der Status des Wurzelknotens wird daraufhin auf die Zustände des Applet-Zustandsautomaten abgebildet. `RUNNING` hält den Skill im Execute-Zustand. Bei `SUCCESS` wird der Baum geregelt heruntergefahren und der Zustand des Skills entsprechend in den Abschlusszustand überführt. Alle übrigen Verhaltensbaumergebnnisse (`FAILURE` oder `INVALID`) führen zu einem Abbruch des Skills.

Für den Aufruf von untergeordneten Skills sind innerhalb der Composite Skill Basisklasse Start-, Abfrage- und Abbruchfunktionen implementiert. Diese kapseln die ROS-2-Action-Kommunikation mit den untergeordneten Skills. Für die Startfunktion wird ein Zielskill und das Parameter-Objekt benötigt. Beim Start wird ein Action-Client initialisiert, auf Serververfügbarkeit gewartet und anschließend das Goal asynchron gesendet. Dabei wird ein *Handle* erstellt, das Client, Status der Goal-Annahme, Metadaten sowie das Ergebnis bündelt. Die Überwachung erfolgt über wiederholtes Abfragen dieses Handles mithilfe der Abfragefunktion. Diese liefert direkt einen Verhaltensbaum-Status zurück. Sobald ein Ergebnis vorliegt, wird der ROS-2-Goalstatus ausgewertet und auf die Zustände `SUCCESS` bzw. `FAILURE` abgebildet. Das Ergebnis wird im Handle abgelegt. Zusätzlich kann ein laufendes Action-Goal explizit über die Abbruchfunktion abgebrochen werden. Zur konsistenten Steuerung mehrerer gleichzeitig aktiver Skills verwaltet die Basisklasse eine Liste aktiver Handles. Über eine zentrale Weiterleitungslogik werden Steuerkommandos (z. B. *hold, unhold, stop*) an alle aktuell registrierten Skills propagiert (via ROS-2-Services). Bei Stop- oder Abbruchereignissen wird ergänzend ein Fallback verwendet, der offene Action-Goals direkt abbricht. Die Zustandsmethoden des Applets greifen auf diese Logik zurück, sodass Zustandswechsel des Composite Skills unmittelbar auf alle laufenden Skills wirken.

Für die einheitliche und einfache Benutzung wurde ein generischer py_trees-Blattknoten implementiert, der einen untergeordneten Skill startet, überwacht und bei Bedarf abbricht. Dazu liest

der Knoten über zwei Blackboard-Schlüssel den Zielskill sowie die Parameter aus. Der untergeordneten Skill wird anschließend über die zuvor genannten Funktionen innerhalb des Verhaltensbaum ausgeführt. Optional kann beim Verlassen des Knotens ein Ergebnis-Callback ausgeführt werden, das Zugriff auf das Handle (und damit auf das Ergebnis) hat, um beispielsweise Änderungen am Wissensmodell anzustoßen.

Weiterhin wurde ein separater, einfacher Blattknoten, zur Auswertung von Bedingungen implementiert. Dieser liefert unmittelbar `SUCCESS` oder `FAILURE`. In Kombination ermöglichen die beiden Blattknoten die Modellierung komplexer, bedingter Abläufe auf Verhaltensbaum-Ebene, während die Ausführung und Fehlerbehandlung der beteiligten Skills weiterhin über die bestehende Applet- und ROS-2-Action-Infrastruktur erfolgt.

5.2.2 Primitive

Greifskill

Der Greifskill wurde erweitert, um den Greifvorgang feiner parametrieren zu können und die Nutzung in übergeordneten Skills zu erleichtern. Dafür wurden optionale Eingabeparameter ergänzt. Einmal zum begrenzen der beim Greifen zulässigen Kraft. Und einmal um die standardmäßig verwendete Zielstellung zu überschreiben, so dass auch Zwischenstellungen gezielt angefahren werden können. Damit wird die Parametrierbarkeit des Skills für übergeordnete Skills erhöht wodurch der Skill flexibler anwendbar wird.

Die Semantik des Eingabe-Kommandos wurde ebenfalls angepasst. Anstelle von *öffnen* und *schließen* werden die Werte `retain` (dt. Halten) und `release` (dt. Freigeben) verwendet. Diese Bezeichner beschreiben den gewünschten Zielzustand des Greifvorgangs und sind weniger an eine konkrete Greifermechanik gebunden. Dies unterstützt die wiederverwendbare Nutzung als untergeordneten Skill in den Composite Skills.

Schließlich wurde die Bewertung der Rückmeldung der Greifersteuerung überarbeitet. In der ursprünglichen Umsetzung wurde ein Greifvorgang nur dann als erfolgreich bewertet, wenn die Zielposition erreicht wurde. Bei Objektkontakt wird die Zielposition häufig nicht erreicht, obwohl ein stabiler Halt vorliegt. In der erweiterten Version wird bei `retain` zusätzlich das Rückmeldeattribut `stalled` berücksichtigt. Ein gesetztes `stalled` wird als Halten unter Last interpretiert und als Erfolg gewertet. Für `release` bleibt das bestehende Verhalten erhalten, sodass Erfolg nur bei vollständig geöffnetem Greifer gemeldet wird.

PTP-Bewegungsprimitive

Die PTP-Bewegungsprimitive wurde um zusätzliche Konfigurationsparameter erweitert, um Punkt-zu-Punkt-Bewegungen situationsabhängig parametrieren zu können. Hierzu wurden optionale Toleranzen für Position und Orientierung ergänzt, mit denen die zulässige Abweichung der Zielpose festgelegt werden kann.

Zusätzlich wurden Skalierungsfaktoren für Geschwindigkeit und Beschleunigung eingeführt. Diese Werte liegen im Bereich $[0, 1]$ und werden von MoveIt 2 als Faktoren auf die maximalen zulässigen Gelenkgeschwindigkeiten und Gelenkbeschleunigungen angewendet. Die Bewegungsplanung erfolgt damit im Gelenkraum unter Einhaltung der entsprechend skalierten Grenzwerte.

Für die Bewegungsplanung wird standardmäßig nun ein stichprobenbasierter Planer aus Open Motion Planning Library (OMPL) verwendet („The Open Motion Planning Library“, n. d.). Die Wahl

des Planers bleibt allerdings als Parameter konfigurierbar. Alle neu eingeführten Parameter sind optional und besitzen Defaultwerte, die über ROS 2-Parameter konfiguriert werden können.

Lineare Bewegungsprimitive

Wie bei der PTP-Primitive wurden Skalierungsfaktoren für Geschwindigkeit und Beschleunigung ergänzt. Auch die Toleranzparameter wurden ebenfalls übernommen.

Zur Erzeugung tatsächlich linearer Bahnen wird für diesen Skill nun der *Pilz Industrial Motion Planner* verwendet. Zusätzlich wurde ein Parameter eingeführt, mit dem Kollisionen in der MoveIt 2 Planungsszene optional zugelassen werden können. Außerdem gibt der Skill nun neben dem Ausführungsstatus auch die erreichte Endpose zurück.

5.3 Skill Implementierung

Durch die Implementierung des Wissensmodells und der Composite Skills Basisklasse, können nun die konzipierten Skills umgesetzt werden. Folgend wird zuerst der generelle Prozessablauf innerhalb eines Composite Skills vor der Ausführung des Verhaltensbaum aufgezeigt. Anschließend folgt die Implementierung des Pick-and-Place-Skill und des Dosier-Skill.

5.3.1 Generischer Skill-Ablauf

Abbildung 5.2 zeigt den generischen Ablauf, der von allen in dieser Arbeit implementierten Composite Skills im Zustand `starting` verwendet wird. Ziel dieses Startzustandes ist es, alle für die Ausführung benötigten Parameter auszulesen, zu validieren und auf dem Blackboard zu speichern.

Zunächst werden die Eingabeparameter des Action-Goals ausgelesen und typgeprüft. Parameter, die direkt als Werte übergeben werden (z. B. Skalierungsfaktoren oder numerische Grenzwerte), werden unmittelbar validiert. Referenzparameter (z. B. Objekt-IDs, Frame-IDs oder Ressourcen-IDs) werden über das Wissensmodell dereferenziert. Die dabei geladenen Elemente werden auf Existenz, erwarteten Typ sowie auf erforderliche Attribute geprüft. Fehlt ein notwendiger Parameter oder schlägt eine Validierung fehl, wechselt der Skill in den *Aborting*-Zustand und liefert eine Fehlermeldung zurück. Auf diese Weise werden fehlerhafte oder unvollständige Aufrufe frühzeitig erkannt und unnötige Ausführungsversuche vermieden.

Auf Basis der validierten Eingaben werden, soweit möglich, bereits im Startzustand abgeleitete Parameter berechnet. Nicht alle Parameter können jedoch zu diesem Zeitpunkt vollständig bestimmt werden. Zum einen stehen bestimmte Informationen erst während der Ausführung zur Verfügung (z. B. Rückgabewerte von Subskills oder Ergebnisse von Planungsanfragen), zum anderen müssen manche Parameter im Verlauf des Skills angepasst werden (z. B. im Rahmen von Retry-Strategien oder der Iteration über Kandidaten). Solche dynamischen Parameterbestimmungen werden daher als eigenes Verhalten innerhalb des Verhaltensbaums umgesetzt.

Damit diese spätere Parametrierung einheitlich erfolgen kann, wird im Startzustand zusätzlich das `py_trees`-Blackboard vorbereitet. Für jeden im weiteren Verlauf aufgerufenen untergeordneten Skill werden dabei bereits die benötigten Blackboard-Schlüssel angelegt. Die konkreten Werte werden entweder unmittelbar im Startzustand eingetragen oder später durch entsprechende Leaf-Knoten gesetzt bzw. überschrieben. Das Blackboard dient somit als zentrale Schnittstelle zwischen Parametrierungslogik und der eigentlichen Ausführung im Verhaltensbaum.

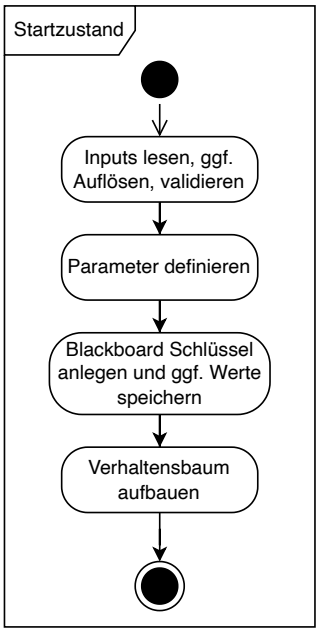


Abbildung 5.2: Generischer Parametrierungsablauf im Startzustand

5.3.2 Basis-Handhabungsklasse

Handhabungsskills (z. B. `Pick`, `Place`) benötigen für die Ausführung die Transformationen ${}^{W}T_{\text{obj}}$, ${}^{tcp}T_{\text{tool0}}$ und ${}^{obj}T_{\text{tcp}}$ sowie den Zugriff auf Objekt- und Ressourcendaten aus dem Wissensmodell (vgl. Abschnitt 4.4.1). Zur Vermeidung redundanter Implementierung wurde daher eine gemeinsame Basisklasse für Handhabungsskills eingeführt. Sie erbt von der Composite Skill Basisklasse und erstellt im `starting`-Zustand einen einheitlichen Ausführungskontext, der von allen Handhabungsskills genutzt wird.

Zentrale Eingaben sind die Objekt-ID sowie die ID der zu verwendenen Ressourcenkombination. Diese referenziert eine Kombination vom Typ `HandlingRobotResource`, welche die IDs eines Roboters, eines Greifers und optional weiterer Komponenten kapselt. Aus dieser Ressourcenkombination werden die benötigten Ressourcen dereferenziert und die Namen ihre Bewegungs- bzw. Greifprimitiven bestimmt. Alle Parameter die von der Handhabungsklasse eingelesen werden sind in Tabelle A.1 dargestellt.

Zusätzlich führt die Basisklasse handhabungsspezifische Vorbedingungen zusammen, die für alle abgeleiteten Skills gelten. Dazu gehören insbesondere Traglast-Prüfungen auf Basis des im Wissensmodell hinterlegten Objektgewichts sowie der maximalen Traglasten von Greifer und Roboter. Weiterhin können Skalierungsfaktoren für Geschwindigkeit und Beschleunigung übergeben werden. Falls sie nicht im Skill-Goal gesetzt sind, werden Defaultwerte aus ROS-Parametern verwendet und an die Bewegungsprimitiven weitergereicht.

5.3.3 Pick-and-Place-Skill

Heuristische Bestimmung von Richtungen und Distanzen

Für die linearen Teilbewegungen der Pick- und Place-Skills werden Richtungen und Distanzen benötigt. Diese Angaben werden bevorzugt aus den im Wissensmodell hinterlegten Frames übernommen. Wenn einzelne Angaben fehlen, ergänzt der Skill sie heuristisch und nutzt die Kollisionsprüfung der Bewegungsplanung, um unzulässige Varianten auszusortieren. Zwingend vorgegeben bleiben die Hubrichtung beim Pick sowie die Annäherungsrichtung beim Place. Die Heuristik ergänzt nur die übrigen, als optional behandelten Richtungen und Distanzen.

Wenn für eine lineare Bewegung kein Vektor hinterlegt ist, erzeugt der Skill Richtungs-Kandidaten aus den Hauptachsen des TCP-Koordinatensystems. Zuerst wird die z-Achse des TCP verwendet. Dabei wird vorausgesetzt, dass $+z$ von der Greiferstruktur wegzeigt und entlang der Werkzeugrichtung liegt, wie es in *ISO 9787:2013* (2013) beschrieben ist. Schlägt die Planung in dieser Richtung fehl, werden weitere Achsenrichtungen getestet, also $\pm x$ und $\pm y$. Dieses Richtungs-Sampling ist über ROS-Parameter konfigurierbar und kann deaktiviert werden. In diesem Fall werden nur Frames akzeptiert, die einen expliziten Vektor enthalten.

Für Distanzen wird ein zulässiger Wertebereich festgelegt, der zu große Verfahrwege verhindert. Innerhalb dieses Bereichs werden Kandidatendistanzen erzeugt und nacheinander geprüft. Sind für ein Objekt die maximalen Dimensionen in allen Raumrichtungen hinterlegt, wird der maximale Wert davon als Untergrenze genutzt. Somit ist ein Herausziehen gesichert falls sich das Objekt beispielsweise in einer Bohrung befindet. Die so abgeleitete Distanz wird anschließend auf den zulässigen Wertebereich begrenzt. Sind keine Objekt Dimensionen vorhanden, werden Kandidatendistanzen allein aus dem Wertebereich abgeleitet und als diskretisiertes Raster erzeugt. Auch die Anzahl tatsächlich getesteter Distanzen pro Richtung ist über ROS-Parameter von außen konfigurierbar, um gegebenenfalls die Laufzeit zu verringern.

Die Suche erfolgt über Kombinationen aus Richtung und Distanz. Für jede Kombination wird eine Planung angestoßen. Die erste kollisionsfreie Lösung wird übernommen. Scheitern alle Kandidaten, gilt der betreffende Frame oder Versuch als nicht ausführbar.

Pick-Skill

Der Pick-Skill nutzt den in der Handhabungsklasse bereitgestellten Ausführungskontext und benötigt zusätzlich die Objektpose aus dem Objekt-Basis-Frame sowie geeignete Greifposen in Form von `GraspFrames`. Standardmäßig werden diese aus der Pick-Frame Liste des Zielobjekts entnommen. Optional kann auch direkt ein `GraspFrame` oder eine entsprechende ID vorgegeben werden. In beiden Fällen werden nur solche Frames akzeptiert, welche die in Abschnitt 4.4.1 geforderte Hubrichtung enthalten. Alle zulässigen Frames werden in einer Kandidatenliste gesammelt. Bei direkter Parametrierung über einen *GraspFrame* enthält diese Liste genau diesen als Eintrag.

Zusätzlich kann ein Sicherheitsfaktor übergeben werden. Falls dieser Parameter nicht gesetzt ist, wird der Wert aus einem ROS-Parameter übernommen. Der Sicherheitsfaktor wird verwendet, wenn für den gewählten `GraspFrame` keine Greifkraft hinterlegt ist. In diesem Fall wird ersatzweise die statische Greifkraft (Objektattribut) mit dem Sicherheitsfaktor multipliziert:

$$F_{\text{Greif}} = S \cdot F_{\text{statisch}} \tag{5.1}$$

In späteren Erweiterungen, können zusätzliche Einflüsse berücksichtigt werden, etwa der Abstand zwischen Greifpunkt und Masseschwerpunkt des Objekts oder die Gelenkbeschleunigungen des Roboters. Alle Parameter des Skills sind in Tabelle A.2 dargestellt.

Eine zentrale Vorbedingung für die Ausführung ist ein freier Greifer. Dafür wird der Belegungszustand über das Greiferattribut `occupancy` abgebildet, das die Zustände `free`, `occupied` und `unknown` unterscheidet und damit unabhängig von der konkreten Greiferbauart bleibt.

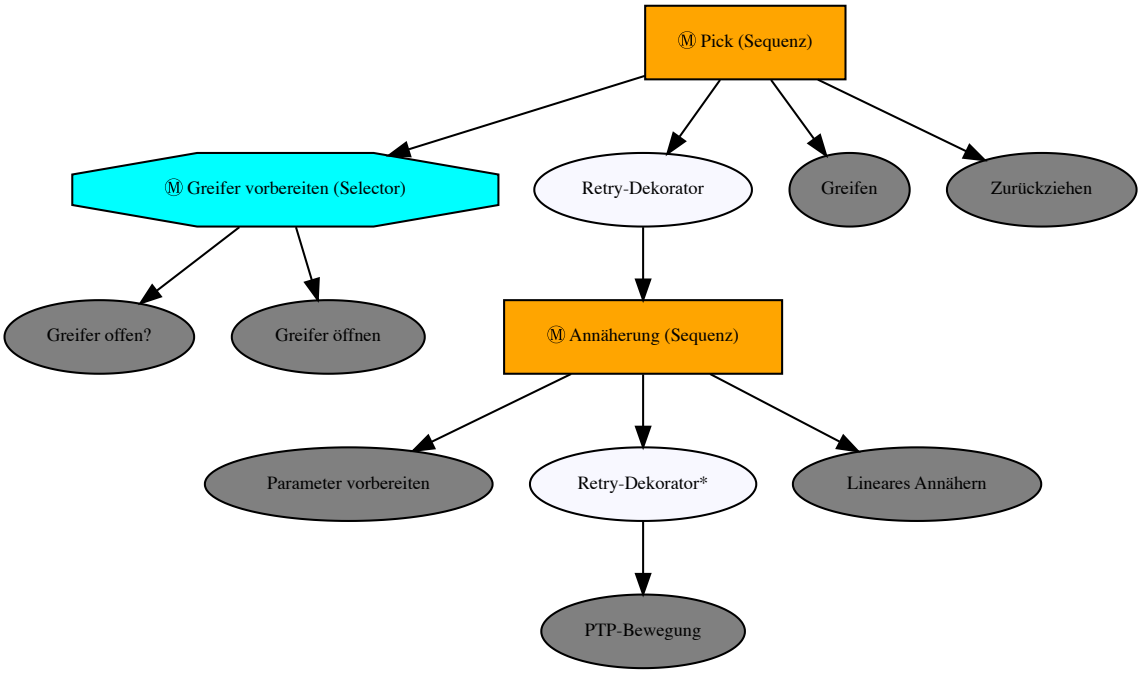


Abbildung 5.3: Verhaltensbaum des Pick-Skills

Der Ablauf des Pick Skills ist als Verhaltensbaum modelliert und in Abbildung 5.3 dargestellt. Im Folgenden wird die Funktionsweise auf Ebene der Teilsequenzen beschrieben.

Zu Beginn wird der Greifer in einen definierten Ausgangszustand überführt. Zuvor wurde bereits geprüft, ob der Greifer frei oder belegt ist. Diese Information ist jedoch nicht ausreichend, da ein Greifer seinen Greifmechanismus aktiv haben kann, ohne ein Objekt zu halten. Deshalb wird, falls der Mechanismus aktiv ist, die Greiferprimitive mit dem Kommando `release` ausführt. Im Ergebnis Callback wird anschließend das Wissensmodell entsprechend aktualisiert.

Nach der Vorbereitung folgt die Annäherungssequenz. Um den genauen Ablauf besser nachvollziehen zu können, ist die Annäherungssequenz in Abbildung 5.4 dargestellt. Die Sequenz ist von einem Wiederholungsdekorator umgeben, um mehrere im Wissensmodell hinterlegte GraspFrames nacheinander zu testen, falls beim Skillaufruf kein konkreter GraspFrame angegeben wurde. Die maximale Anzahl der zu prüfenden Frames wird über einen ROS Parameter begrenzt, um überlange Laufzeiten zu vermeiden. Für jeden Frame liest ein Blattknoten die relevanten Informationen aus und erzeugt Kandidatenlisten für Annäherungsrichtungen und Annäherungsdistanzen. Auf dieser Basis werden die Parameter für den jeweiligen Versuch berechnet und im `py_trees` Blackboard gespeichert.

Die PTP-Bewegung wird ebenfalls innerhalb eines Wiederholungsdekorators ausgeführt, um die verschiedenen Annäherungsparameter zu testen. Scheitert ein PTP-Versuch, werden im Callback des Blattknotens die Parameter angepasst. Es wird zur nächsten Distanz oder zur nächsten Richtung gewechselt, sofern noch Kandidaten vorhanden sind. Sind alle Varianten ausgeschöpft, wird der Wiederholungsblock als Misserfolg bewertet und der nächste GraspFrame wird versucht, sofern dieser vorhanden ist. Führt eine PTP-Bewegung zum Ziel, wird anschließend die finale lineare Annäherung ausgeführt. Schlägt diese fehl, wird ebenfalls der nächste GraspFrame versucht, sofern

verfügbar.

Nach erfolgreicher finaler Annäherung wird das Objekt gegriffen. Das Greifen ist in einem Selektor umgesetzt, sodass bei einem fehlgeschlagenen Greifversuch ein Öffnen des Greifers angestoßen wird. Dadurch kann das anschließende lineare Zurückziehen dennoch ausgeführt werden, da der Selektor Erfolg zurückgibt, egal ob das Objekt wirklich gegriffen werden konnte oder nicht. Dieses Verhalten stellt sicher, dass der Roboter im Fehlerfall, sofern möglich, in einen freien Zustand zurückkehrt.

Schlägt das Greifen fehl, werden im Callback auch die Parameter für das lineare Zurückziehen im Blackboard so gesetzt, dass sie der inversen Bewegung der zuvor ausgeführten linearen Annäherung entsprechen. Dadurch werden Kollisionen beim nachfolgenden Rückzug vermieden.

Ist das Greifen erfolgreich, wird im Callback der Greiferstatus im Wissensmodell aktualisiert. Zusätzlich wird dem Objekt Basis-Frame der Greifer-TCP als Referenz zugewiesen und die Objektpose entsprechend eingetragen. Durch die Referenzierung auf einen Ressourcen-TCP aktualisiert das Wissensmodell im Hintergrund die MoveIt 2 Planungsszene. Das Objekt wird als am Greifer befestigt eingetragen und bewegt sich fortan auch in der Planungsszene mit dem Greifer.

Da das Zurückführen in einen freien Zustand bei fehlgeschlagenem Greifen ansonsten zu einem scheinbaren Gesamterfolg führen kann, wird am Ende des Verhaltensbaums ein abschließender Bedingungsblattknoten verwendet. Dieser prüft eine Statusvariable, die nur bei tatsächlich erfolgreichem Greifen gesetzt wird. Auf diese Weise gibt der Verhaltensbaum zuverlässig den tatsächlichen Endstatus der Pick Ausführung zurück.

Place-Skill

Der Place-Skill nutzt ebenfalls den in der Handhabungsklasse bereitgestellten Ausführungskontext. Da sich das Objekt während der Ausführung im Greifer befindet, ist die Greifpose bereits festgelegt. Der Soll-Zustand wird daher über die Zielpose des Objekts beschrieben. Diese kann auf drei Arten angegeben werden:

1. als Pose mit zugehörigem Referenzframe in den Eingabeparametern
2. als Frame-ID, die auf einen Frame im Wissensmodell verweist
3. als Assembly-ID, wobei die Elementliste der Assembly die Zielpose des Handhabungsobjekts enthält

Für den Place-Skill ist eine Annäherungsrichtung zur Zielpose erforderlich, entweder direkt parametriert oder gekapselt in einem `GraspFrame`. Die weitere Richtung und die Distanzparameter sind optional und werden bei Bedarf gemäß Abschnitt 5.3.3 heuristisch ergänzt.

Optional kann eine Abwurfhöhe angegeben werden. Ist diese größer als Null, wird die Zielposition im Weltkoordinatensystem um diesen Betrag in $+z$-Richtung verschoben. Diese Funktion soll die Flexibilität und Wiederverwendbarkeit erhöhen. Alle Parameter des Skills sind in Tabelle **??** dargestellt.

Der Verhaltensbaum des Place-Skills ist in Abbildung 5.5 dargestellt.

Die Ausführung beginnt mit einer PTP-Bewegung in eine Annäherungspose. Diese Bewegung ist von einem Wiederholungsdekorator umgeben, um bei einem Fehlschlag die alternativen Annäherungsdistanzen zur Zielpose zu testen. Der zugehörige Blattknoten besitzt eine Callback Funktion, die in Abhängigkeit vom Ausführungsergebnis die Parameter der nachfolgenden linearen Bewegung anpasst.

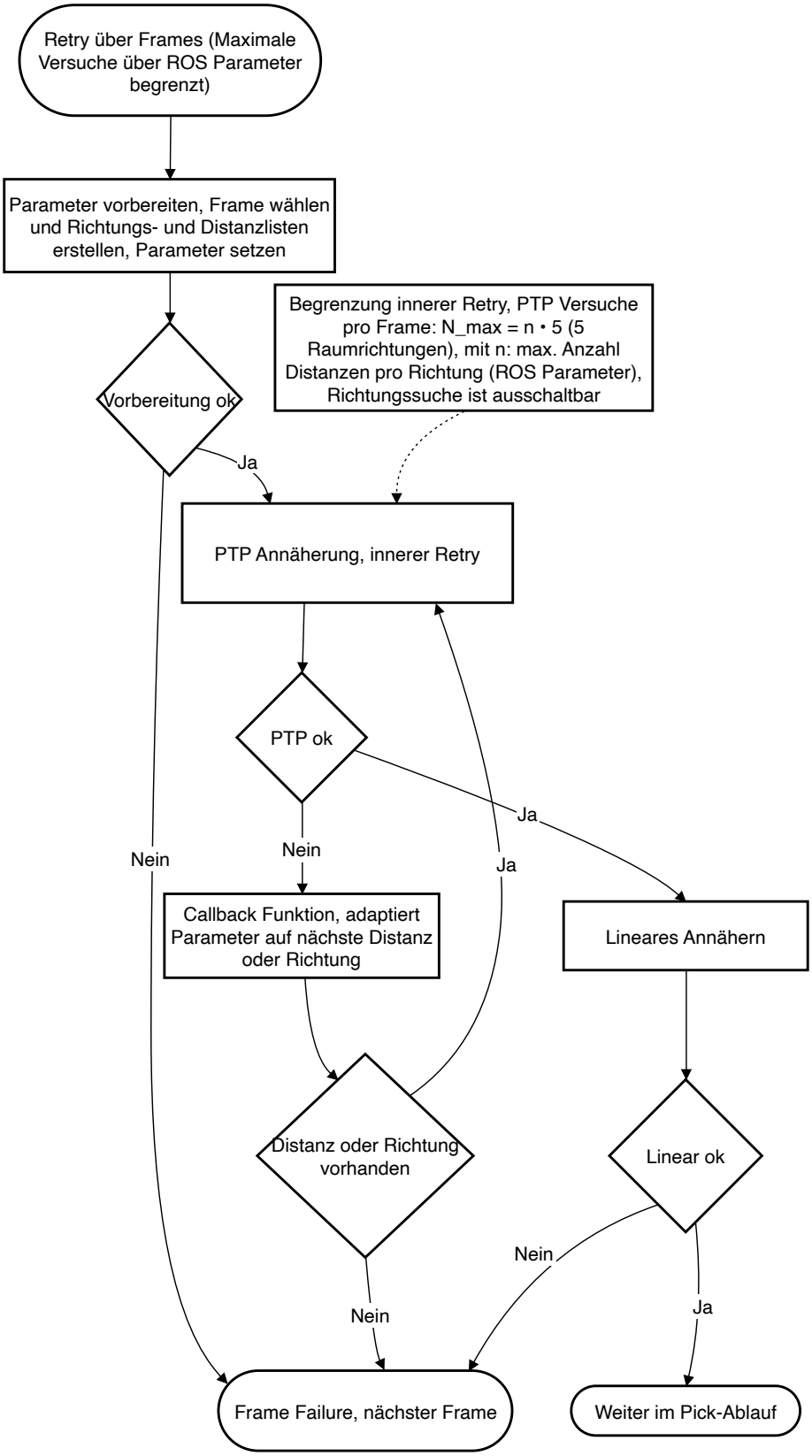


Abbildung 5.4: Abstrahierte Darstellung der Annäherungssequenz

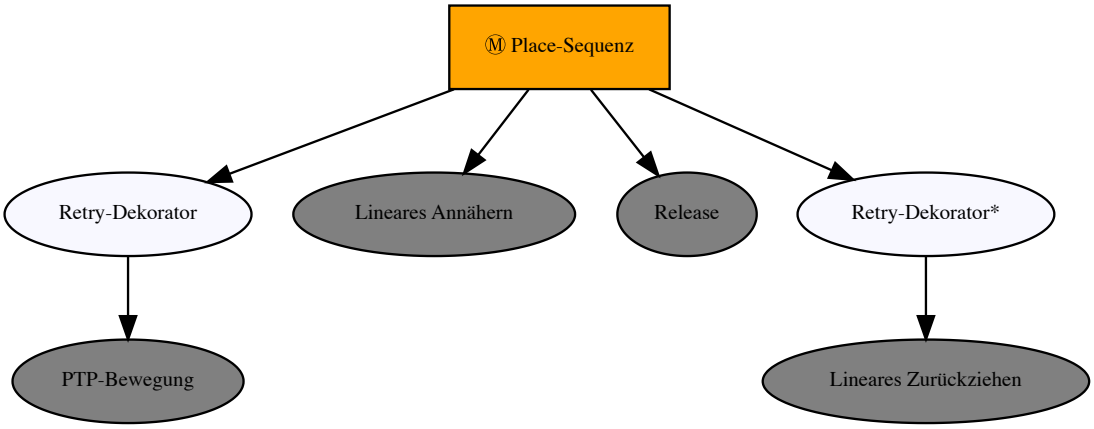


Abbildung 5.5: Verhaltensbaum des Place-Skills

Ist die PTP-Bewegung erfolgreich, folgt die lineare Bewegung zur Zielpose. Anschließend wird der Greifer im *Release*-Blattknoten geöffnet. Bei erfolgreicher Ausführung werden im Callback die aktualisierte Objektpose sowie der neue Greiferstatus an das Wissensmodell übertragen. Die Objektpose wird dabei aus der zurückgegebenen Pose der PTP-Primitive berechnet, wodurch eine möglichst genaue Eintragung der finalen Objektlage erreicht wird.

Im Anschluss wird eine lineare Rückzugsbewegung ausgeführt. Da der Greifer nach dem Ablegen wieder frei ist, können hierbei, sofern konfiguriert, mehrere Rückzugsrichtungen getestet werden. Als primäre Richtung wird die negative z Richtung des Greifer TCP Koordinatensystems verwendet. Ist die Rückzugsbewegung erfolgreich, wird der Place-Skill mit dem Status Erfolg abgeschlossen.

Komposition zu Pick-and-Place

Der Pick-and-Place-Skill komponiert die Subskills `Pick` und `Place` und nutzt ebenfalls die Basis-Handhabungsklasse. Die minimalen Eingangsparameter sind in Tabelle 5.1 dargestellt. Optionale Geschwindigkeits- und Beschleunigungsfaktoren werden wie in der Handhabungsklasse beschrieben übernommen und an die Subskills weitergereicht. Auch die optionale Abwurfhöhe des Place-Skills kann auf dieser Ebene gesetzt und weitergegeben werden.

Tabelle 5.1: Minimale Eingangsparameter des Pick-and-Place-Skills

Typ	Name	Beschreibung	Bedingung	Datentyp	Einheit
Input	`object`	Objekt-ID	Erforderlich	`str`	-
Input	`resource`	Ressourcenkombination-ID	Erforderlich	`str`	-
Input	`assembly`	Assembly-ID	Optional	`str`	-

Die relevanten `GraspFrames` werden aus den Pick- und Place-Listen des Zielobjekts geladen. Wie in Abschnitt 4.4.1 beschrieben, wird daraus eine Menge kompatibler Posenpaare gebildet. Dazu werden nur solche Pick-Frames berücksichtigt, die eine Hubrichtung besitzen, sowie solche Place-Frames, die eine Annäherungsrichtung bereitstellen. Aus den verbleibenden Frames werden alle Paarungen erzeugt.

Für jede Paarung wird geprüft, ob der Orientierungsunterschied unterhalb einer vorgegebenen Winkeltoleranz liegt und der euklidische Positionsabstand eine Positionstoleranz nicht überschrei-

tet. Erfüllt ein Paar beide Kriterien, wird es als Kandidat gespeichert. Positions- und Winkeltoleranz sind über ROS-Parameter konfigurierbar. Zusätzlich kann die Anzahl der zu prüfenden Paare begrenzt werden, um die Laufzeit zu kontrollieren.

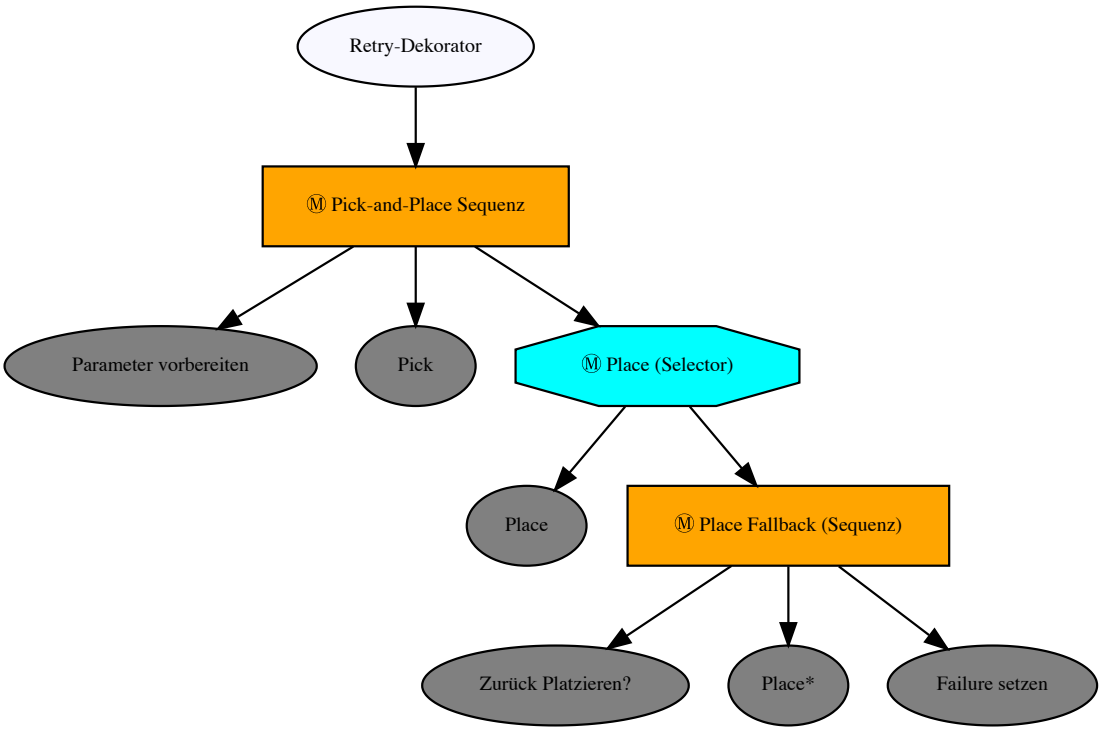


Abbildung 5.6: Verhaltensbaum des Pick-and-Place-Skills

Der vollständige Verhaltensbaum des Pick-and-Place-Skills ist in Abbildung 5.6 dargestellt. Die gesamte Pick-and-Place-Sequenz wird von einem Wiederholungsdekorator umschlossen, um bei Fehlschlägen nacheinander verschiedene Kandidatenpaare aus der Liste verfügbarer GraspFrame-Paare zu testen. Die maximale Anzahl der Versuche wird durch einen Konfigurationsparameter begrenzt und zusätzlich durch die Anzahl der vorhandenen Kandidatenpaare bestimmt.

Im ersten Blattknoten der Sequenz, werden die Parameter vorbereitet und im `py_trees` Blackboard abgelegt. Zusätzlich wird geprüft, ob der Greifer im Wissensmodell weiterhin den Belegungsstatus `free` besitzt. Ist dies nicht der Fall, wird die Fehlversuchsanzahl des Wiederholungsdekorators auf den maximal zulässigen Wert gesetzt, sodass keine weiteren Neuversuche gestartet werden. Damit wird verhindert, dass nach einem Fehler in einem Teilschritt weitere Kandidatenpaare unnötig abgearbeitet werden. Ein solcher Zustand wird zwar durch die Vorbedingungen des Pick-Skills grundsätzlich abgefangen, könnte jedoch bei langen Kandidatenlisten zu unerwünschten Verzögerungen führen.

Nach der Parametrierung wird der Pick-Skill ausgeführt. Bei erfolgreicher Ausführung wird anschließend innerhalb eines Selektors der Place-Skill aufgerufen. Scheitert das reguläre Ablegen, wird eine Fallback-Sequenz aktiviert, die versucht, das Objekt in die Ausgangspose zurückzulegen. Dadurch kann danach ein weiterer Pick-and-Place-Versuch mit einem anderen Kandidatenpaar durchgeführt werden, was die Zuverlässigkeit des Gesamtablaufs erhöht (NFA-4). Dieses Verhalten ist nicht in allen Szenarien gewünscht. Im Sinne der Benutzbarkeit (NFA-1) kann die Fallback-Logik daher über einen Konfigurationsparameter deaktiviert werden. Diese Konfiguration wird im ersten Schritt der Fallback-Sequenz geprüft. Abschließend setzt die Fallback-Sequenz ihren Rückgabesta-

tus explizit auf `Failure`. Auf diese Weise wird der aktuelle Versuch als fehlgeschlagen bewertet und der Wiederholungsdekorator wechselt zum nächsten Kandidatenpaar. Der gesamte Ablauf der Wiederholungslogik ist zur besseren Veranschaulichung nochmal in Abbildung 5.7 dargestellt.

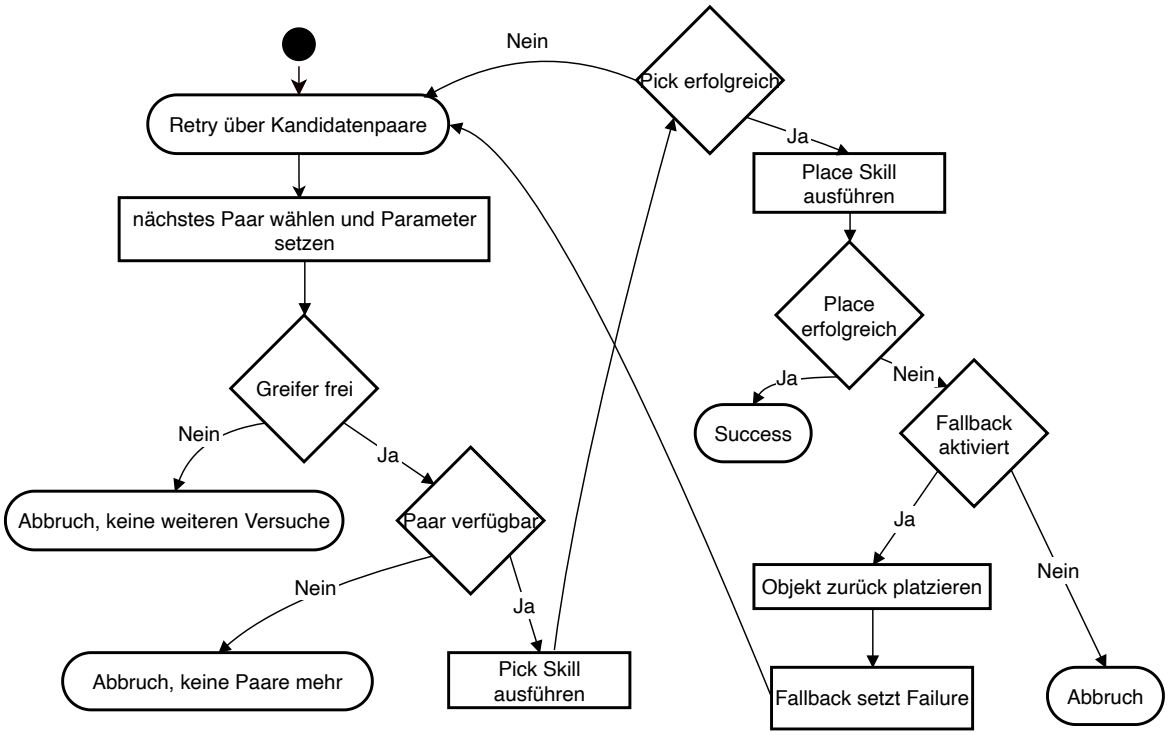


Abbildung 5.7: Frame-Wiederholungslogik beim Pick-and-Place-Skill

5.3.4 Dosier-Skill

Pfad-Bewegungsprimitive

Mit der Pfad-Bewegungsprimitive (`MovePathCartesianSkill`) wurde ein Skill implementiert, der den Roboter entlang einer vorgegebenen Liste kartesischer Posen verfährt. Als Eingabe dient eine Posenliste im Weltkoordinatensystem. Auf dieser Grundlage wird eine kollisionsgeprüfte Trajektorie erzeugt, die die Wegpunkte in der vorgegebenen Reihenfolge abfährt.

Die Bewegungsprimitive unterstützt zwei Betriebsarten, die über Parameter ausgewählt werden. Im Planungsmodus wird ausschließlich die Trajektorie berechnet. Bei erfolgreicher Planung wird die Trajektorie innerhalb des Skills zwischengespeichert und der Skill meldet Erfolg an den Action Client zurück. Im Ausführungsmodus wird die zuvor gespeicherte Trajektorie ausgeführt. Die Trennung von Planung und Ausführung ermöglicht es, die Bewegungserzeugung unabhängig von nachgelagerten Prozessschritten zu behandeln und Ausführungsaktionen erst dann zu starten, wenn eine gültige Trajektorie vorliegt.

Bahnplanung und Zeitparametrierung

Die Bahnplanung erfolgt über den MoveIt 2 Service *GetCartesianPath*. Aus den kartesischen Wegpunkten, dem Startzustand, der verwendeten Move Group und dem Endeffektor Link wird eine

Service-Anfrage aufgebaut und an MoveIt 2 übergeben. MoveIt 2 berechnet daraus eine Gelenkraumtrajektorie, die die kartesischen Wegpunkte abfährt.

Der von *GetCartesianPath* zurückgegebene Pfad liegt als zeitparametrisierte Gelenkraumtrajektorie vor. Die Zeitparametrierung berücksichtigt Skalierungsfaktoren für maximale Gelenkgeschwindigkeiten und Gelenkbeschleunigungen. Diese Faktoren werden mit den in der MoveIt 2 Konfiguration hinterlegten Grenzwerten der Gelenke verrechnet. Die Pfad-Bewegungsprimitive übernimmt die Skalierungsfaktoren als Eingabe und übergibt sie an `GetCartesianPath`. In der im Projekt verwendeten MoveIt 2 Version ermöglicht `GetCartesianPath` jedoch keine Vorgabe einer konstanten kartesischen Endeffektorgeschwindigkeit entlang des Pfades.

Im Rahmen dieser Arbeit wird keine zusätzliche Zeitparametrierung implementiert. Somit wird weder eine exakt konstante kartesische Endeffektorgeschwindigkeit entlang des Pfades erzwungen noch eine feste Gesamtdauer der Bewegung vorgegeben. Eine bahnbezogene Parametrierung dieser Art erfordert eine explizite Zeitverteilung entlang der kartesischen Bahnlänge unter gleichzeitiger Einhaltung aller Gelenkgrenzen und Dynamikrestriktionen. Dies würde eine Erweiterung oder einen Austausch der in MoveIt 2 verwendeten Trajektorienverarbeitung sowie eine zusätzliche Validierung erfordern. Der dafür notwendige Implementierungs- und Testaufwand überschreitet den Umfang der vorliegenden Bachelorarbeit.

Bahnparametrierung über Posenliste und G-Code

Wie in 4.4.2 konzipiert, soll die Bahn entweder als explizite Posenliste oder als G-Code-Datei vorgegeben werden. G-Code ist ein in *ISO 6983-1:2009* (2009) beschriebenes, textbasiertes NC Programmformat, in dem Bewegungen und zugehörige Parameter über Adresswörter und Funktionscodes angegeben werden. In Tabelle 5.2 sind wichtige Befehle und ihre Nutzung im Kontext dieser Arbeit abgebildet. In Liste 5.1 wird außerdem gezeigt, wie G-Code typischerweise aussieht.

Für die G-Code-Dateien werden ausschließlich Programme im absoluten Modus `G90` unterstützt. Der inkrementelle Modus `G91` wird als nicht zulässig bewertet und führt zu einem Fehler. Die Einheiten werden über `G20` und `G21` erkannt und anschließend auf Meter normiert.

Das Einlesen erfolgt in zwei Phasen. In der Initialisierungsphase ist genau ein Verfahrbefehl `G00` zulässig, aus dem die Startpose abgeleitet wird. Mit dem ersten interpolierten Fahrbefehl `G01` beginnt die Bahnphase. In dieser Phase wird jede Änderung der kartesischen Koordinaten X, Y und Z als neuer Bahnpunkt interpretiert und der Posenliste hinzugefügt. Schnelle Verfahrbewegungen `G00` nach Beginn der Bahnphase sind nicht erlaubt, um sprunghafte Bewegungen während des Dosierprozesses auszuschließen.

Neben den kartesischen Koordinaten werden die Rotationsachsen A, B und C ausgewertet. Diese Werte werden als Euler-Winkel interpretiert und in Quaternionen umgerechnet. Das Ergebnis der Umwandlung ist damit unabhängig von der ursprünglichen Bahnrepräsentation. Der Dosier-Skill erhält in beiden Fällen eine vollständige Posenliste als interne Arbeitsgrundlage.

Listing 5.1: G-Code-Ausschnitt

```
N0 G90
N1 G00 X-24 Y25.5 Z23 A0 B180 C0
N2 G01 X24 F10
N3 X24.235 Y25.482
N4 X24.464 Y25.427
...
```

Tabelle 5.2: Unterstützter G Code Teilumfang für die Bahnkonvertierung

Befehl	Bedeutung und Nutzung in dieser Arbeit
G00	Rapid Move, nur zur Initialisierung der Startpose
G01	Lineare Bewegung, erzeugt Bahnpunkte
G90	Absoluter Modus, zulässig
G91	Inkrementeller Modus, nicht zulässig, führt zu Fehler
G20	Einheiten in inch, werden auf Meter normiert
G21	Einheiten in mm, werden auf Meter normiert
X Y Z	Kartesische Position
A B C	Orientierung, als Euler Winkel interpretiert

Verhaltenslogik des Dosier-Skills

Der Ablauf des Dosier-Skills ist als Verhaltensbaum in Abbildung 5.8 dargestellt. Alle Eingabeparameter sind in Tabelle A.4 dokumentiert. Nach erfolgreicher Definition der Posenliste wird zunächst die erste Pose über eine PTP-Bewegung angefahren. Aus dieser Annäherungspose wird anschließend die Pfad-Bewegungsprimitive im Planungsmodus aufgerufen. Wird eine gültige Trajektorie gefunden, wird der Dispenser aktiviert und die Pfad-Bewegungsprimitive im Ausführungsmodus erneut ausgeführt. In diesem Schritt wird die zuvor geplante Trajektorie abgefahren.

Die Trennung von Planung und Ausführung stellt sicher, dass der Dispenser erst dann eingeschaltet wird, wenn eine ausführbare Trajektorie vorliegt. Die Ausführung der Pfadbewegung ist in einem Dekorator gekapselt, der einen möglichen Fehlerstatus für `py_trees` in einen Erfolgsstatus überführt (*FailureIsSuccess*). Dadurch wird gewährleistet, dass die Sequenz unabhängig vom Ergebnis der Pfadausführung fortgesetzt wird und der Dispenser im folgenden Schritt zuverlässig wieder deaktiviert werden kann. Der tatsächliche Ergebnisstatus der Pfadausführung wird in der Callback-Funktion der PfadBewegungsprimitive gespeichert und nach dem Ausschalten des Dispensers ausgewertet. Auf dieser Grundlage wird abschließend der korrekte Endstatus an die Wurzel des Verhaltensbaums zurückgegeben.

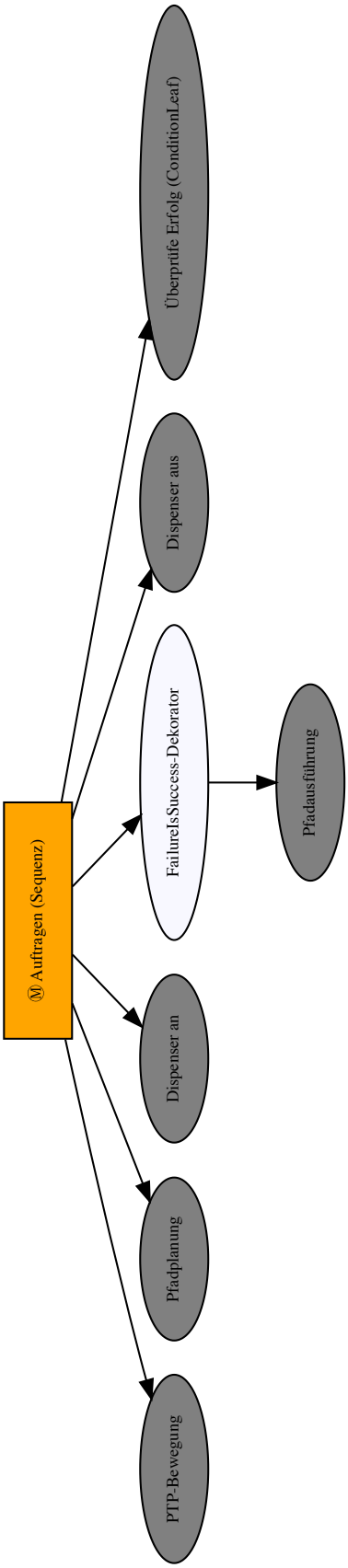


Abbildung 5.8: Verhaltensbaum des Dosier-Skills

Kapitel 6 Evaluation und Diskussion

6.1 Evaluationskonzept

Im Folgenden wird das entwickelte System anhand zweier anwendungsnaher Montageszenarien qualitativ evaluiert. Ziel der Evaluation ist es zu überprüfen, ob die Anforderungen aus Abschnitt 4.3.3 erreicht wurden. Daraufhin soll ein Fazit gezogen werden inwieweit die Ziele der Arbeit aus Abschnitt 4.3.1 erreicht wurden.

Die Evaluation folgt dabei einem szenariobasierten Ansatz. In Kapitel 6.2 werden für zwei ausgewählte Montageszenarien konkrete Testfälle definiert, in welchen die Skills ausgeführt werden. Für jeden Testfall werden Aufbau, Parametrisierung und beobachtete Abläufe dargestellt.

In den nachfolgenden Abschnitten 6.3 und 6.3.2 werden die zuvor beschriebenen Beobachtungen systematisch den funktionalen und nicht-funktionalen Anforderungen zugeordnet und hinsichtlich ihrer Erfüllung bewertet. Für jede Anforderung wird auf Basis der beschriebenen Testfälle eine qualitative Einstufung des Erfüllungsgrades vorgenommen. Hierzu wird eine dreistufige Skala verwendet:

- *nicht erfüllt*: die Anforderung wird durch die vorliegende Implementierung im Rahmen der betrachteten Szenarien nicht erfüllt.
- *teilweise erfüllt*: wesentliche Aspekte der Anforderung sind umgesetzt, Teile fehlen oder sind nur in Ansätzen vorhanden.
- *vollständig erfüllt*: die Anforderung wird im Rahmen der untersuchten Szenarien ohne erkennbare Einschränkungen erfüllt.

Die Einstufung ist als qualitative Einordnung zu verstehen und wird jeweils durch eine kurze Begründung unterstützt.

Für eine abschließende empirische Bewertung der Skills wäre letztlich ein Test an einer realen Roboteranlage erforderlich. Im Rahmen dieser Arbeit beschränkt sich die Evaluation bewusst auf den Bewegungsablauf der Skills in der Simulationsumgebung. Kräfte und Momente werden nicht gemessen oder geregelt. Entsprechend fließen Aspekte wie tatsächliche Greifkräfte, Klebstoffmengen oder elastische Verformungen der Umgebung nicht in die Betrachtung ein. Die Ergebnisse der Evaluation sind daher als prototypischer Funktionsnachweis der entwickelten Skills und der zugrunde liegenden Architektur zu verstehen.

6.2 Szenarien und Testfälle

Nachstehend werden die Testszenarien einschließlich der durchgeführten Tests beschrieben und die gemachten Beobachtungen dokumentiert.

6.2.1 Szenario 1 – Kleben eines Kameragehäuses

Beschreibung der Umgebung und des Prozessablaufs

Im betrachteten Szenario wird ein Kameragehäuse montiert. Dazu wird zunächst auf die untere Gehäusehälfte entlang einer späteren Kontaktkontur Klebstoff aufgetragen. Anschließend wird die obere Gehäusehälfte aufgesetzt. Abschließend wird ein Stecker auf dem oberen Gehäuseteil platziert. Der Prozess wird hierbei von einem Roboter mit Greifer durchgeführt. Der Dosier-Roboter wird entsprechend ausschließlich im Umgebungsmodell modelliert, so dass der Dosier-Skill eine Ausführung überhaupt zulässt. Der vollständige Szenarioaufbau ist in Abbildung 6.1 dargestellt.

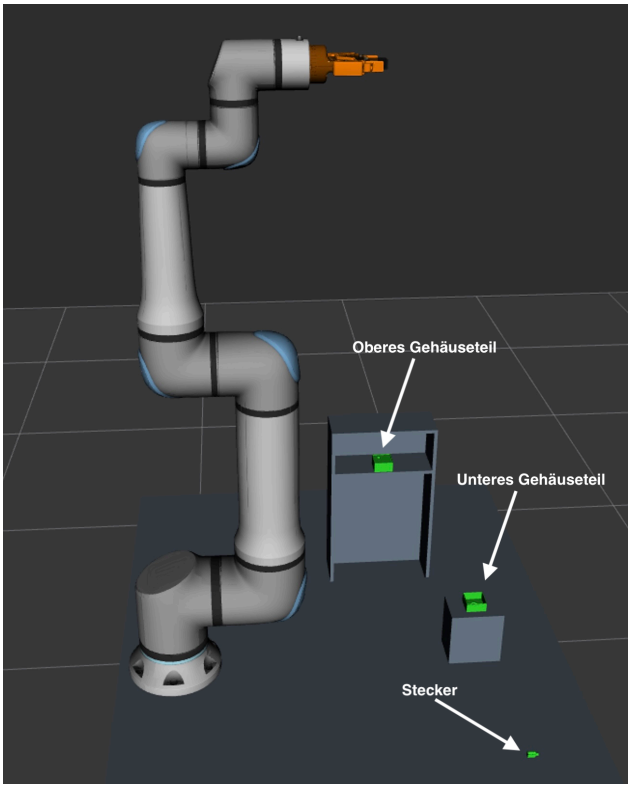


Abbildung 6.1: Szenario 1 dargestellt in RViz

Testfälle

In Tabelle 6.1 sind die durchgeführten Tests innerhalb Szenario 1 dargestellt. Für den Dosier-Skill wurden die Testfälle in aufsteigender Schwierigkeit angeordnet. In S1-D2 wird durch die Variation in Z-Richtung nicht die wahre Bauteilkontur abgefahren. Das erhöhen in Z-Richtung soll demonstrieren, dass der Dosier-Skill nicht auf planare Konturen begrenzt ist.

Bei der Hubbewegung beim Pick-Skill sowie bei der linearen Annäherung bei Place befindet sich das Objekt entweder zu Beginn oder am Ende in einem flächigen Kontaktzustand mit der Umgebung. Die Kollisionserkennung von MoveIt 2 erkennt dies bereits als Kollision an. Deshalb musste für genannte Bewegungen die Kollisionserkennung ausgeschaltet werden, damit die Skills vollständig ausgeführt werden können.

Tabelle 6.1: Testfälle Szenario 1 (Kleben eines Kameragehäuses)

ID	Skill	Aufbau / Variation	Beobachtung / Ergebnis
S1-D1	Dosieren	Planare 2D-Kontur entlang des unteren Gehäuseteils; konstante Orientierung (Tool-Z nach $-Z_W$).	Testfall vollständig ausführbar, daher **bestanden**.
S1-D2	Dosieren	3D-Kontur mit Z-Variation; Orientierung konstant.	Testfall vollständig ausführbar, daher **bestanden**.
S1-D3	Dosieren	Planare Kontur; variable Orientierung (Wechsel in Ecken).	Häufig keine gültige Trajektorie über `GetCartesianPath`. Deshalb **nicht bestanden**.
S1-P1	Pick&Place	Obere Gehäusehälfte aus Regal; erschwerter Zugang; mehrere Greifkandidaten; Nutzung der Heuristiken.	Greifen und Platzieren möglich, jedoch Hubbewegung bei Pick und lineare Annäherung bei Place nur mit ausgeschalteter Kollisionserkennung erfolgreich. Daher **bestanden mit Workaround**.
S1-P2	Pick&Place	Stecker liegt flach; Anheben senkrecht, Einführen in Längsrichtung, mehrere Greifkandidaten; Nutzung der Heuristiken.	Testfall vollständig ausführbar mit gleicher Einschränkung wie bei S1-P1. Daher **bestanden mit Workaround**.

6.2.2 Szenario 2 – Montage von Batteriezellen

Beschreibung der Umgebung und des Prozessablaufs

Im zweiten Szenario wird die Montage eines Batteriestacks betrachtet. Dazu sollen mehrere Batterien in eine vorgesehene Halterung eingesetzt werden. Für dieses Szenario wurden drei Testfälle definiert, die den Pick-and-Place Skill weiter Testen sollen. Der vollständige Aufbau und Endzustand ist in Abbildung 6.2 dargestellt.

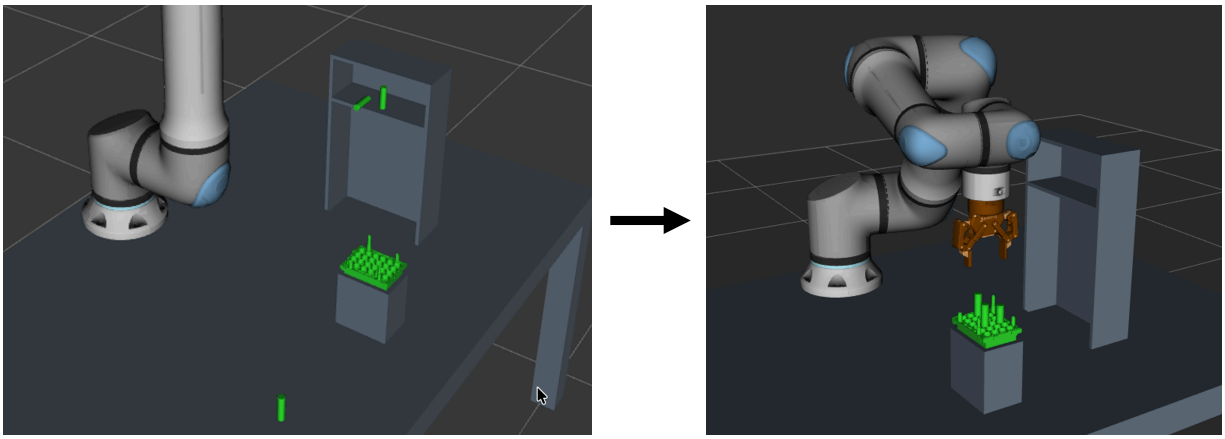


Abbildung 6.2: Szenario 2 dargestellt in RViz

Testfälle

Die folgenden Testfälle prüfen den Pick-and-Place-Skill beim Einsetzen von Batteriezellen in die Halterung. Ein Testfall gilt als bestanden, wenn Greifen, Transfer und Platzierung ohne Planungsabbruch ausgeführt werden können. Für die durchgeführten Tests wurde die Implementierung vor Durchführung der Tests aus Szenario 2 bereits angepasst, um auf die Ergebnisse aus Szenario 1 zu reagieren. Bei der Hubbewegung und beim linearen Annähern an die Ablageposition wird somit nun keine Kollisionserkennung mehr durchgeführt. Alle Testfälle und Ergebnisse sind in Tabelle 6.2 dargestellt.

Tabelle 6.2: Testfälle Szenario 2 (Montage von Batteriezellen)

ID	Skill	Aufbau / Variation	Beobachtung / Ergebnis
S2-P1	Pick&Place	Batterie steht aufrecht auf dem Tisch; Top-Griff von oben; Platzierung in Halterung.	Testfall vollständig ausführbar, daher **bestanden**.
S2-P2	Pick&Place	Batterie liegt im Regal; Greifen und Herausziehen aus dem Regal; anschließende Platzierung in Halterung.	Testfall vollständig ausführbar, daher **bestanden**.
S2-P3	Pick&Place	Batterie steht im Regal; Seiten-Griff mit Hub nach oben; Platzierung in Halterung.	Kollisionsfrei nur unter expliziter Angabe der Hubdistanz durchführbar. **Bestanden nur mit expliziter Parametrierung**.

Während die ersten beiden Testfälle ohne Auffälligkeiten bestanden wurden, konnte bei S2-P3 festgestellt werden, dass es hier zur Kollision kommen kann wenn die Hubdistanz nicht explizit und korrekt angegeben wird. Denn durch das zuvor angesprochene Ausschalten der Kollisionsprüfung für diese Bewegung, würde eine zu hohe Hubdistanz dazu führen, dass der Greifer mit der Decke des Regals kollidiert.

6.2.3 Integrationstests

Neben dem Testen der Skill-Funktionalitäten soll außerdem verifiziert werden, dass die Zustandsweiterleitung an untergeordnete Skills zuverlässig funktioniert. Der kritische Testfall ist insbesondere *Hold*, da hier die Anforderung besteht, dass beim darauffolgendem Aufruf von *Unhold*, die Ausführung an besagtem Punkt weiterläuft. Dies wurde an drei Testfällen abgebildet in 6.3 demonstriert.

Tabelle 6.3: Integrationstests zur Laufzeitsteuerung (Hold/Unhold)

ID	Aufbau / Variation	Beobachtung / Ergebnis
IT-1	Hold/Unhold während kartesischer Annäherung bzw. Rückzug.	Zustandsweiterleitung erfolgreich; nach Unhold Fortsetzung möglich. **bestanden**.
IT-2	Hold/Unhold während PTP PTP Bewegung.	Zustandsweiterleitung erfolgreich; nach Unhold Fortsetzung möglich. **bestanden**.
IT-3	Hold/Unhold nahe Greif- bzw. Platzieroperation.	Zustandsweiterleitung erfolgreich; nach Unhold Fortsetzung möglich. **bestanden**.

6.3 Evaluation der Anforderungen

Im folgenden wird bewertet, inwieweit die Anforderungen aus Abschnitt 4.3.3 erfüllt wurden. Eine Zusammenfassung der Evaluation findet sich in Tabelle 6.4. Diese Einstufungen werden anschließend genauer erläutert.

Tabelle 6.4: Übersicht zur Erfüllung der Anforderungen

ID	Anforderung	Evidenz	Bewertung
FA-1	Aufgabenorientierte Parametrierung von Composite Skills	Betrachtung der übergebenen Parameter in den Szenarien S1/S2.	vollständig erfüllt
FA-2	Wissensmodell für Umgebungsinformationen	Struktur und Wiederverwendbarkeit des Datenmodells	teilweise erfüllt
FA-3	Kommunikation der Skills mit dem Wissensmodell	Funktion und Nutzbarkeit der Schnittstellenfunktionen in S1&S2	vollständig erfüllt
FA-4	Vielfalt und Optionalität von Parametern	Schnittstellenumfang: alternative Parametrierungswege (z. B. Pose vs. Assembly-Referenz; Bahn als Pose-Liste vs. G-Code); Nutzbarkeit in S1/S2.	vollständig erfüllt
FA-5	Abbildung komplexer Abläufe in Composite Skills	Nutzbarkeit der implementierten Verhaltensbaum-Logik für Entwickler	vollständig erfüllt
FA-6	Weitergabe von Zuständen an untergeordnete Skills	Beobachtete Zustandsweitergabe an aktive Subskills in IT1 bis IT3.	vollständig erfüllt
NFA-1	Konfigurierbarkeit der Skills	Umfang der Vorhandenen ROS-Parameter	teilweise erfüllt
NFA-2	Portierbarkeit der Skills	Entkopplung über Verbindung von Ressourcen und Fähigkeiten; Einschränkungen durch Greiferunabhängige Heuristiken; keine Hardware-Validierung.	teilweise erfüllt
NFA-3	Kollisionsfreie Bewegung	Kollisionsprüfung über MoveIt 2; notwendige Deaktivierung der Kollisionsprüfung in linearen Bewegungen in S1/S2.	nicht erfüllt
NFA-4	Robustheit bei unvollständigen Eingaben	Optionale Parameter + Heuristiken/Fallbacks im Pick&Place; beim Dosier-Skill im Wesentlichen Eingabevalidierung und Abbruch.	teilweise erfüllt

6.3.1 Evaluierung der funktionalen Anforderungen

FA-1 - Aufgabenorientierte Parametrierung von Composite Skills, vollständig erfüllt

Die an den Taskcontroller übergebenen Parameter sind ausschließlich aufgabenbezogene Parameter.

Für den Pick-and-Place-Skill wurden die ID der Ressourcenkombination, die ID des Objekts sowie die ID der Ziel-Assembly übergeben. Für den Dosier-Skill wurden Durchflussrate, Ressource- und Objekt-ID, Dateipfad sowie Skalierungsfaktoren für Geschwindigkeit und Beschleunigung übergeben.

Als Einschränkung gilt, dass Geschwindigkeits- und Beschleunigungsfaktoren nur indirekt aufgabenrelevant sind. Eine Parametrierung über eine explizite Vorschubgeschwindigkeit wäre erstrebenswert. Dies wurde in der vorliegenden Implementierung jedoch nicht umgesetzt.

FA-2 - Wissensmodell für Umgebungsinformationen, teilweise erfüllt

Das Wissensmodell stellt ein objektorientiertes Datenmodell der Umgebung bereit. Im Sinne von Z-3 ist entscheidend, ob das Modell allgemein anwendbar ist und nicht primär auf die in dieser Arbeit entwickelten Skills zugeschnitten wurde.

Die Umgebung wird im Modell im Wesentlichen durch Objekte, Ressourcen, Ressourcenkombinationen und relevante Koordinatensysteme beschrieben. Für Ressourcen und Ressourcenkombinationen wurden vererbte Klassen eingeführt, die die in dieser Arbeit benötigten Typen abdecken, zum Beispiel die Kombination aus Handhabungsroboter, Roboter und Greifer. Das Modell ist dabei erweiterbar und kann zukünftig um weitere Ressourcentypen ergänzt werden.

Die Objektbeschreibung ist überwiegend allgemein und nicht szenariospezifisch. Eine Einschränkung ergibt sich bei zwei Listenattributen, die Greifposen im Ist- und Soll-Zustand speichern. Zu jeder Greifpose wird ein konkretes Greifermodell referenziert. Damit handelt es sich eher um ein Prozessattribut als um ein Objektattribut. Im Sinne der Wiederverwendbarkeit wäre hierfür künftig eine Trennung von Objekt- und Prozessdaten sinnvoll.

Neben der Modellierung übernimmt die Wissensmodell-Node zusätzliche Aufgaben, unter anderem die automatische Synchronisation mit der MoveIt-Planungsumgebung.

FA-3 - Kommunikation der Skills mit dem Wissensmodell, vollständig erfüllt

Für die Kommunikation der Skills mit dem Wissensmodell wurde eine Schnittstellenklasse implementiert, die das Wissensmodell-Topic abonniert und den aktuellen Stand des Modells als Attribut vorhält. Skill-Entwickler erhalten damit einen einfachen Lesezugriff auf das Umgebungsmodell. Zusätzlich wurden Funktionen bereitgestellt, um neue Umgebungsattribute hinzuzufügen oder bestehende Einträge mit aktualisierten Informationen zu überschreiben. Auf diese Weise kann das Wissensmodell mit einem einzelnen Funktionsaufruf aus dem Skill heraus modifiziert werden. Darüber hinaus stehen Funktionen zur Verfügung, um gezielt nach Koordinatensystemen zu suchen und Koordinatensysteme sowie Vektoren in andere Bezugsrahmen zu transformieren.

FA-4 - Vielfalt und Optionalität von Parametern, vollständig erfüllt

Für die Evaluation wird hier die Parametrierungsschnittstelle herangezogen. Maßgeblich ist dabei, welche Parametertypen unterstützt werden und ob für dieselbe Information alternative Parame-

trierungsvarianten vorgesehen sind. In der vorliegenden Implementierung bieten die Skills mehrere Möglichkeiten, äquivalente Informationen zu übergeben. Beim *Pick*-Skill kann optional auf eine konkrete Greifpose referenziert werden. Der Pick-and-Place-Skill nutzt diese direkte Referenzierung. Beim *Place*-Skill kann die Zielinformation entweder indirekt über eine Assembly-Referenz oder direkt in Form einer Pose angegeben werden. Der Dosier-Skill erlaubt die Übergabe der Bahn sowohl als Liste von Posen als auch über eine G-Code-Datei.

Damit ist das Weglassen optionaler Parameter auf Ebene der Schnittstelle grundsätzlich möglich. Somit stehen sowohl Anwendern, bei der Nutzung, als auch Entwicklern beim Erstellen neuer Skills mehrere Parametrierungsvarianten zur Verfügung.

FA-5 - Abbildung komplexer Abläufe in Composite Skills, vollständig erfüllt

Durch den Einsatz von Verhaltensbäumen und dem einheitlichen Interface zum Aufrufen von untergeordneten Skills konnten auf einfachem Weg komplexe Verhaltensweisen erstellt werden. Deutlich wird dies insbesondere an dem Pick-and-Place-Skill und seinen untergeordneten Skills. Die vielen Wiederholungs- oder Fallback-Logiken verdeutlichen dies (s. Abbildung 5.3, 5.5 und 5.6). Das Interface zum Aufrufen der untergeordneten Skills ermöglicht außerdem die Implementierung einer Callback-Funktion, in der das Ergebnis der Skill und dessen Rückgabewerte ausgewertet werden können.

Im Rahmen dieser Arbeit wurden die Callback-Funktionen unter anderem genutzt, um abhängig vom Ausführungsergebnis das Wissensmodell zu aktualisieren. Auch die Auswertung von Rückgabewerten, wie der End-Pose beim Place-Skill konnte umgesetzt werden.

Somit liegt der Hauptaufwand beim Implementieren in der Berechnung der benötigten Parameter und nicht im manuellen Skripting komplexer Abläufe oder im Erstellen von sehr umfangreichen Zustandsautomaten.

FA-6 - Weitergabe von Zuständen an untergeordnete Skills, vollständig erfüllt

Die Basisklasse der Composite Skills verwaltet alle aktuell aktiven Skill-Aufrufe in einer Liste. Die Tests aus Tabelle 6.3 haben gezeigt, dass die Zustände des Composite Skills an die untergeordneten Skills weitergegeben werden. Was innerhalb der *Hold*- und *Unhold*-Zustände eines Skills letztendlich passiert, hängt weiterhin von der konkreten Implementierung innerhalb des Skills ab.

6.3.2 Evaluierung der nicht-funktionalen Anforderungen

NFA-1 - Konfigurierbarkeit der Skills, teilweise erfüllt

Zur Konfiguration der Skills wurden mehrere ROS-Parameter eingeführt, um sie an unterschiedliche Anwendungen anpassen zu können. Beispielsweise kann die maximale Anzahl an Frame-Versuchen beim Greifen begrenzt werden, und für die Handhabungs-Skills lassen sich Geschwindigkeits- und Beschleunigungsfaktoren einstellen. Für den Dosier-Skill wurden hingegen keine Konfigurationsparameter implementiert. Eine mögliche Erweiterung wäre hier ein Parameter, der einen Offset der abgefahrenen Bahn in TCP-Richtung erlaubt.

NFA-2 - Portierbarkeit der Skills, teilweise erfüllt

Die Portierbarkeit wird durch die Entkopplung von konkreter Hardware über das Umgebungsmodell unterstützt. Ressourcen werden dort mit Fähigkeiten einschließlich der jeweiligen Action-Server-Namen beschrieben. Die Skills werden mit den konkreten Ressourcen parametrisiert, die sie verwenden sollen, und prüfen, ob diese die erforderlichen Bewegungsprimitive bereitstellen. Solange gleichartige Primitive die gleichen Parameter erwarten, sind die Skills in diesem Rahmen hardwareunabhängig einsetzbar. Ein anderer Robotertyp, zum Beispiel ein SCARA-Roboter, benötigt für den Pick-and-Place-Skill entsprechend nur PTP und lineare Bewegungsprimitive. Erfolgt die Bewegungsplanung über MoveIt 2, wird zudem erkannt, wenn eine Pose aufgrund eingeschränkter Freiheitsgrade nicht erreichbar ist.

Einschränkungen ergeben sich insbesondere beim Pick- und beim Place-Skill. Annäherungsrichtungen werden derzeit unabhängig vom verwendeten Greifertyp in fünf Raumrichtungen generiert. Für einen Sauggreifer wäre typischerweise nur eine Annäherung in positiver Z-Richtung sinnvoll. Die Richtungsheuristik sollte daher künftig greiferabhängig ausgelegt werden.

Eine vollständige Bewertung der Portierbarkeit würde Tests mit unterschiedlicher Hardware erfordern, dies ist im Rahmen dieser Arbeit nicht erfolgt.

NFA-3 - Kollisionsfreie Bewegung, nicht erfüllt

Die von den Planern generierten Trajektorien werden in MoveIt 2 auf Kollisionen geprüft, und die Planungszene wird mit dem Wissensmodell synchronisiert. Wie in Abschnitt 6.2.1 beschrieben, musste beim Pick- und beim Place-Skill für die lineare Hubbewegung sowie die lineare Annäherungsbewegung die Kollisionsprüfung deaktiviert werden. Andernfalls wurden die Kontakte des Objekts mit der Umgebung als Kollision gewertet, weshalb die Primitive für die lineare Bewegung regelmäßig fehlschlug. Bei ungeeigneter Parametrierung kann es somit zu Kollisionen zwischen Objekt, Greifer und Umgebung kommen.

NFA-4 - Robustheit bei unvollständigen Eingaben, teilweise erfüllt

NFA-4 bewertet das Laufzeitverhalten der Skills bei fehlenden oder unvollständigen Angaben. Im Fokus steht, ob Eingaben durch Default- oder Fallback-Mechanismen ergänzt werden können. Dies ist von der reinen Optionalität von Parametern in der Schnittstelle zu unterscheiden (vgl. FA-4).

Für Pick und Place sind Annäherungsrichtung, Rückzugsrichtung sowie Distanzen als optionale Parameter ausgelegt. Fehlen diese Angaben, werden über Kollisionsprüfung geeignete Bewegungsrichtungen und Distanzen bestimmt. In den Testszenarien führte dies in den betrachteten Fällen zu ausführbaren Bewegungen. Zusätzlich erhöht die Fallback-Logik zum Zurückplatzieren an den Ursprungsort die Zuverlässigkeit. Der Pick-and-Place-Skill unterstützt außerdem Listen von Posen, aus denen Kandidaten ausprobiert werden können.

Für den Dosier-Skill wurden keine speziellen Robustheitsmechanismen für unvollständige oder fehlerhafte Eingaben implementiert. Die zentrale Eingabe ist die abzufahrende Bahn, die innerhalb der betrachteten Systemgrenze nicht sinnvoll aus anderen Informationen abgeleitet werden kann. Ohne zusätzliche Prozessmodelle oder sensorbasierte Rückkopplung, zum Beispiel zur Erkennung von Materialauftrag oder Bahnausführung, bleiben als robuste Strategien im Wesentlichen Eingabeprüfung und ein kontrollierter Abbruch bei unplausiblen oder nicht ausführbaren Bahnen.

6.4 Diskussion der Ziele

Im Folgenden wird diskutiert, welchen Beitrag die vorliegende Arbeit zur Erreichung der in Abschnitt 4.3.1 definierten Ziele leisten kann. Im Mittelpunkt steht dabei, welche Aspekte durch die Implementierung und die Testszenarien demonstriert werden konnten und welche Einschränkungen sich aus den Randbedingungen und den Erkenntnissen aus den Testszenarien ergeben.

6.4.1 Z-1 – Montageoperationen nach Norm

Ziel Z-1fordert die Abdeckung grundlegender Montageoperationen aus den Bereichen Handhabung (VDI 2860) und Fügen (DIN 8593). Für den Bereich der Handhabung wurde mit dem Pick-, Place- und dem daraus zusammengesetzten Pick-and-Place-Skill eine zentrale Montageoperation umgesetzt. Die Testszenarien zeigen, dass Objekte rein kinematisch aus einer definierten Ausgangspose in eine definierte Zielpose überführt werden können und damit das „Weitergeben“ im Sinne der Handhabung nach *VDI-Richtlinie 2860* (1990) grundsätzlich abbildbar ist.

Gleichzeitig zeigen die Ergebnisse Einschränkungen. Für die lineare Hubbewegung beim Greifen sowie die lineare Annäherungsbewegung beim Platzieren musste die Kollisionsprüfung deaktiviert werden. Damit konnte NFA-3nicht erfüllt werden. Darüber hinaus wurde der Skill rein kinematisch evaluiert. Aspekte wie Greifkräfte, geeignete Geschwindigkeiten oder Beschleunigungen wurden konzeptionell berücksichtigt, jedoch nicht experimentell validiert.

Zusammengefasst kann gesagt werden, dass der Pick-and-Place-Skill den Funktionsnachweis der Parameter- und Bewegungsstruktur grundsätzlich erbringt. Das Weitergeben nach *VDI-Richtlinie 2860* (1990) wird abgebildet. Eine Finale Validierung müsste allerdings in einer realen Umgebung mit einer größeren Anzahl an Objekten durchgeführt werden, um auch einen empirischen Nachweis der Funktion erbringen.

Für den Bereich des Fügens wurde das Kleben als repräsentative Operation gewählt, bestehend aus dem Auftragen des Klebstoffs entlang einer Bahn und dem anschließenden Fügen der Bauteile. Der Materialauftrag wird durch einen Dosier-Skill umgesetzt, der eine Bahn aus G-Code oder eine Posen-Liste einliest und abfährt. Hierbei wird erfolgreich das Planen der Bahn und die Ausführung getrennt, wodurch zuverlässig das gleichzeitige Starten der Bahnausführung und des Dispensers möglich ist. Die Tests zeigen, dass das Abfahren von Bahnen mit konstanter Orientierung zuverlässig möglich ist, während Bahnen mit variabler Orientierung nicht zuverlässig umsetzbar waren. Zudem ergibt sich aufgrund der in MoveIt 2 verwendeten Zeitparametrierung im Gelenkraum keine konstante Endeffektor-Geschwindigkeit. Damit kann im aktuellen Stand kein gleichmäßiger Materialauftrag garantiert werden, was für einen realen Klebeprozess eine wesentliche Einschränkung darstellt. Um einen realen Dosierprozess abzubilden, ist die aktuelle Implementierung also nicht ausreichend.

6.4.2 Z-2 – Aufgabenorientierte Programmierung

Durch die Einführung eines Wissensmoduls zur Speicherung von Umgebungsinformationen (FA-2) sowie durch die Bereitstellung von Kommunikationsschnittstellen für Lese- und Schreibzugriffe (FA-3) wurde eine Parametrierung der Composite Skills über objekt- und prozessbezogene Informationen ermöglicht. In den Aktionsplänen reicht es für Handhabungsoperationen aus, dem Taskcontroller mitzuteilen, welches Objekt mit welcher Ressource an welchen Zielort bzw. in wel-

che Assembly überführt werden soll. Damit erfolgt die Parametrierung über high-level objekt und prozessbezogener Parameter, was ein zentrales Merkmal aufgabenorientierter Programmierung ist.

Gleichzeitig zeigen die Ergebnisse, dass prozessbezogene Parameter in der aktuellen Umsetzung noch nicht einheitlich über das Wissensmodell abgebildet werden. Für den Dosier-Skill werden prozessbezogene Informationen (z. B. die G-Code-Datei) durch den Anwender in Form eines Dateipfads bereitgestellt. Im Pick-and-Place-Kontext werden prozessbezogene Informationen dagegen teilweise als Objektattribute modelliert, etwa in Form von Greifposen im Wissensmodell. Damit sind zwei praktikable, aber unterschiedliche Mechanismen im System vorhanden. Insbesondere das Speichern als Objektattribut ist nicht skalierbar, da eine größere Anzahl betrachteter Prozesse, zu einer ebenso erhöhten Anzahl an Objektattributen führen würde.

6.4.3 Z-3 – Flexibilität und Wiederverwendbarkeit

Durch den skillbasierten Ansatz in Kombination mit einem Wissensmodell werden Ausführungsstrategie und konkrete Ausführungsparameter weitgehend entkoppelt. Bei Produkt- oder Umgebungsvariationen müssen daher primär die relevanten Objektattribute im Wissensmodell angepasst werden, während die Bewegungsstrategie und der grundlegende Parametrierprozess unverändert bleiben. In den betrachteten Testszenarien konnten Variationen, etwa in der Pose oder Zugänglichkeit der Batterien, durch Anpassungen in der Initialisierungsdatei des Wissensmodells umgesetzt werden. Das manuelle Eintragen neuer Attribute (z. B. Posen) ist jedoch nicht intuitiv und damit potenziell aufwendig. Ein Ansatz für zukünftige Arbeiten besteht darin, diesen Prozess zu vereinfachen.

Zusätzliche Flexibilität ergibt sich durch das konfigurierbare Suchen von passenden Greifposen. Sollte eine Greifpose nicht erreicht werden können, wird durch den Wiederholungs-Dekorator im Pick-and-Place-Skill eine andere Greifpose benutzt. Somit kann auf unerwartete Variationen in der Umgebung reagiert werden, die bestimmte Greifposen nicht zugänglich machen.

Eingeschränkt wird die Flexibilität aktuell dadurch, dass die Skills Greifposen inklusive Richtungen erwarten, welche für die aktuelle Prozessumgebung funktionieren. Eine Reduktion dieser prozessabhängigen Parameter würde die Flexibilität weiter erhöhen.

Die Wiederverwendbarkeit der entwickelten Skills wird durch die Entkopplung von konkreter Hardware und Skill-Logik unterstützt. Hardwareabhängige Bewegungsprimitive werden im Wissensmodell in Form von Ressourcen-Fähigkeiten und den zugehörigen Action-Server-IDs referenziert. Damit können die Skills prinzipiell mit unterschiedlicher Hardware verwendet werden, sofern die referenzierten Ressourcen die benötigten Primitive bereitstellen und diese die gleiche jeweils erwarteten Parameter erwarten. Für Entwickler ergibt sich daraus insbesondere die Notwendigkeit, die Schnittstellen der Primitiven zu standardisieren. Beispielsweise dass eine PTP-Bewegung grundsätzlich über eine Zielpose sowie Geschwindigkeits- und Beschleunigungsfaktoren beschrieben werden soll. Bei einem Roboter, der nicht die Fähigkeit hat seine Orientierung zu verändern (s. SCARA-Roboter), ist der Zielposen-Parameter nicht zwangsweise zu erwarten.

Auch neue Umgebungen mit anderen Produkten können mit dem entwickelten System grundsätzlich abgebildet werden, da hierfür, analog zur Flexibilität, vor allem das Wissensmodell angepasst werden muss.

6.4.4 Z-4 – Integration in die Softwarearchitektur

Ein weiteres Ziel dieser Arbeit bestand darin, die entwickelten Skills in das bestehende Framework zu integrieren. Dazu wurde konsequent auf den vorhandenen Skill-Basisklassen aufgebaut, sodass die Ausführung der Composite Skills (insbesondere der Verhaltensbäume) in den äußeren Skill-Zustandsautomaten eingebettet ist. Ebenso wurde die Weitergabe von Zustandswechseln an die im Verhaltensbaum ausgeführten Sub-Skills umgesetzt.

Die erfolgreiche Integration wurde in den Testszenarien praktisch nachgewiesen. Die Laufzeitinteraktion über `Hold/Unhold` funktionierte wie vorgesehen. Die Implementierten Skills konnten also folglich erfolgreich in die bestehende Architektur integriert werden.

6.5 Limitationen und Ausblick

Aus der Evaluation der Anforderungen und der Diskussion der Ziele werden im Folgenden Potenziale zusammengefasst, um das System im Hinblick auf Anforderungen und Zielerreichung weiterzuentwickeln.

6.5.1 Pick-and-Place-Skill

Die Flexibilität des Pick-and-Place-Skills wird aktuell dadurch eingeschränkt, dass prozessabhängige Greifposen als Parameter benötigt werden.

Die Testszenarien deuten jedoch an, dass für den Greifvorgang allein nicht zwingend die im konkreten Prozess gültige Greifposenmenge bekannt sein muss. Stattdessen könnte es ausreichen, die Menge der im Endzustand erreichbaren Greifposen zu kennen. Die Teilmenge, die zusätzlich für den Pick-Vorgang geeignet ist, kann der Skill zur Laufzeit durch die Kollisionsprüfung von MoveIt 2 selbst bestimmen. Falls in zukünftigen Implementierungen eine durchgängige Kollisionsprüfung über den gesamten Ablauf möglich ist, könnte perspektivisch sogar eine prozessunabhängige Greifposenmenge genügen. Konflikte zwischen dem Pick- und dem Place-Prozess könnten dann über Strategien wie Umgreifen aufgelöst werden. Hub- und Annäherungsrichtungen müssten mit der aktuellen Implementierung trotzdem noch vorgegeben sein und dementsprechend als separater Parameter existieren.

Eine Erweiterung um eine eigenständige Bestimmung der nutzbaren Greifposen würde zwar die Flexibilität erhöhen und den Bedarf an prozessabhängigen Eingaben reduzieren, geht jedoch mit erhöhter Laufzeit einher. Entsprechend wäre zu untersuchen, wie der Suchraum durch Heuristiken oder Zusatzwissen sinnvoll eingeschränkt werden kann.

Wie bereits angedeutet, sollten zukünftige Implementierungen eine durchgängige Kollisionsprüfung unterstützen. Kernproblem ist hier, dass das Objekt aus einem Kontakzustand mit der Umgebung herausgelöst (Pick), bzw. in einen Kontaktzustand befördert werden muss (Place). Währenddessen ist das Objekt als am Greifer befestigtes Objekt in der MoveIt 2 Planungsszene eingetragen, wodurch eine Kollision erkannt wird. Eine Möglichkeit dieses Problem zu lösen, ist die Kollisionsmatrix von MoveIt 2 zu nutzen. In dieser können Objektpaarungen eingetragen werden, zwischen welchen Kollisionen zugelassen werden sollen. Dies würde zum Beispiel verhindern, dass der Robotergreifer beim Hochheben in das Regal aus den Testszenarien fahren kann. Hierfür müsste allerdings das Synchronisationsmodul innerhalb der Wissensmodell-Node die Information bekommen,

um welche Objektpaarung es genau geht. Folglich müsste hier ein erweiterter Informationsfluss stattfinden.

Weiterhin könnte der Bewegungsablauf des Pick-and-Place-Skills prozessabhängig effizienter gestaltet werden. In Abschnitt 4.4.1 wurde erläutert, wieso die lineare Hubrichtung benötigt wird. Allerdings gilt dies eben nur in spezifischen Situationen. Eine solche konkrete Richtung ist nicht in allen Situationen überhaupt notwendig. Wenn ein Objekt beispielsweise auf einem freien Tisch liegt, ist ein lineares Anheben nicht notwendig und es könnte sofort mit der ersten PTP-Bewegung des Place-Skill gestartet werden. Dies würde den gesamten Bewegungsablauf effizienter gestalten. Ein effizienterer Bewegungsablauf senkt die Taktzeit, wodurch der Skill auch in hochdynamischen Prozessen anwendbar wird. Dadurch wird also die Wiederverwendbarkeit gesteigert.

6.5.2 Dosier-Skill

Der Dosier Skill hat es nicht geschafft den angedachten Prozess praktikabel abzudecken. Hier besteht Potenzial zur Verbesserung des Primitven Skills, der den Roboter entlang der Bahn fahren lassen soll. Ansätze könnten hier sein, eine anderen Planer zu nutzen oder zu entwickeln, der zuverlässiger eine Liste aus Posen mit unterschiedlicher Orientierung abfahren kann. Das allein macht den Skill allerdings noch nicht praktikabel. Dafür muss zusätzlich untersucht werden, wie eine Zeitparametrierung bezogen auf den Endeffektor umgesetzt werden kann. Anschließend könnte der Skill real getestet werden, um konkretere Anforderungen an den Skill insbesondere hinsichtlich Dosier-Ressource zu definieren und ihn daraufhin zu verbessern.

6.5.3 Architektur

Hinsichtlich der generellen Architektur besteht Verbesserungspotenzial, insbesondere im Parametrierungsprozess. Aktuell erhalten die Skills ihre Parameter als String-Nachricht und wandeln diese anschließend in ein Dictionary um. Dieses Vorgehen ist durch das Action-Interface vorgegeben, das von allen Skills genutzt werden soll. Aus Sicht der Typsicherheit ist dies jedoch nachteilig, da jeder Skill zunächst für jeden Parameter prüfen muss, ob er in den erwarteten Datentyp konvertiert werden kann.

Wären die übergebenen Parameter ausschließlich String-Identifier, die lediglich als Referenz dienen, um Werte aus dem Umgebungsmodell abzurufen, wäre dieser Ansatz grundsätzlich akzeptabel. Dies würde jedoch bedeuten, dass sämtliche prozessbezogenen Parameter im Umgebungsmodell gespeichert werden müssten. Da das Umgebungsmodell primär den Zustand der Umgebung abbilden soll, ist dies konzeptionell ungeeignet und könnte zu Unübersichtlichkeit führen. Im umgesetzten System zeigt sich dies beispielsweise schon an den Greifposen-Listen. Würden nun noch weitere Skills ergänzt, würde die Anzahl an Objektattributen kontinuierlich steigen.

Zukünftige Arbeiten könnten daher untersuchen, wie Composite Skills auf eine skalierbare Art und Weise auf Prozessparameter Zugriff bekommen können.

Kapitel 7 Fazit

Diese Arbeit hat untersucht, wie typische Montageprozesse durch skillbasierte, parametrisierbare Roboter-Skills so abgebildet werden können, dass Anwendungen aufgabenorientiert und mit geringerem Programmieraufwand konfigurierbar sind. Vor dem Hintergrund zunehmender Unsicherheiten in der Produktion und eines Mangels an qualifiziertem Personal wurde damit adressiert, wie sich Flexibilität und Wiederverwendbarkeit roboterbasierter Lösungen verbessern lassen.

Die Ergebnisse zeigen, dass der skillbasierte Ansatz auch für typische Montageprozesse funktioniert. Für das Weitergeben nach *VDI-Richtlinie 2860* (1990) wurde ein allgemeiner Bewegungsablauf konzipiert, der im Rahmen der durchgeführten Tests erfolgreich verifiziert werden konnte. Auch für den betrachteten Dosierprozess konnte ein grundsätzlicher Funktionsnachweis erbracht werden, allerdings besteht hier noch Verbesserungspotenzial hinsichtlich des implementierten Primitive-Skills zum Abfahren der Dosierbahn. Das Abfahren einer Trajektorie mit variabler Orientierung konnte nicht zuverlässig ermöglicht werden, und es muss weiter untersucht werden, wie eine konstante Endeffektorgeschwindigkeit während des Prozesses umgesetzt werden kann. Die implementierten Skills wurden über Verhaltensbäume modelliert, wodurch sich mit vergleichsweise geringem Aufwand komplexe Verhaltensweisen umsetzen ließen.

Für alle implementierten Skills konnte außerdem gezeigt werden, dass damit eine aufgabenorientierte Programmierung eines Roboters möglich ist. Für die Parametrierung reicht es aus, abstrakte Parameter wie Bezüge zu bestimmten Ressourcen oder Objekten mitzugeben. Low-level Parameter werden nicht benötigt, da sich die Skills über das Wissensmodell selbst parametrieren. Im Zusammenspiel mit einem allgemein nutzbaren Wissensmodell führt dies zu einer deutlichen Erhöhung von Flexibilität und Wiederverwendbarkeit. Gleichzeitig stoßen die erreichten Ergebnisse noch an Grenzen, was ihre Nutzbarkeit für reale Prozesse angeht, da bislang nur der Bewegungsablauf der Skills evaluiert wurde, ohne wirkende Kräfte zu berücksichtigen. Für eine finale Validierung sollte daher ein empirischer Nachweis der Funktion unter realen Bedingungen erfolgen. Darüber hinaus ist die Repräsentation prozessbezogener Parameter im System noch nicht durchgängig konsistent, weil je nach Skill unterschiedliche Mechanismen verwendet werden. Das ist in den gezeigten Beispielen praktikabel, wird aber bei wachsender Anzahl an Skills schnell unübersichtlich.

In Summe zeigt die Arbeit, dass skillbasierte Programmierung in Kombination mit einem Wissensmodell eine aufgabenorientierte, flexible und wiederverwendbare Umsetzung standardisierter Montageprozesse ermöglichen kann. Als nächster Schritt ist es naheliegend, weitere Skills zu implementieren und damit auch ein breiteres Spektrum an Montageoperationen abzudecken. Entscheidend ist außerdem der Nachweis, dass sich das Konzept mit anderen Steuerungsformen erweitern lässt, etwa mit kraftgeregelter Steuerung für anspruchsvollere Fügevorgänge.

Anhang A Anhangname

Tabelle A.1: Parameter der Handhabungs Basis-Klasse

Name	Beschreibung	Bedingung	Datentyp	Einheit
object	Objekt-ID	Erforderlich	`str`	–
resource	Ressourcenkombination-ID	Erforderlich	`str`	–
speed	Geschwindigkeitsfaktor	Optional	`float`	–
acceleration	Beschleunigungsfaktor	Optional	`float`	–

Tabelle A.2: Eingangsparameter des *Pick*-Sills

Name	Beschreibung	Bedingung	Datentyp	Einheit
safety_factor	Sicherheitsfaktor für die Greifkraftberechnung	Optional	`float`	–
pick_frame	Frame mit Greifpose	Optional	GraspFrame oder ID	–

Tabelle A.3: Eingangsparameter des `Place`-Skills

Name	Beschreibung	Bedingung	Datentyp	Einheit
`assembly`	Assembly-ID	Optional	`str`	–
`frame`	Frame/Frame-ID, enthält Zielpose	Optional	`Frame` / `str`	–
`pose`	Zielpose	Optional	`Pose`	–
`reference_frame`	Referenzframe, in dem `pose` angegeben ist.	Optional	`str`	–
`approach_vector`	Annäherungsrichtung	Optional	`Vector`	–
`approach_distance`	Länge der linearen Annäherung	Optional	`float`	m
`retreat_vector`	Rückzugsrichtung	Optional	`Vector`	–
`retreat_distance`	Länge der linearen Rückzugsbewegung	Optional	`float`	m
`place_frame`	Kapselt die oberen 4 Parameter innerhalb eines Objekts	Optional	–	m
`drop_height`	Zusätzliche Abwurf-Höhe	Optional	`float`	m

Tabelle A.4: Eingangsparameter des `Dispensing`-Skills

Name	Beschreibung	Bedingung	Datentyp	Einheit
`object`	Objekt-ID	Optional	`str`	–
`resource`	Frame/Frame-ID, enthält Zielpose	Optional	`Frame` / `str`	–
`flow_rate`	Durchflussrate	Optional	`float`	N/A
`file_path`	Pfad zur Datei	Optional	`str`	–
`path`	Liste aus Posen	Optional	`Pose`	–
`max_velocity_scaling_factor`	Faktor für maximal zu erreichende Gelenkgeschwindigkeit	Optional	`float`	–
`max_acceleration_scaling_factor`	Faktor für maximal zu erreichende Gelenkbeschleunigung	Optional	`Vector`	–

Literaturverzeichnis

Archibald, C., & Petriu, E., (1993). Skills-oriented robot programming. *Proceedings of the International Conference on Intelligent Autonomous Systems (IAS-3)*, 104–115.

Bill, M., Müller, D. C., Kraus, D. W., & Bieller, D. S., (2024). *World robotics 2024* (Techn. Ber.). International Federation of Robotics.

DIN 8593-0, (2003). DIN Deutsches Institut für Normung e. V.

DIN 8593-3, (2003). DIN Deutsches Institut für Normung e. V.

DIN 8593-6, (2003). DIN Deutsches Institut für Normung e. V.

DIN 8593-8, (2003). DIN Deutsches Institut für Normung e. V.

Geng, P., Heuss, L., Rauh, T., Müller, J., & Daub, R., (2023). Framework for the rapid development and deployment of customized industrial robotic applications (D. Herberger & M. Hübner, Hrsg.) [Publisher: Hannover : publish-Ing.]. *Conference on Production Systems and Logistics*. https://doi.org/10.15488/15305

Haun, M., (2013). *Handbuch Robotik: Programmieren und Einsatz intelligenter Roboter*. Springer Berlin Heidelberg. https://doi.org/10.1007/978-3-642-39858-2

Hertzberg, J., Lingemann, K., & Nüchter, A., (2012). *Mobile Roboter: Eine Einführung aus Sicht der Informatik*. Springer Berlin Heidelberg. https://doi.org/10.1007/978-3-642-01726-1

Heuss, L., Blank, A., Dengler, S., Zikeli, G. L., Reinhart, G., & Franke, J., (2019). Modular robot software framework for the intelligent and flexible composition of its skills [Series Title: IFIP Advances in Information and Communication Technology]. In F. Ameri, K. E. Stecke, G. Von Cieminski & D. Kiritsis (Hrsg.), *Advances in production management systems. production management for the factory of the future* (S. 248–256, Bd. 566). Springer International Publishing. https://doi.org/10.1007/978-3-030-30000-5_32

Heuss, L., Gebauer, D., & Reinhart, G., (2024). Concept for the automated adaption of abstract planning domains for specific application cases in skills-based industrial robotics. *Journal of Intelligent Manufacturing*, *35*(8), 4233–4258. https://doi.org/10.1007/s10845-023-02211-3

Heuss, L., Gonnermann, C., & Reinhart, G., (2022a). Capability-based Frameworks for Industrial Robot Skills: a Survey. *The International Journal of Advanced Manufacturing Technology*, (9), 6269–6285. https://doi.org/10.1109/CASE49997.2022.9926648

Heuss, L., Gonnermann, C., & Reinhart, G., (2022b). An extendable framework for intelligent and easily configurable skills-based industrial robot applications. *The International Journal of Advanced Manufacturing Technology*, *120*(9), 6269–6285. https://doi.org/10.1007/s00170-022-09071-w

ISO 6983-1:2009 [Automation systems and integration. Numerical control of machines. Program format and definitions of address words. Part 1. Data format for positioning, line motion and contouring control systems], (2009). International Organization for Standardization.

ISO 8373:2021: Robotics — Vocabulary, (2021, November). International Organization for Standardization.

ISO 9787:2013: Robots and robotic devices — Coordinate systems and motion nomenclatures, (2013). International Organization for Standardization.

JAKA Robotics, (2024, 16. Mai). *Graphic Programming of JAKA Cobot*. Verfügbar 30. Juli 2025 unter https://www.youtube.com/watch?v=xfUGhhRpumE

Järvenpää, E., Siltala, N., Hylli, O., & Lanz, M., (2019). The development of an ontology for describing the capabilities of manufacturing resources [Publisher: Springer Science and Business Media LLC]. *Journal of Intelligent Manufacturing*, *30*(2), 959–978. https://doi.org/10.1007/s10845-018-1427-6

Jones, S. W., (2020, 24. März). *Onboard Evolution of Human-Understandable Behaviour Trees for Robot Swarms* [Doctoral thesis]. University of Bristol und University of the West of England. Verfügbar 26. Dezember 2025 unter https://hdl.handle.net/1983/aae1655d-a19f-43ca-8fd4-5007f5c070d5

Kocher, A., Hildebrandt, C., Vieira Da Silva, L. M., & Fay, A., (2020). A formal capability and skill model for use in plug and produce scenarios. *2020 25th IEEE International Conference on Emerging Technologies and Factory Automation (ETFA)*, 1663–1670. https://doi.org/10.1109/etfa46521.2020.9211874

Kurscheid, S., & Daub, R., (2024). Automated Robot Skill Parameterization Based On Task Specifications. *2024 IEEE 29th International Conference on Emerging Technologies and Factory Automation (ETFA)*, 01–04. https://doi.org/10.1109/ETFA61755.2024.10711102

Latif, K., Adam, A., Yusof, Y., & Kadir, A. Z. A., (2021). A review of g code, STEP, STEP-NC, and open architecture control technologies based embedded CNC systems. *The International Journal of Advanced Manufacturing Technology*, *114*(9), 2549–2566. https://doi.org/10.1007/s00170-021-06741-z

Lotter, B., & Wiendahl, H.-P. (Hrsg.), (2012). *Montage in der industriellen Produktion: Ein Handbuch für die Praxis*. Springer Berlin Heidelberg. https://doi.org/10.1007/978-3-642-29061-9

Macenski, S., Foote, T., Gerkey, B., Lalancette, C., & Woodall, W., (2022). Robot operating system 2: Design, architecture, and uses in the wild. *Science Robotics*, *7*(66), eabm6074. https://doi.org/10.1126/scirobotics.abm6074

Malakuti, S., Bock, J., Weser, M., Venet, P., Zimmermann, P., Wiegand, M., Grothoff, J., Wagner, C., & Bayha, A., (2018). Challenges in Skill-based Engineering of Industrial Automation Systems. *2018 IEEE 23rd International Conference on Emerging Technologies and Factory Automation (ETFA)*, *1*, 67–74. https://doi.org/10.1109/ETFA.2018.8502635

Mayr, M., Rovida, F., & Krueger, V., (2023, 29. Juni). SkiROS2: A skill-based robot control platform for ROS. https://doi.org/10.48550/arXiv.2306.17030

Montage- und Handhabungstechnik; Handhabungsfunktionen, Handhabungseinrichtungen; Begriffe, Definitionen, Symbole, (1990). VDI Verein Deutscher Ingenieure e.V.

MoveIt 2 Documentation, (2025). PickNik Robotics. Verfügbar 22. Dezember 2025 unter https://moveit.picknik.ai/main/index.html

The Open Motion Planning Library, (n. d.). Verfügbar 29. Dezember 2025 unter https://ompl.kavrakilab.org/

Owen-Hill, A., (2019, 21. Februar). *This simple programming trick avoids robot collisions* [RoboDK blog]. Verfügbar 16. November 2025 unter https://robodk.com/blog/programming-tips-collisions/

Pedersen, M. R., Nalpantidis, L., Andersen, R. S., Schou, C., Bøgh, S., Krüger, V., & Madsen, O., (2016). Robot skills for manufacturing: From concept to industrial deployment. *Robotics and Computer-Integrated Manufacturing*, *37*, 282–291. https://doi.org/10.1016/j.rcim.2015.04.002

Pott, A., & Dietz, T., (2019). *Industrielle Robotersysteme: Entscheiderwissen für die Planung und Umsetzung wirtschaftlicher Roboterlösungen*. Springer Fachmedien Wiesbaden. https://doi.org/10.1007/978-3-658-25345-5

Pydantic: Data validation using python type hints (Version 2.12.5), (2025, 26. November). Verfügbar 17. November 2025 unter https://docs.pydantic.dev/latest/

Romero, D., Taisch, M., Acerbi, F., Andersen, A.-L., Braun, G., Bressanelli, G., Chari, A., Mendes, G. H. d. S., Ebel, M., Frank, A., Gonzales, C., Hankammer, S., Khan, M., Kyriakoulis, N., Lenz, J., Marchetti, S., May, G., Medini, K., Müller, J., ... Wuest, T., (2024, Oktober). *The 2024 World Manufacturing Report* (Report). World Manufacturing Foundation.

ROSActions, (n. d.). Verfügbar 1. Dezember 2025 unter https://design.ros2.org/articles/actions.html?utm_source=chatgpt.com

Siciliano, B., & Khatib, O., (2016). *Springer Handbook of Robotics*. Springer International Publishing AG. http://ebookcentral.proquest.com/lib/munchentech/detail.action?docID=4614797

Sorensen, L. C., Mathiesen, S., Waspe, R., & Schlette, C., (2020). Towards digital twins for industrial assembly - improving robot solutions by intuitive user guidance and robot programming. *2020 25th IEEE International Conference on Emerging Technologies and Factory Automation (ETFA)*, 1480–1484. https://doi.org/10.1109/ETFA46521.2020.9212072

Stonier, D., (n. d.). *py_trees Documentation, Introduction* (2.4.0). Verfügbar 26. Dezember 2025 unter https://py-trees.readthedocs.io/en/devel/introduction.html

Van De Ginste, L., De Cock, A., Van Alboom, A., Huysentruyt, S., Aghezzaf, E.-H., & Cottyn, J., (2023). A formal skill model to enable reconfigurable assembly systems [Publisher: Informa UK Limited]. *International Journal of Production Research*, *61*(19), 6451–6466. https://doi.org/10.1080/00207543.2022.2128922

Van De Ginste, L., De Cock, A., Van Alboom, A., Singh, Y., Aghezzaf, E.-H., & Cottyn, J., (2021). A formal skill model facilitating the design and operation of flexible assembly workstations [ISSN: 1868-4238, 1868-422X]. In *IFIP advances in information and communication technology* (S. 108–116). Springer International Publishing. https://doi.org/10.1007/978-3-030-85914-5_12

Veit, H., & Gunther, R., (2018). Skills in Assembly. https://mediatum.ub.tum.de/1428286

Volkmann, M., Legler, T., Wagner, A., & Ruskowski, M., (2020). A cad feature-based manufacturing approach with opc ua skills. *Procedia Manufacturing*, *51*, 416–423.

Weber, W., (2019). *Industrieroboter: Methoden der Steuerung und Regelung* (4., aktualisierte Auflage). Hanser. https://doi.org/10.3139/9783446460607

Wizard Easy Programming [Robotics], (n. d.). Verfügbar 30. Juli 2025 unter https://new.abb.com/products/robotics/de/software-digitale-loesungen/applikationssoftware/wizard

Zhu, J., Gienger, M., & Kober, J., (2022). Learning task-parameterized skills from few demonstrations. *IEEE Robotics and Automation Letters*, *7*(2), 4063–4070.

Erklärung

Ich versichere hiermit, dass ich die von mir eingereichte Arbeit selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt habe.



Garching, 29. Dezember 2025 (Unterschrift)